

삼성청년 SW·AI아카데미

2. 선형 회귀 모델

2. 선형 회귀 모델

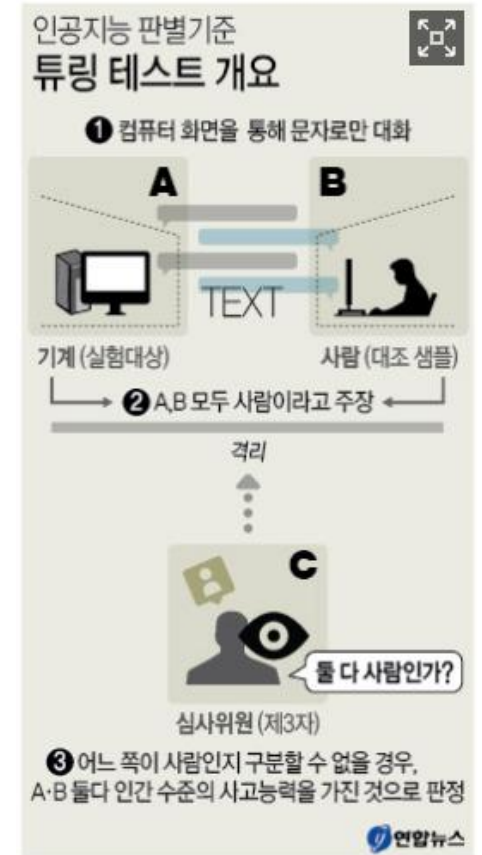
• INDEX

- 머신 러닝
- 선형 회귀
 - 정규 방정식
 - 경사 하강법
- 로지스틱 회귀
- 데이터 전처리

머신러닝

✓ AI

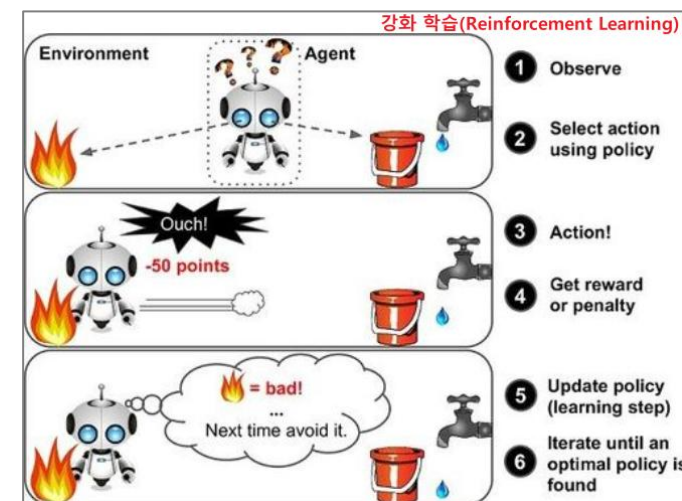
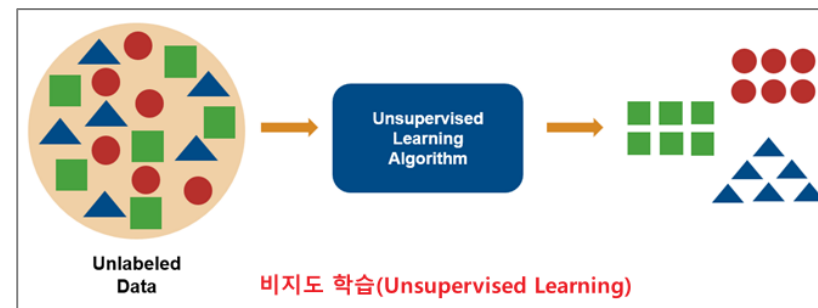
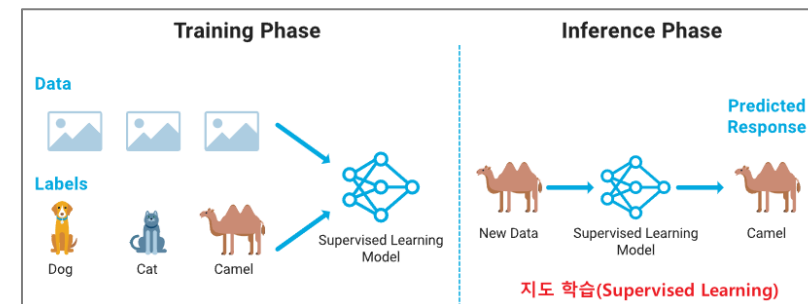
- Artificial Intelligence
- 인간의 학습, 추론, 지각 능력 등을 컴퓨터를 통해 구현하는 가장 포괄적인 분야
- AI의 최종 목표는 기계가 사람처럼 보고, 듣고, 말하고, 생각하게 만드는 것
- AI를 구현하는 두 가지 접근법
 - 규칙 기반
 - 전문가의 지식과 규칙을 사람이 직접 컴퓨터에 코드로 입력하는 방식
 - 한계: 세상의 모든 규칙을 만들 수 없고, 예외 상황에 대처하기 어려움
 - **학습 기반(머신러닝)**
 - 컴퓨터에게 데이터를 주고, 그 데이터 안에서 규칙(패턴)을 스스로 배우게 하는 방식
 - **현대 AI의 핵심 동력이며, 우리가 배울 분야**



이재운 기자 / 20140609
@yonhap_graphics(트위터)

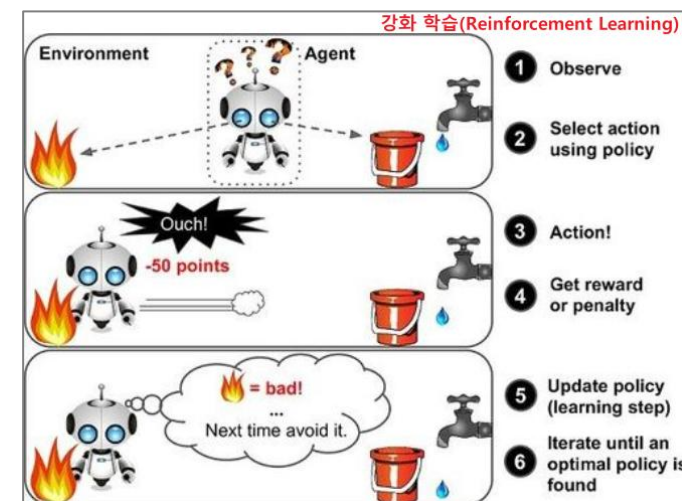
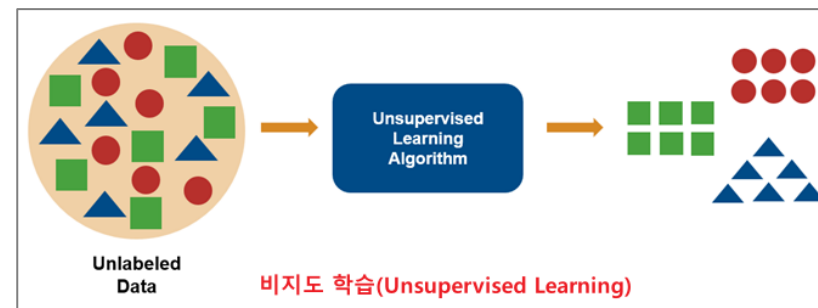
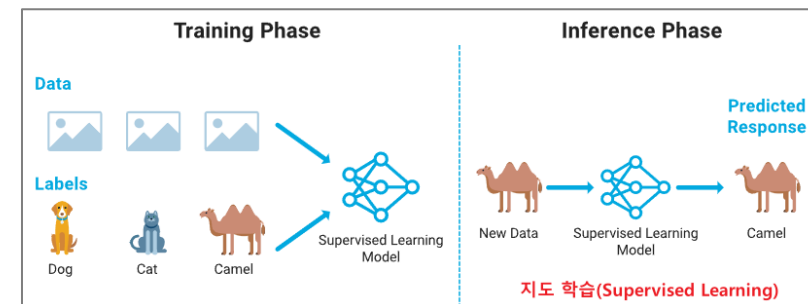
✓ 머신러닝

- 컴퓨터에게 데이터를 주고, 그 데이터 안에서 규칙(패턴)을 스스로 학습하게 하는 방식
- 학습 종류
 - 지도 학습(Supervised Learning)
 - 정답지가 있는 데이터를 학습시키는 방법
 - 의료 진단, 스팸 필터링, 주가/부동산 가격 예측 등에 활용
 - 비지도 학습(Unsupervised Learning)
 - 정답이 없는 데이터의 숨겨진 구조나 규칙을 찾는 방법
 - 추천 시스템, 이상 거래 탐지, 고객 세분화 등에 활용
 - 강화 학습(reinforcement Learning)
 - '상' 과 '벌' 을 통해서 최적의 행동 방식을 학습하는 방법 (알파고)
 - 로봇 제어, 자율 주행, 게임 AI 등에 활용
- 일반적으로 비지도 -> 지도 -> 강화 학습으로 복합적인 학습으로 결과물 생성



✓ 머신러닝

- 컴퓨터에게 데이터를 주고, 그 데이터 안에서 규칙(패턴)을 스스로 학습하게 하는 방식
- 학습 종류
 - 지도 학습(Supervised Learning)
 - 정답지가 있는 데이터를 학습시키는 방법
 - 의료 진단, 스팸 필터링, 주가/부동산 가격 예측 등에 활용
 - 비지도 학습(Unsupervised Learning)
 - 정답이 없는 데이터의 숨겨진 구조나 규칙을 찾는 방법
 - 추천 시스템, 이상 거래 탐지, 고객 세분화 등에 활용
 - 강화 학습(reinforcement Learning)
 - '상' 과 '벌' 을 통해서 최적의 행동 방식을 학습하는 방법 (알파고)
 - 로봇 제어, 자율 주행, 게임 AI 등에 활용
- 일반적으로 비지도 -> 지도 -> 강화 학습으로 복합적인 학습으로 결과물 생성



✓ 지도 학습(Supervised Learning)

- 정답(레이블, Label)이 있는 데이터(피쳐, Feature)를 학습시키는 방법
- 우리가 맞춰야 할 ‘정답’에 따라 두 가지의 큰 유형으로 나뉨
- 지도 학습 유형
 - 회귀(Regression)
 - 연속적인 숫자 값을 예측하는 문제
 - Feature와 Label을 기반으로 학습한 모델이 “새로운 Feature”의 Label을 예측하는 것
 - ex) 공부한 시간에 따른 시험 점수 예측하기
 - 분류(Classification)
 - 주어진 Feature가 어떤 그룹(카테고리)에 속하는 지 예측하는 문제
 - ex) 메일의 내용을 보고 ‘스팸’ 인지, ‘정상’인지 판단하기

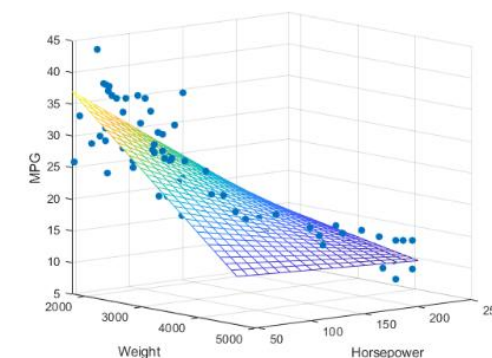
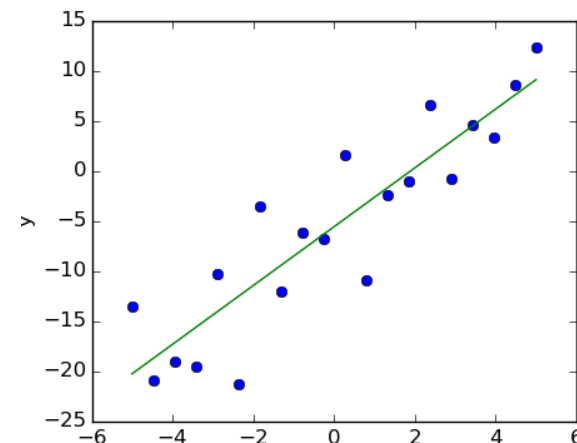
✓ 지도 학습(Supervised Learning)

- 정답(레이블, Label)이 있는 데이터(피쳐, Feature)를 학습시키는 방법
- 우리가 맞춰야 할 ‘정답’에 따라 두 가지의 큰 유형으로 나뉨
- 지도 학습 유형
 - 회귀(Regression) => 회귀 문제를 푸는 가장 기본적이면서 강력한 도구가 “선형 회귀”
 - 연속적인 숫자 값을 예측하는 문제
 - Feature와 Label을 기반으로 학습한 모델이 “새로운 Feature”의 Label을 예측하는 것
 - ex) 공부한 시간에 따른 시험 점수 예측하기
 - 분류(Classification)
 - 주어진 Feature가 어떤 그룹(카테고리)에 속하는 지 예측하는 문제
 - ex) 메일의 내용을 보고 ‘스팸’ 인지, ‘정상’인지 판단하기

선형 회귀

✓ 선형 회귀(linear regression)

- 예측을 “**하나의 직선**”으로 하는 가장 간단하고 직관적인 방법 (다차원일 경우에는 평면/초평면)
- (단순 선형 회귀) 하나의 특성(Feature)만 있을 때
 - 공부한 시간(x)에 따른 시험 점수(y) 예측
 - $y = wx + b$
 - w** (가중치, Weight): 직선의 기울기. 공부 시간이 1시간 늘 때 점수가 몇 점 오르는지를 나타냄
 - b** (편향, Bias): y절편. 공부를 하나도 안 했을 때의 기본 점수를 의미
- (다중 선형 회귀) 여러 특성이 있을 때
 - 캐럿(x_1), 투명도(x_2), 깊이(x_3) 를 모두 고려한 다이아몬드 가격(y) 예측
 - $y = w_1x_1 + w_2x_2 + w_3x_3 + \dots + b$
 - 각각의 특성에 대한 최적의 **가중치**와 **편향**을 찾는 것이 목표

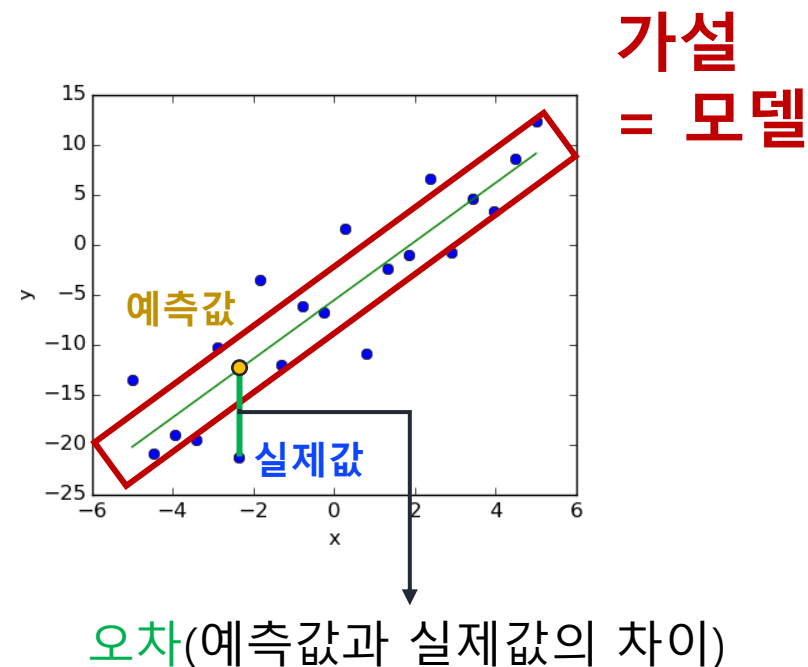


✓ 가설(Hypothesis)

- 우리가 예측에 사용할 “직선의 방정식(모델)”을 지칭하는 용어
- 그러면 어떤 가설(모델)이 좋은 가설일까?
 - 실제 데이터들과의 거리(오차)가 가장 작은 직선
 - 이 값은 어떻게 구할까?

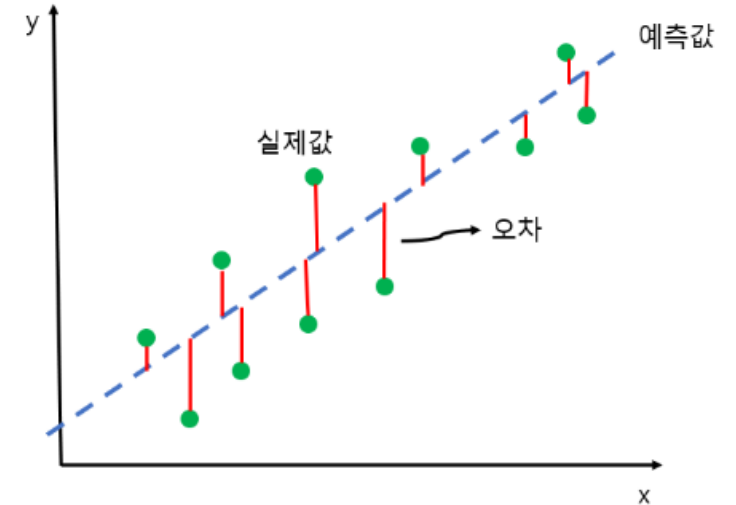
✓ 비용 함수(Cost Function)

- 모델(가설)이 얼마나 좋은 모델인 지 나타내는 점수 (like 시간복잡도)
- 엄연히 다르긴 하지만 손실 함수(loss function), 목적 함수(Objective Function) 라고도 함
- 비용 함수 종류
 - (회귀) 평균 제곱 오차(MSE, Mean Squared Error)
 - (회귀) 평균 절대 오차(MAE, Mean Absolute Error)
 - (분류) 교차 엔트로피
 - ...

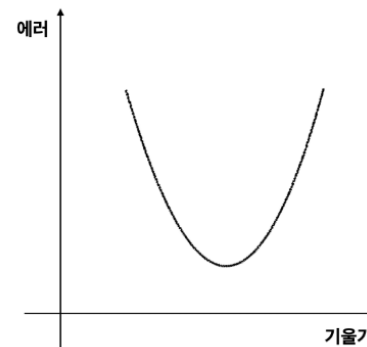


✓ 평균 제곱 오차(MSE, Mean Squared Error) (1/2)

- 모델의 예측값이 실제 정답과 얼마나 차이 나는지 측정하는 방법
- **점수가 낮을수록** 모델의 성능이 좋다는 의미
- 계산 방법
 1. 각 데이터들의 실제값과 예측값의 차이(오차)를 구한다.
 2. 각 데이터들의 오차를 모두 제곱하여 양수로 만들고 더하기
(제곱하는 이유: 양수 오차와 음수 오차의 상쇄 막기 + 큰 오차에 페널티 부여)
 3. 제곱 총합을 데이터 개수로 나누어 평균을 구함



$$MSE = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$



MSE 그래프

✓ 평균 제곱 오차(MSE, Mean Squared Error) (2/2)

• 예시) 시험 성적 예측 모델

- 학생A: 1시간 공부해서 60점, 학생B: 2시간 공부해서 70점, 학생C: 3시간 공부해서 90점
- 우리의 예측 모델: 예측 점수: $15 * (\text{공부 시간}) + 45$ (15 (가중치, w)와 45 (편향, b)는 임의의 초기값)
- 학생 A의 예측: $15 * 1 + 45 = 60$, 오차: 실제(60) - 예측(60) = 0
- 학생 B의 예측: $15 * 2 + 45 = 75$, 오차: 실제(70) - 예측(75) = (-5)
- 학생 C의 예측: $15 * 3 + 45 = 90$, 오차: 실제(90) - 예측(90) = 0
- 각 오차를 제곱하여 더한 뒤, 학생 수로 나누기: $(0^2 + (-5)^2 + 0^2) / \text{학생 수}(3) = 25 / 3 = 8.33\cdots$

❖ 결국 우리의 목표는 MSE 점수(비용)를 0에 가장 가깝게 만드는 최적의 모델(w 와 b)을 찾는 것!!!!

✓ 평균 제곱 오차(MSE, Mean Squared Error) (2/2)

• 예시) 시험 성적 예측 모델

- 학생 A: 1시간 공부해서 60점, 학생 B: 2시간 공부해서 70점, 학생 C: 3시간 공부해서 90점
- 우리의 예측 모델: 예측 점수: $15 * (\text{공부 시간}) + 45$ (15 (가중치, w)와 45 (편향, b)는 임의의 초기값)
- 학생 A의 예측: $15 * 1 + 45 = 60$, 오차: 실제(60) - 예측(60) = 0
- 학생 B의 예측: $15 * 2 + 45 = 75$, 오차: 실제(70) - 예측(75) = (-5)
- 학생 C의 예측: $15 * 3 + 45 = 90$, 오차: 실제(90) - 예측(90) = 0
- 각 오차를 제곱하여 더한 뒤, 학생 수로 나누기: $(0^2 + (-5)^2 + 0^2) / \text{학생 수}(3) = 25 / 3 = 8.33\cdots$

❖ 결국 우리의 목표는 **MSE 점수(비용)를 0에 가장 가깝게 만드는 최적의 모델(w 와 b)을 찾는 것!!!!**

=> 이 원리가 **최소제곱법**(MLS, Method of Least Squares)

✓ 최소 제곱법(MLS, Method of Least Squares)

- 예측 모델($y = wx + b$)에서 비용을 최소화하는 w (가중치)와 b (편향)을 찾는 원리
- 그러면 비용을 어떻게 최소화해서 최적의 모델을 찾을까?
- 최적의 w 와 b 를 찾는 방법

* 비용은 평균 제곱 오차(MSE)를 채택

$$MSE = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

1. 정규 방정식(Normal Equation)

- 복잡한 미분 방정식을 풀어서 한 번에 정답을 계산해내는 방법
- 만들어진 수학기공식을 이용해, 최적의 파라미터를 한 번에 계산

2. 경사 하강법(Gradient Descent)

- 최적의 파라미터를 점진적으로 찾아나가는 방법
- 대부분의 모델이 이 방식을 채택

• 최소 제곱법(MLS, Method of Least Squares)

• 예측 모델($y = wx + b$)에서 비용을 최소화하는 w (가중치)와 b (편향)을 찾는 원리

• 그러면 비용을 어떻게 최소화해서 최적의 모델을 찾을까?

• 최적의 w 와 b 를 찾는 방법

1. 정규 방정식(Normal Equation)

• 복잡한 미분 방정식을 풀어서 한 번에 정답을 계산해내는 방법

• 만들어진 수학기공식을 이용해, 최적의 파라미터를 한 번에 계산

2. 경사 하강법(Gradient Descent)

• 최적의 파라미터를 점진적으로 찾아나가는 방법

• 대부분의 모델이 이 방식을 채택

MSE 그래프

정규 방정식

✓ 정규 방정식 (1/2)

- 복잡한 미분 방정식을 풀어서 한 번에 정답을 계산해내는 방법
- 만들어진 수학기공식을 이용해, 최적의 파라미터를 한 번에 계산

$$\theta = (X^T X)^{-1} X^T y$$

- θ (세타): 찾으려는 최종 목표, 모든 가중치(w)와 편향(b)을 담고 있는 결과값
- X : 입력 데이터(feature) 행렬, y : 맞춰야 할 정답(레이블) 벡터
- 예시) 시험 성적 예측 모델
 - 학생A: 1시간 공부해서 60점, 학생B: 2시간 공부해서 70점, 학생C: 3시간 공부해서 90점

$$X = \begin{bmatrix} 1 & 1 \\ 1 & 2 \\ 1 & 3 \end{bmatrix} \quad y = \begin{bmatrix} 60 \\ 70 \\ 90 \end{bmatrix} \quad \theta = \begin{bmatrix} b \\ w \end{bmatrix} = \begin{bmatrix} 43.33 \\ 15 \end{bmatrix}$$

- 예측 점수 = $15 * (\text{공부 시간}) + 43.33$

✓ 정규 방정식 (2/2)

- 장점
 - 데이터의 특성(feature)가 적을 때, 빠르고 정확하게 해를 찾음
 - 추가 설정이 필요없음
- 단점
 - 데이터의 특성(feature)가 많으면 속도가 매우 느려짐
 - 역행렬이 존재하지 않으면 계산할 수 없음
 - 역행렬 계산 시 미세한 오차로 인해 계산 결과가 완전히 틀어질 수 있음

$$\theta = (X^{\overset{\text{전치}}{\boxed{T}}X)^{\overset{\text{역행렬}}{\boxed{-1}}}X^T y$$

➤ 이런 역행렬이 존재하지 않으면 계산할 수 없는 문제는 “특이값 분해(SVD)” 로 해결

❖ 특이값 분해(SVD)

- 역행렬이 없는 행렬을 역행렬이 있는 가장 유사한 행렬을 만들어내는 것
- 깊은내용이니 진행 X

경사 하강법

✓ 경사 하강법 (Gradient Descent) (1/2)

- 최적의 파라미터를 점진적으로 찾아 나가는 방법
- 비용 함수의 기울기가 0 이 되는 지점이 가장 비용이 낮은 지점
- 동작 과정
 1. 랜덤한 지점에서 시작해서 현재 지점의 “기울기”를 구한다.
 2. 해당 기울기만큼 움직인다.
(움직이는 보폭을 “**학습률(Learning Rate)**” 라고 함)
 3. 이동한 위치에서 다시 경사를 확인하고, 학습률만큼 이동한다.
 4. 위 과정을 계속 반복하면서, 가장 낮은 지점(기울기가 0)에 도달

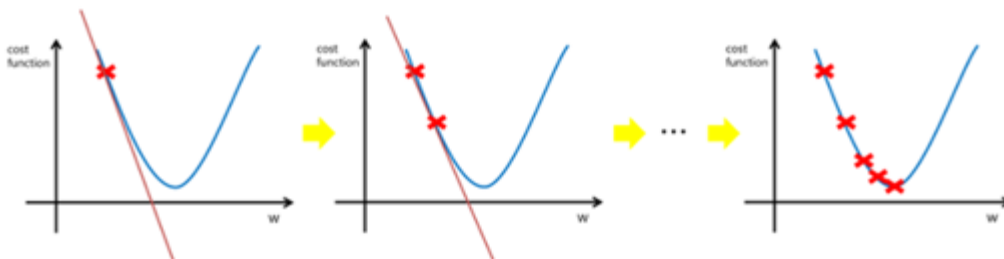
* 비용은 평균 제곱 오차(MSE)를 채택

[참고]기울기 구하는 방법

$$MSE = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

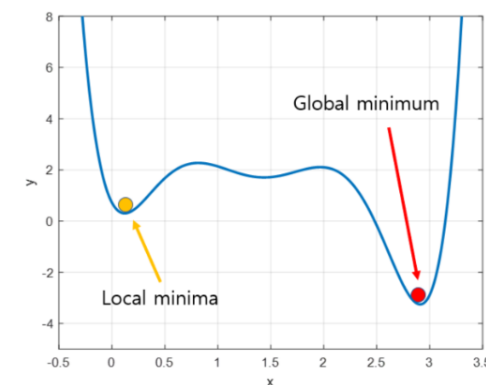
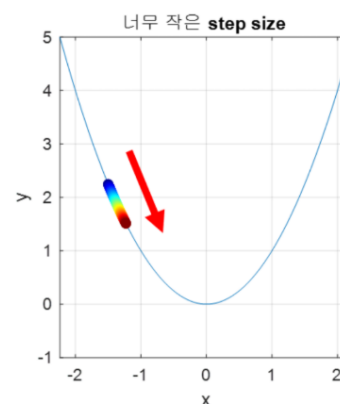
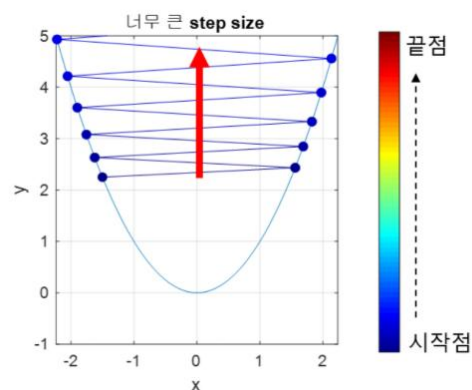
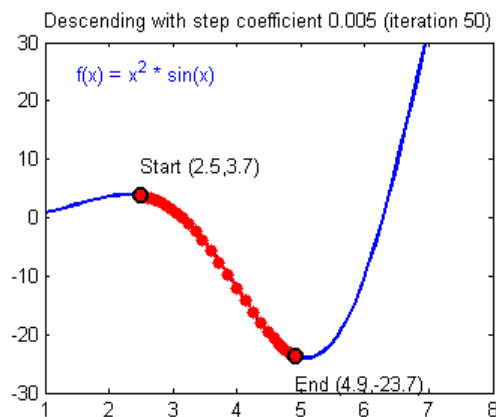


$$\nabla_{\theta} = \frac{2}{m} X_b^T (X_b \theta - y)$$



✓ 경사 하강법 (Gradient Descent) (2/2)

- 기울기만큼 이동(학습률)할 때, 반대 방향으로 이동함 (음수 기울기면, 양수 방향으로 이동해야 함)
- 학습률 설정
 - 학습률이 너무 크면, 보폭이 너무 커서 반대편으로 올라갈 수 있음 (발산, overshooting)
 - 학습률이 너무 작으면, 너무 오래 걸림
- 지역 극솟값(local minima) 문제: 시작 위치가 랜덤이기 때문에 Global minimum이 아닌 local minimum에 빠질 수 있음



✓ 경사 하강법 (Gradient Descent)에서의 용어/파라미터 (1/2)

- 기울기(gradient)
 - 전체 데이터가 만들어내는 “평균적인 오차”의 기울기
- 이터레이션(iterations)
 - 데이터의 일부 혹은 전체를 보고, 현재 위치의 경사를 계산하고, 한 걸음 이동하는 과정
- 에포크(Epoch)
 - 전체 데이터를 모두 한번 훑어보면, 1에포크
- 학습률(Learning Rate, alpha)
 - 한번에 얼마나 크게 움직일지를 결정하는 보폭

✓ 경사 하강법 (Gradient Descent)에서의 용어/파라미터 (2/2)

- 허용 오차(Tolerance, tol)
 - 학습 조기 종료 조건
 - 모델의 오차 감소량이 매우 작아지면, 학습해도 성능향상이 없겠다고 판단하여 학습을 멈춤
 - 너무 세세한 오차까지 학습해서 데이터의 노이즈까지 배워버리는 문제를 방지
- 배치 크기(batch)
 - 기울기를 구할 때, 전체 데이터 오차의 기울기를 한 번에 구하지 않음 => 너무 느림
 - 작은 묶음(배치)로 나눠서 처리
 - 전체 데이터 오차의 기울기를 한 번에 구하는 경우에는 “1 이터레이션” = “1 에포크”
- 기울기 누적 스텝(Gradient Accumulation Steps)
 - 실질적인 배치 크기를 늘리는 고급 기법
 - 매 배치마다 업데이트하는 것이 아닌, n 번의 배치 간격으로 누적한 기울기를 기반으로 이동

✓ 경사 하강법 (Gradient Descent) 학습 전략

1. 배치 경사 하강법(Batch Gradient Descent)

- 전체 데이터를 훑어보고, 기울기를 계산
- 1 epoch = 1 iteration
- 정확하지만, 데이터가 크면 매우 느림

2. 확률적 경사 하강법(SGD, Stochastic Gradient Descent)

- 하나의 데이터만 보고 바로 기울기를 계산
- 매우 빠르지만, 불안

3. 미니배치 경사 하강법(Mini-batch Gradient Descent)

- 적당한 수의 배치(batch)만 보고, 기울기를 계산
- 속도와 안정성을 모두 잡은 방법
- 배치 수는 정해진 답이 없으며, 실험적으로 구해야 함

로지스틱 회귀

✓ 지도 학습(Supervised Learning)

- 정답(레이블, Label)이 있는 데이터(피쳐, Feature)를 학습시키는 방법
- 우리가 맞춰야 할 ‘정답’에 따라 두 가지의 큰 유형으로 나뉨
- 지도 학습 유형
 - 회귀(Regression)
 - 연속적인 숫자 값을 예측하는 문제
 - Feature와 Label을 기반으로 학습한 모델이 “새로운 Feature”의 Label을 예측하는 것
 - ex) 공부한 시간에 따른 시험 점수 예측하기
 - **분류(Classification)** => 분류 문제를 푸는 가장 기본적이면서 강력한 도구가 “로지스틱 회귀”
 - 주어진 Feature가 어떤 그룹(카테고리)에 속하는 지 예측하는 문제
 - ex) 메일의 내용을 보고 ‘스팸’ 인지, ‘정상’인지 판단하기

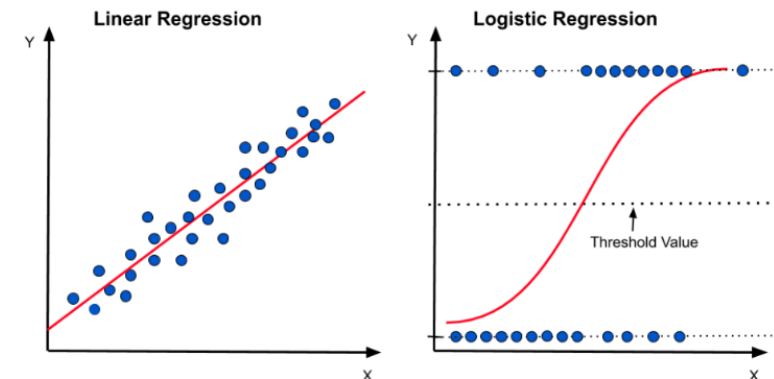
✓ 근데 그냥 선형 회귀로 분류 문제를 풀면 안되나요? 네, 안됩니다. (단호) (1/3)

1. 예측값이 범위를 벗어남

- 선형 회귀는 끊임없이 이어지는 직선으로 되어 있고, 어떠한 입력값이든 비례하는 값을 예측함
- 이는 분류 범위를 훨씬 벗어나는 예측값을 계산하게 됨
- 예를 들면, 환자가 암에 걸릴 확률이 150% or -230%는 의미가 없음

✓ 100% 이상은 100%, 0% 이하는 0% 식으로 결과만 자르면 안되나요?

- 임시 방편에 불과함
- 마치 0점, 1점 과녁에서 1점 방향으로 쏘았다고 1점을 주는 것과 같음



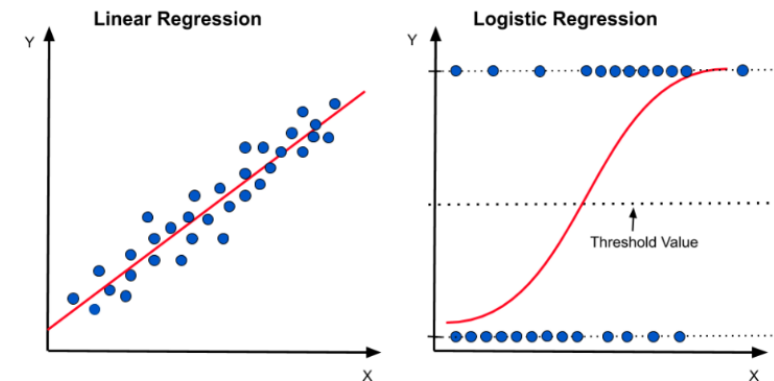
✓ 근데 그냥 선형 회귀로 분류 문제를 풀면 안되나요? 네, 안됩니다. (단호) (2/3)

2. 이상치(Outlier)에 매우 민감

- 선형 회귀는 모든 데이터와의 평균적인 거리를 최소화하는 것이 목적
- 이상치가 추가된다면, 이상치에 따라 직선의 기울기가 바뀌게 됨
- 이로 인해서 “분류 기준선(Decision Boundary)”가 완전히 망가지는 현상이 발생

✓ 회귀 문제에서도 이상치가 나오면 동일한 문제가 발생하는 거 아니가요?

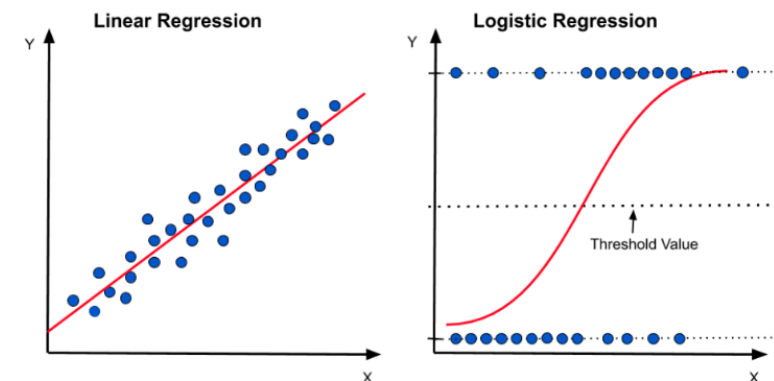
- 회귀에서는 단지 “하나의 예측을 크게 틀리게 만드는 것” 정도이지만,
- 분류에서는 “전체 규칙 자체를 바꿔버림”



✓ 근데 그냥 선형 회귀로 분류 문제를 풀면 안되나요? 네, 안됩니다. (단호) (3/3)

3. 다중 분류 문제에서 잘못된 관계를 학습

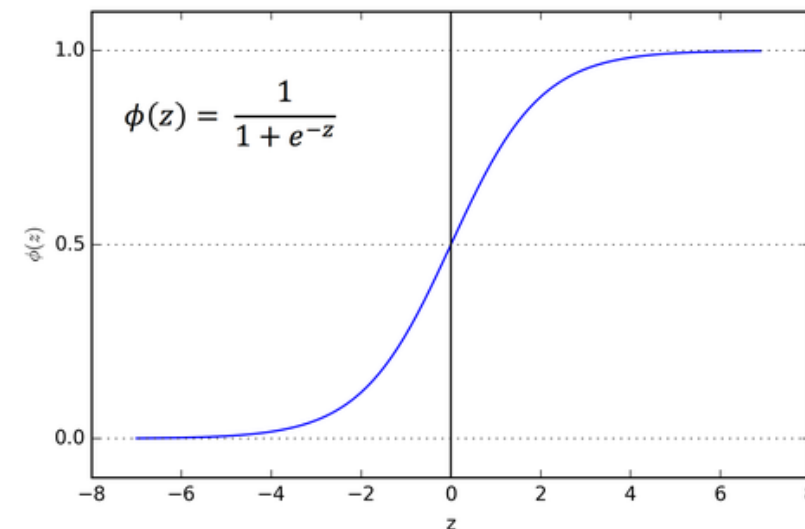
- 분류할 그룹이 3개 이상일 경우에 그룹들간에 수학적 관계가 있다고 착각함
- 예) 3개의 그룹을 분류("개", "고양이", "새")
 - 분류를 위해 개=1, 고양이를=2, 새=3으로 설정
 - "개(1)"와 "새(3)"의 중간은 "고양이(2)"와 같이 실제로는 존재하지 않는 거짓된 순서나 크기 관계를 학습하게 됨



- 그러면 위의 문제(1.예측값 오류, 2.이상치, 3.다중 분류 문제)를 어떻게 해결할까?
 - 선형 회귀의 예측값을 “0과 1사이의 값으로 바꿔서” 생각하자. (1번, 2번 문제 해결 가능)
 - “**시그모이드(Sigmoid)**” 함수의 등장!!

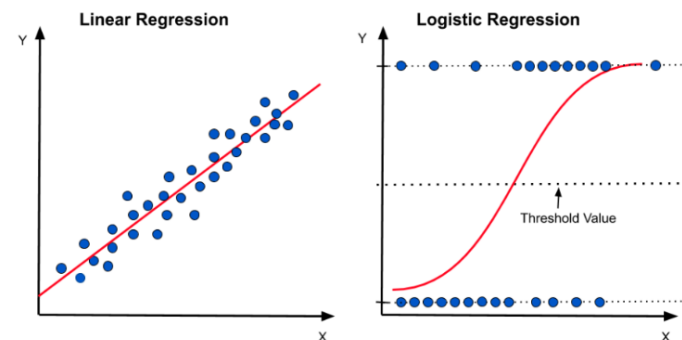
✓ 시그모이드(Sigmoid) 함수

- 어떤 숫자든 입력받아 그 값을 0과 1 사이의 값으로 압축해주는 S자 모양의 함수
- 음의 무한대부터 양의 무한대까지 어떤 숫자든 넣을 수 있고, 결과는 항상 0과 1 사이의 값으로 나옴
- 공식의 특징
 - z 가 아주 큰 양수이면, 분모가 0에 가까워져서 결과는 1에 가까워짐
 - z 가 아주 큰 음수이면, 분모가 무한대에 가까워져서 결과는 0에 가까워짐
- 한계:
 - 이후에 배울 역전파에서 기울기를 전달하는 과정에서 기울기가 없어지는, **기울기 소실(Vanishing Gradient)** 문제가 발생할 수 있음
 - ReLU 함수와 같은 다른 활성화 함수들이 더 널리 사용됨 (이후 학습 예정)



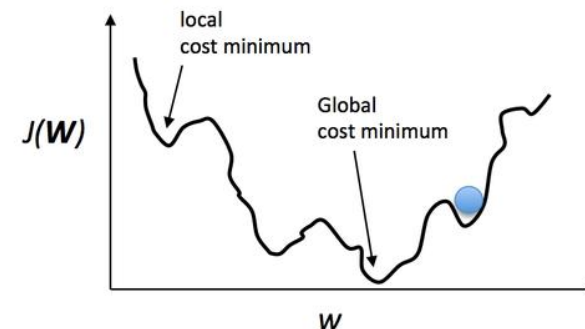
✓ 로지스틱 회귀(Logistic Regression)

- 선형 회귀 + **시그모이드 함수**
- 데이터를 특정 그룹으로 나누는 분류 방법(이진 분류)
- 기존 채점 기준인 MSE(평균 제곱 오차) 대신 “**이진 교차 엔트로피(BCE)**”를 사용



❖ 평균 제곱 오차를 사용하지 못하는 이유는?

1. 경사하강법이 잘 작동하려면 비용 함수 모양이 매끄러워야 하는데, 시그모이드는 울퉁불퉁한 모양을 만들어냄.
이는 수 많은 지역 최저점(Local minima)으로 학습에 어려움을 줌
2. 모델이 학습에 게을러지는데, 정답이 “1” 이고, 예측값이 “0.01” 이라고 하면 아주 큰 실수이지만, 정작 모델은 작은 예측값에 따라, 기울기도 0에 가까워지고, 그로 인해 움직이지 않는 “**기울기 소실 문제**”가 발생



✓ 이진 교차 엔트로피(BCE, Binary Cross-Entropy)

- 두 가지 중 하나의 답을 맞혀야 하는 이진 분류 문제를 위한 전용 손실 함수

$$L = -[y \log(\hat{p}) + (1 - y) \log(1 - \hat{p})]$$

- 정답이 1인 경우
 - 모델이 1에 가까운 확률을 예측할수록 0에 가까운 손실을 줌 (잘함)
 - 모델이 0에 가까운 확률을 예측할수록 무한대에 가까운 손실을 줌 (못함)
- 정답이 0인 경우
 - 모델이 0에 가까운 확률을 예측할수록 0에 가까운 손실을 줌 (잘함)
 - 모델이 1에 가까운 확률을 예측할수록 무한대에 가까운 손실을 줌 (못함)
- **결론:** 틀린 답에 대해, 모델이 얼마나 강하게 확신했느냐에 따라 벌점의 크기를 기하급수적으로 늘리는 것

✓ 로지스틱 회귀(Logistic Regression) 완성

- 데이터를 특정 그룹으로 나누는 분류 방법(이진 분류)
- 선형 회귀 + 시그모이드 함수 + **이진 교차 엔트로피(BCE)**
- 이후 학습은 선형 회귀와 마찬가지로 “**경사하강법**”을 사용하고, 이진 교차 엔트로피(BCE) 비용을 최소화하는 방향으로 w 와 b 를 계속 업데이트 함
- 그러면 아까 “시그모이드 함수”로도 해결 못 한 “**다중분류문제**”는 어떻게 해결할까?
 - 시그모이드 함수 대신 “**소프트맥스(Softmax)**” 를 활용

✓ 소프트맥스(Softmax) 함수

- 모델의 출력(점수)들을 받아서, 모든 클래스에 대한 확률의 총합이 반드시 1이 되도록 만들어주는 함수
- 즉, 전체를 하나의 확률 분포로 바꿈 + 지수 함수는 점수 차이를 극명하게 벌림
- 예를 들어 모델이 계산을 마친 후 점수가 [개: 2.0, 고양이: 1.0, 새: 0.1] 일 때,
 - 소프트맥스 함수의 출력: [개: 0.7, 고양이: 0.2, 새: 0.1] (총합=1.0)

$$p_j = \frac{e^{z_j}}{\sum_{k=1}^K e^{z_k}}$$

$$j = 1, 2, \dots, K$$

- 여러 선택지 중 단 하나의 정답을 고르는 “다중 클래스 분류”에 사용 됨

❖ 헷갈릴 수 있는 시그모이드(Sigmoid) 와의 차이

- 문제: 이 사진이 '고양이'일 확률은? (0.7) → "Yes"
- 문제: 이 사진이 '새'일 확률은? (0.1) → "No"
- 독립적인 출력을 하며, 모든 출력값의 합이 1이 된다는 보장이 없음
- 이진 분류에 적합

✓ 소프트맥스 회귀(Softmax Regression)

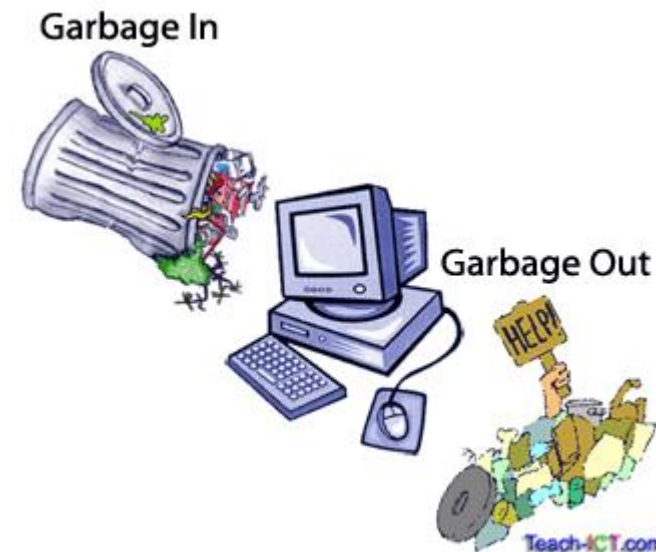
- 세 개 이상의 그룹(클래스)에서 하나를 예측(다중 분류)
- 선형 회귀 + **소프트맥스(Softmax) 함수** + **범주형 교차 엔트로피(CCE, Categorical Cross Entropy)**
- 범주형 교차 엔트로피 (별로 안중요함)
 - 손실은 실제 정답인 클래스에 대해 모델이 예측한 확률값에만 집중하여 계산
 - 실제 정답(y_i)은 정답 클래스에서만 1이고 나머지에서는 0

$$\text{Loss} = - \sum_{i=1}^K y_i \cdot \log(\hat{p}_i)$$

데이터 전처리

✓ 데이터 전처리

- 쓰레기를 넣으면, 쓰레기가 나온다! (Garbage in, Garbage Out)
- 모델이 잘 학습할 수 있도록 데이터를 깨끗하고 정돈된 상태로 바꾸는 모든 과정
- 컴퓨터는 유연하지 못하다..!
 1. **결측치**: 비어있는 값(NaN)은 수학 계산을 불가능하게 만들
 2. **문자열/범주형 데이터**: '사과', '바나나' 같은 문자열을 직접 계산할 수 없음
 3. **스케일링**: '키(cm)', '몸무게(kg)'처럼 단위와 범위가 전혀 다른 데이터가 섞이면, 컴퓨터는 단순히 "키"가 더 중요한 특성이라고 착각할 수 있음



➤ 위 문제를 해결해보자!

✓ 데이터 전처리 - 결측치

- 비어있는 값으로써, 데이터 전처리에서 일반적으로 가장 먼저 마주하는 문제
- 결측치 처리 방법1) 삭제 (Deletion)
 - 비어있는 데이터가 포함된 행이나 열을 그냥 삭제
 - 간단하고 빠르지만, 소중한 데이터를 잃어버릴 수 있음 (데이터가 적을 때는 치명적)
- 결측치 처리 방법2) 대치(Imputation)
 - 비어있는 값을 그럴듯한 값으로 채워넣기
 - 숫자형 데이터인 경우 (키, 몸무게, ... 등)
 - 평균
 - 중앙값
 - 범주형 데이터 (혈액형, 등급, ... 등)
 - 최빈값

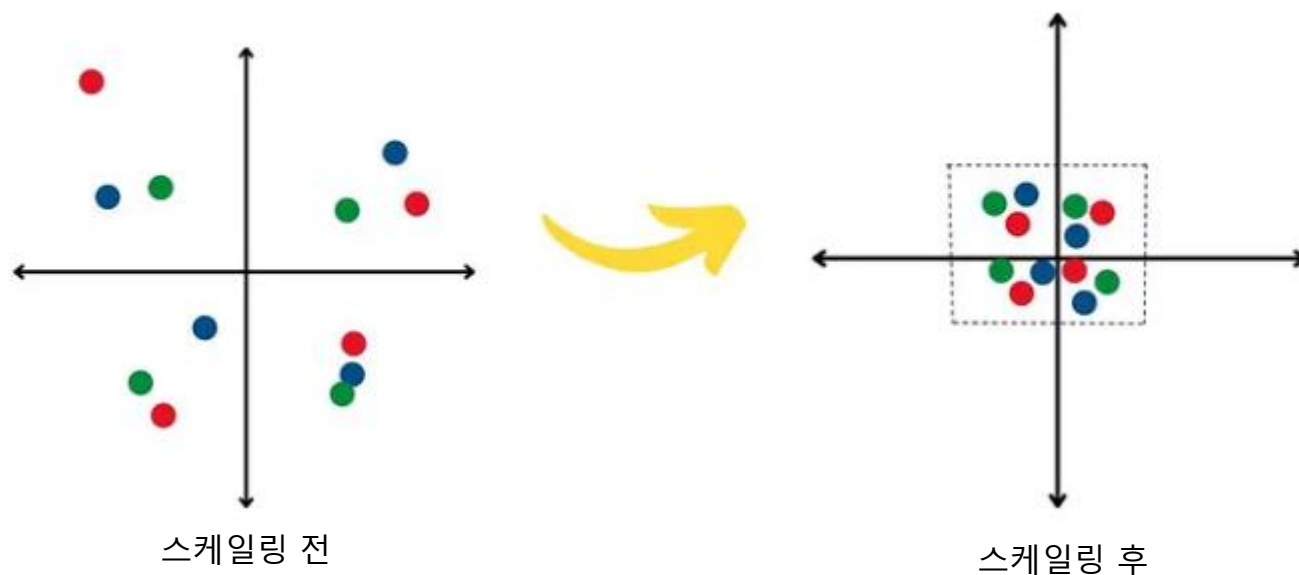
✓ 데이터 전처리 - 문자열/범주형 데이터

- 컴퓨터가 알아들을 수 있도록 문자 데이터를 숫자로 변환하는 과정
- 레이블 인코딩 / 원-핫 인코딩 / 순서형 인코딩 / 임베딩 등 다양한 방식으로 변환 (*이후에 학습 예정)



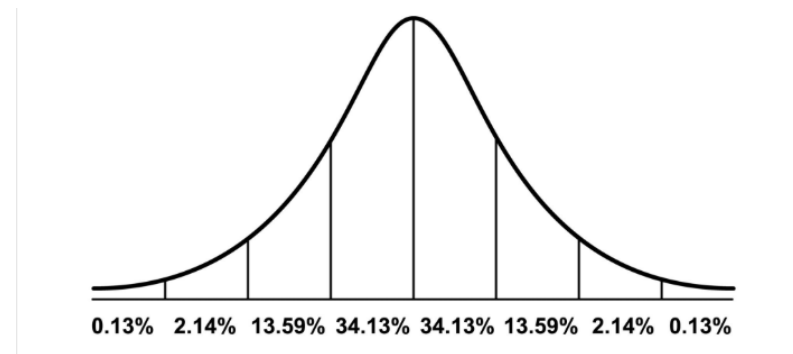
✓ 데이터 전처리 - 스케일링

- 각 특성의 단위를 맞춰서 모델이 공정하게 학습하도록 만드는 과정
- 마치 키(180cm)와 몸무게(70kg)를 비슷한 범위의 값으로 조정
- 대표적인 방법으로 **표준화**와 **정규화**가 있음



✓ 데이터 전처리 - 스케일링 - 표준화

- 모델에게 '나이'와 '연봉' 데이터를 그대로 주면 어떻게 될까?
 - 값의 범위가 훨씬 큰 '연봉'을 더 중요한 정보로 착각할 수 있음
- 제각각인 데이터의 단위를 공평하게 맞춰주는 작업
- 데이터의 평균을 0, 표준편차를 1로 변환
 - 평균 0 : 모든 데이터를 0을 기준으로 맞춤 (ex: 체중계 초기 무게를 0으로 설정)
 - 표준편차 1: 각 데이터가 평균으로부터 얼마나 떨어져 있는지(표준편차)를 기준으로 데이터의 스케일을 조절
- 특징
 - **이상치**에 상대적으로 덜 민감하고, 정규분포를 따를 때 효과적 => 이상치가 들어와도 분모인 표준편차도 커져서 상대적 거리가 어느 정도 유지 (***이상치**: 다른 값들에 비해 극단적으로 다른 값)
 - 모든 특성에 공평한 영향력을 부여



✓ 데이터 전처리 - 스케일링 - 정규화

- 흩어져 있는 모든 데이터의 값을 0과 1 사이의 범위로 변환
- 데이터의 범위를 0~1로 명확하게 제한하고 싶은 경우에 활용
 - ex) 이미지 처리에서 픽셀값은 보통 0~255로 표현되며, 이를 정규화하면 학습이 훨씬 쉬워짐
- 특징
 - 데이터의 최소/최대값을 명확히 알면 유용함
 - 이상치에 민감
- **일반적으로 표준화를 더 선호**

✓ 데이터 분리 및 파이프라인 구축은 이후에 진행

✓ [참고] 시각화 라이브러리

- Matplotlib
 - 가장 기본적이고 강력한 데이터 시각화 라이브러리
 - 높은 자유도를 가지고 있지만, 약간 투박해 보일 수 있음
 - numpy에 최적화된 편
- Seaborn
 - Matplotlib을 기반으로 한 시각화 라이브러리
 - 쉬운 문법과 멋진 디자인
 - Pandas에 특화
- 오늘 배울 실습에서 사용되는 Matplotlib, Seaborn 코드는 일단 실행하고, 자세한 내용은 내일 학습

3. 데이터 EDA

● INDEX

- 데이터 EDA
 - 데이터 시각화
- 모델 학습
- 성능 평가
- 성능 검증
- 비지도 학습
 - 군집화
 - 차원축소
- 참고

데이터 EDA

✓ 탐색적 데이터 분석(EDA, Exploratory Data Analysis)

- 데이터를 본격적으로 분석하기 전에 여러 각도에서 관찰하고 **이해**하는 과정
- 데이터에 대한 깊은 이해를 얻을 수 있으며, 모든 프로젝트의 첫 과정!
- 중요성
 1. 분석의 방향성 제시
 - 데이터의 특징을 파악하여 어떤 분석 기법을 적용하고, 어떤 모델을 구축할지 결정하는데 도움을 줌
 2. 모델의 성능 향상
 - 데이터의 문제점을 미리 파악하고 해결함으로써 모델의 정확성과 안정성을 높일 수 있음
 3. 새로운 인사이트 발견
 - 예상치 못한 데이터의 패턴이나 관계를 발견하여, 새로운 인사이트로 활용할 수 있음

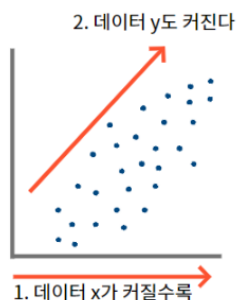
✓ 탐색적 데이터 분석(EDA) 진행 순서

❖ **주의!** 순서를 작성하긴 했지만, 이 과정은 항상 한 방향으로만 진행되는 것이 아닌, 각 단계를 오가면서 반복적으로 진행

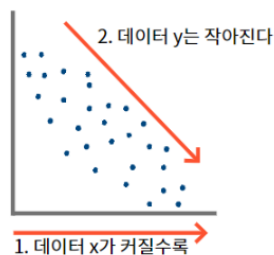
1. 데이터를 불러오고, 기본 정보를 확인 (타입, 결측치, 통계량 등)
2. 데이터 정제 및 전처리
 - 결측치, 이상치, 중복값을 식별하고 처리하고, 올바르지 않은 데이터 타입 수정
 - 분석에 사용되지 않을 변수(열) 제거
3. 개별 변수 분석
 - 변수의 분포와 특징을 이해
4. 변수 간 관계 분석
 - 변수들 사이의 관계, 패턴, 상관관계를 파악
5. 분석 결과 정리

✓ [참고] 상관관계

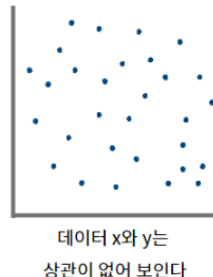
- 두 변수가 함께 변화하는 관련성을 의미
- 하나의 변수가 증가/감소할 때, 다른 변수가 어떤 일정한 관련성을 띄며 함께 변화하는지를 나타내는 지표
- 일반적으로 -1에서 1 사이의 값을 가지며, 이 값을 **상관 계수(Correlation Coefficient)**라고 함
 - 1. 양의 상관관계 (1에 가까움): 한 변수가 증가할 때 다른 변수도 증가하는 경향
 - 2. 음의 상관관계 (-1에 가까움): 한 변수가 증가할 때 다른 변수는 감소하는 경향
 - 3. 상관관계 없음 (0에 가까움): 두 변수 사이에 뚜렷한 선형 관계가 없음



양의 상관관계



음의 상관관계



상관관계 없음

❖ 상관관계가 높다고 해서 원인과 결과(인과관계)를 의미하는 것은 아님

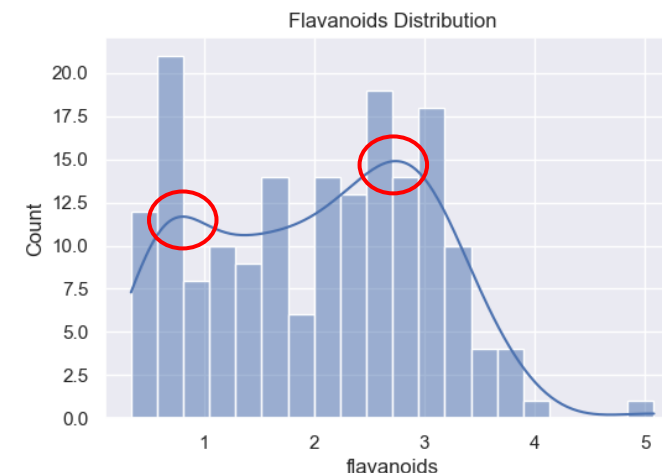
데이터 시각화

✓ 데이터 시각화

- 숫자로 가득한 표를 보는 것보다는 그래프를 보는 것이 데이터 패턴을 훨씬 직관적으로 파악할 수 있음
 - Matplotlib 과 Seaborn 을 활용하여 데이터를 시각화
 - **Matplotlib**
 - 파이썬 시각화의 근간이 되는 라이브러리
 - 그래프의 세부적인 요소, 크기, 레이아웃을 설정할 수 있음
 - **Seaborn**
 - Matplotlib을 더 예쁘고 통계적으로 의미 있는 그래프로 쉽게 만들어주는 라이브러리
 - Matplotlib을 기반으로 동작함
- Matplotlib을 기반으로 제목/축/라벨 등을 추가하고, Seaborn으로 데이터를 그래프로 그릴 예정

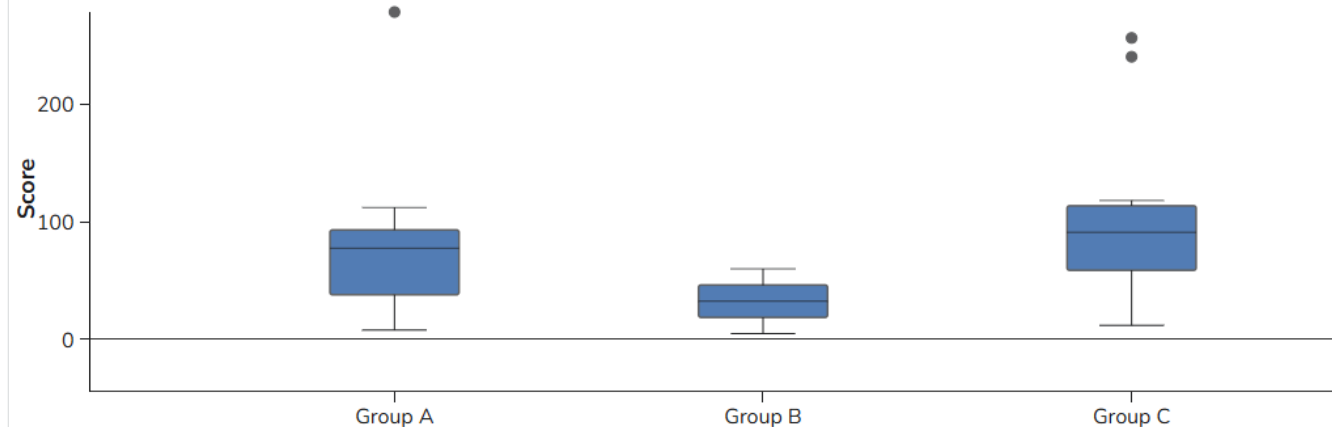
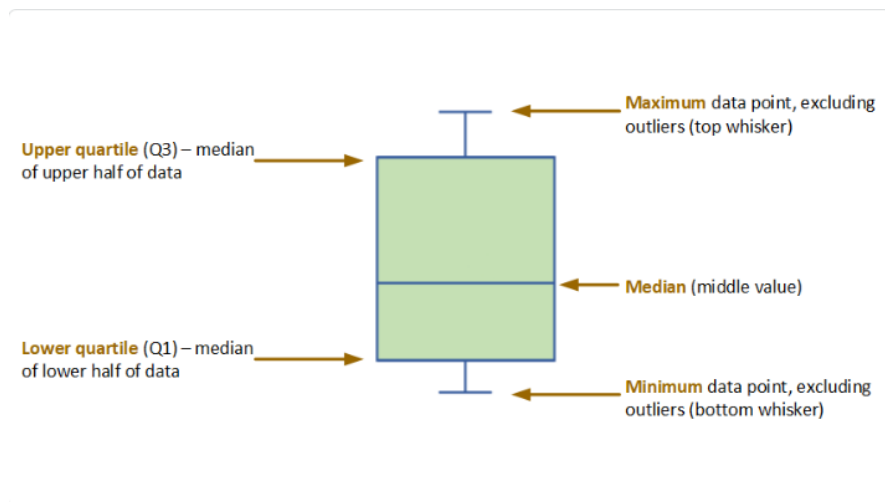
✓ 단변수 분석 - 히스토그램(Histogram)

- 특정 수치형 변수 **하나의 데이터 분포를 파악**하기 위해서 사용
- 데이터의 전체 범위를 여러 개의 동일한 구간(bin)으로 나눈 뒤, 각 구간에 속하는 데이터의 개수를 막대 그래프로 표현
- 장점
 - 직관적으로 분포를 파악할 수 있음
 - 이상치 탐지가 수월
- 단점
 - 구간(bin) 설정에 민감
- 가령 이 그래프를 살펴보면, 봉우리가 2군데가 있는데, 이는 두 개의 서로 다른 하위 그룹이 데이터에 섞여있다는 의미
 - ex) 초등학교에 있는 모든 사람의 키를 그래프로 그릴 경우
 - 초등학생들의 평균 키를 중심으로 한 '초등학생 그룹'의 봉우리
 - 선생님들의 평균 키를 중심으로 한 '선생님 그룹'의 봉우리
 - 결국 서로 다른 특성을 가진 두 그룹의 데이터가 합쳐지면, 그래프에는 각각의 그룹을 대표하는 봉우리가 생김



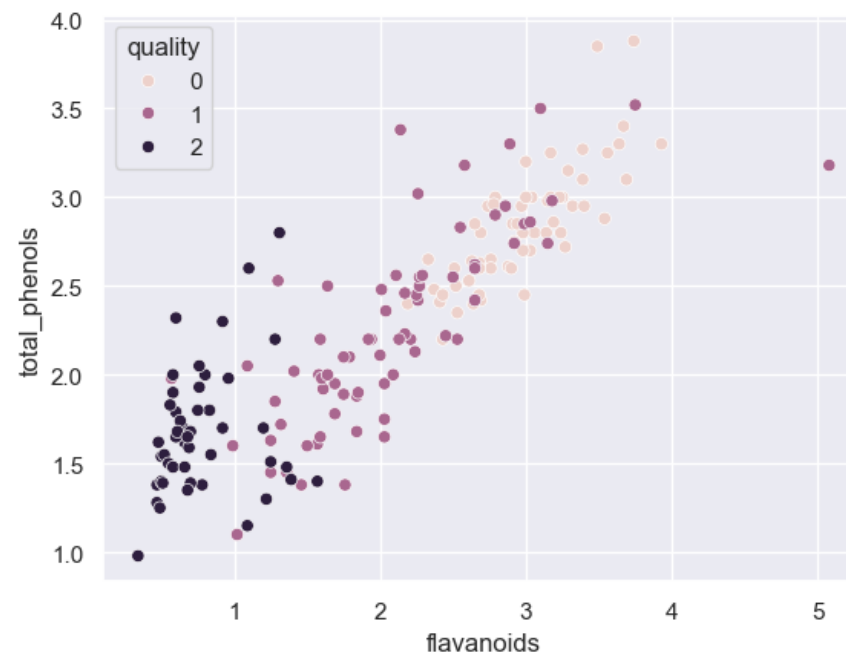
✓ 단변수 분석 - 박스 플롯(Box Plot)

- 수치형 데이터의 분포와 이상치를 한눈에 파악하기 위해 사용되는 그래프
- 데이터의 사분위수(25%, 50%, 75%)를 이용하여 전체 데이터가 어떻게 퍼져있는지를 간결하게 요약
- 다른 값들에 비해 유독 크거나 작은 값, “**이상치**”를 찾는 데 매우 유용
- 데이터 개수를 파악하기 어렵고, 데이터의 구체적인 분포 형태를 알기는 어려움



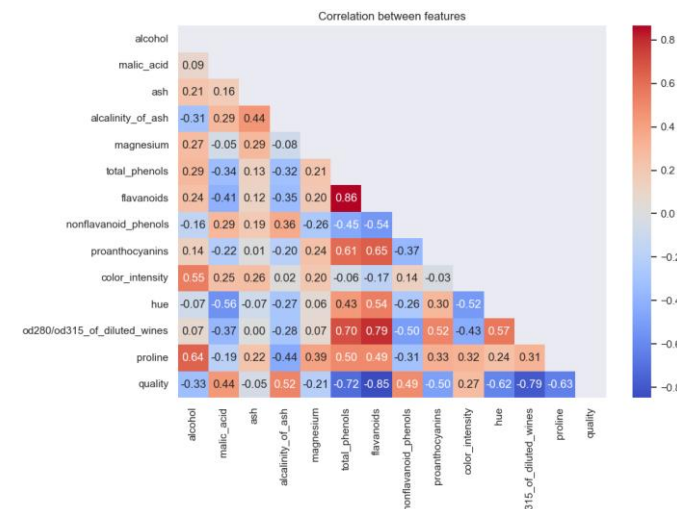
✓ 다변수 분석 - 산점도(Scatter Plot)

- 두 개의 수치형 변수 사이의 관계를 파악하기 위해 점을 찍어 만드는 그래프
- 두 변수가 어떤 관계를 갖고 있는 지 직관적으로 파악할 수 있음
 - 양의 상관관계: 한 변수가 커질 때 다른 변수도 커짐
 - 음의 상관관계: 한 변수가 커질 때 다른 변수는 작아짐
 - 비선형 관계: 점들이 곡선 형태로 분포
 - 무상관: 점들이 아무런 규칙 없이 흩어져 있는 경우
- 장점
 - 패턴이나 이상치를 쉽게 발견할 수 있음
- 단점
 - 데이터가 너무 많으면 점들이 서로 겹쳐서 분포를 제대로 파악하기 어려움
=> 투명도 조절이나 지터링(일부로 노이즈를 추가해서 같은 데이터이지만 분포를 살짝 흩어놓는 것)
 - 상관관계를 보여줄 뿐, 인과관계를 증명하지 않음 (아이스크림과 상어예시)



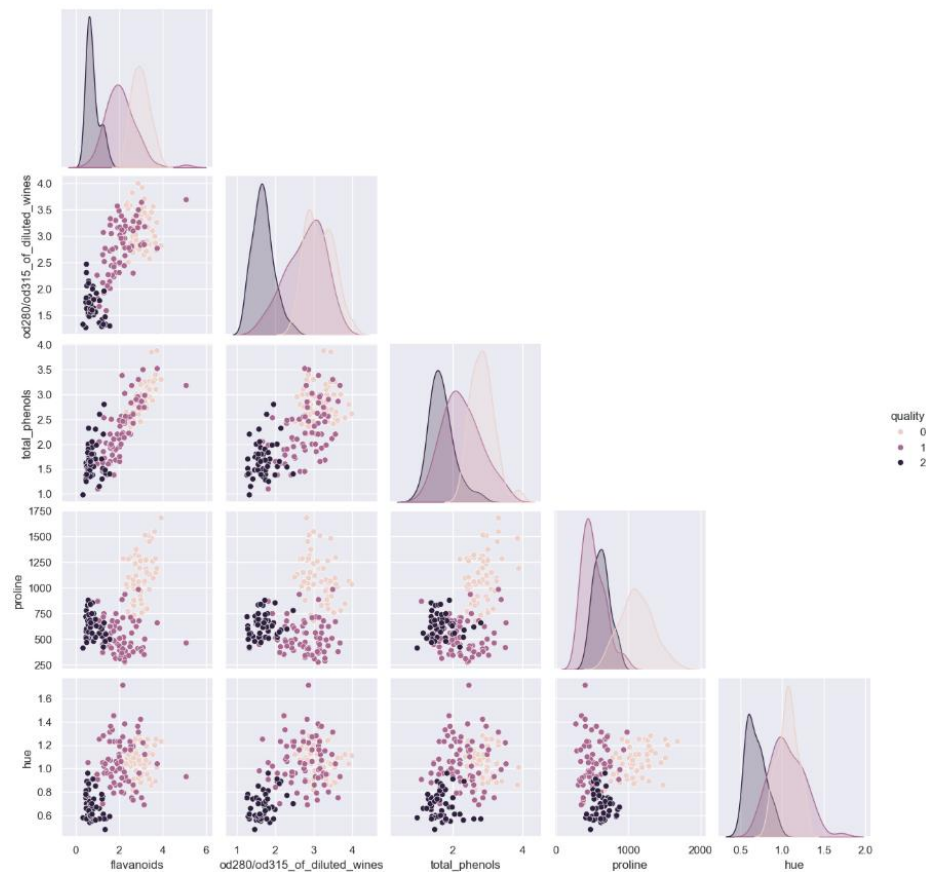
✓ 다변수 분석 - 히트맵(Heatmap)

- 여러 숫자형 변수들 간의 크기를 색상으로 변환하여 보여주는 것
 - 중심은 무채색, 양 끝으로 갈수록 서로 다른 진한 색상을 사용하는 것이 정석
- 주로 상관관계 분석에 많이 사용 됨 (다중공선성 확인)
 - ❖ 다중공선성
 - 독립 변수들 사이의 상관계수가 너무 높아서, 서로가 서로를 너무 잘 설명하는 상태
 - Ex) x_1, x_2 가 각각 결과값 Y 에 어떤 영향을 주는 지 알고 싶는데, x_1 과 x_2 가 같이 움직이면 누가 원인인지 파악하기 어려움
- 수 많은 변수 간의 관계를 한 번에 살펴보고, 상관관계가 높은 변수 쌍을 찾아내기 유용
- 장점
 - 직관적이고, 다변량 관계를 파악할 수 있음
- 단점
 - 정확한 수치를 파악하기 어려움 (어노테이션, 즉 칸 안에 숫자를 넣어 해결)
 - 변수가 너무 많으면 히트맵이 지나치게 커지고 복잡해져서 한눈에 파악하기 어려워짐



✓ 다변수 분석 - 페어 플롯(Pair Plot)

- 여러 변수들 간의 관계를 한 번에 보여주는 그래프
 - 변수들 사이의 관계와 분포를 빠르게 훑어볼 때 좋은 도구
 - 대각선(분포도)
 - 각 성분이 어떻게 분포되어있는 지를 보여줌
 - 대각선 외(산점도)
 - 두 성분 사이의 관계를 보여주는 산점도(Scatter Plot)
 - 장점
 - 짧은 코드로 모든 변수 간의 관계와 분포를 동시에 탐색할 수 있음
 - 단점
 - 변수 개수가 많아지면 계산이 오래 걸림
 - 변수가 10개를 넘어가면 너무 복잡해짐
- => 중요하다고 생각되는 몇 개의 변수만 선택해서 그리는 것이 일반적



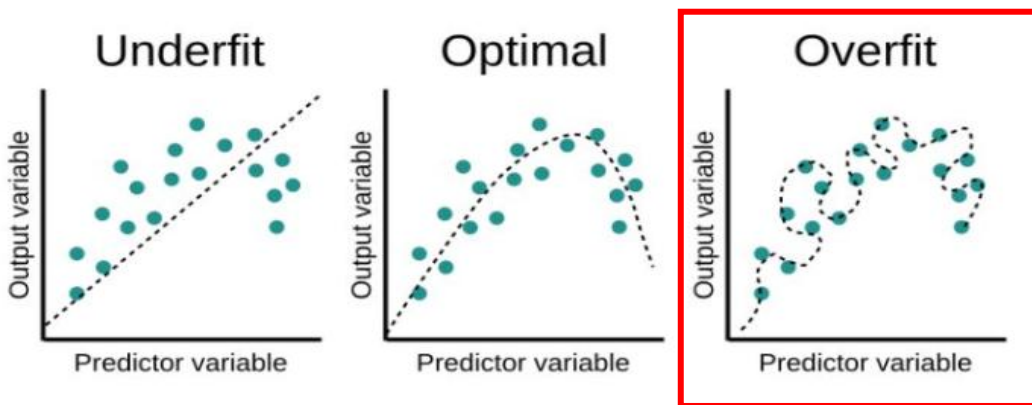
모델 학습

✓ EDA를 통해 데이터 분석이 끝났으니, 모델에 데이터를 학습시켜 보자!

- 우리의 목표는 학습 후에 새로운 문제도 잘 해결하는 “일반화 능력”을 키우는 것이다.
 - 모델이 주어진 모든 데이터를 학습한다면, 새로운 데이터에 대해서 항상 좋은 결과를 응답할까?
 - 예를 들어,
 - 모의고사 문제와 정답 심지어 오타까지 외운 학생이 과연 수능을 잘 볼 수 있을까?
 - **잘 볼 수 있다**
 - **하지만 정말 못 풀 수도 있다!**
 - 모의고사 문제에만 익숙해져서, 실전 수능에서는 문제풀이 능력이 현저하게 떨어질 수 있다.
 - 마찬가지로, 모델 또한 주어진 데이터에만 너무 ‘과하게’ 최적화되어,
훈련(기존 데이터)에서는 좋은 성능을 보이지만, 실제 테스트(새로운 데이터)에서는 성능이 나빠지는 현상
- 이것이 “과적합(Overfitting)”

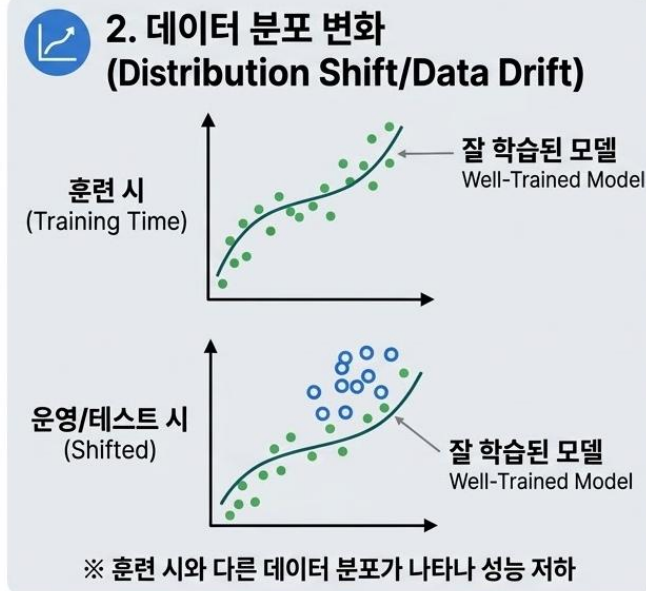
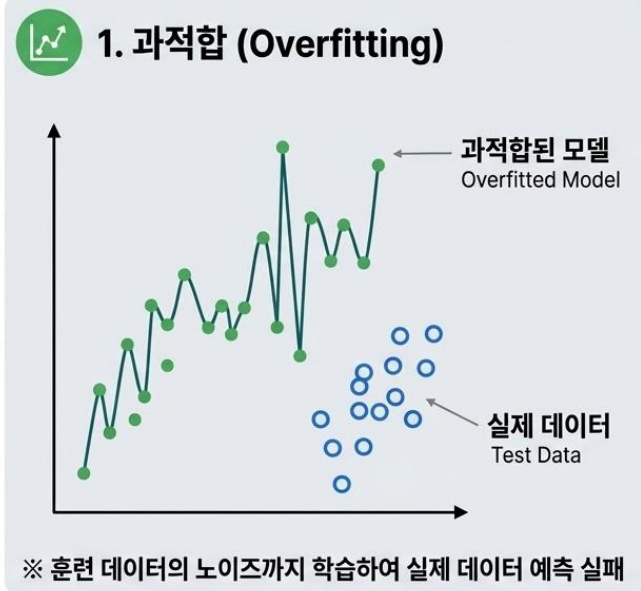
✓ 과적합(Overfitting)

- 머신러닝 모델이 훈련 데이터(Training Data)에만 너무 익숙해져서, 데이터의 중요한 패턴을 넘어 사소한 잡음(noise)까지 전부 외워버리는 현상
- 이미 학습한 훈련 데이터에서는 좋은 성능을 보이지만, 테스트 데이터(새로운 데이터)에서는 낮은 성능을 보임
- 일반화를 하지 못하고, 훈련 데이터에 따라 예측이 크게 흔들림
- 데이터를 더 학습시키기 or 모델 복잡도 낮추기 or 가중치 규제(특정 데이터에 민감하지 않도록 하기) or Dropout(학습 시 일부 뉴런을 무작위로 끄기) or Early Stopping(적당할 때 학습 멈추기)



✓ 분포 변화(Shift)

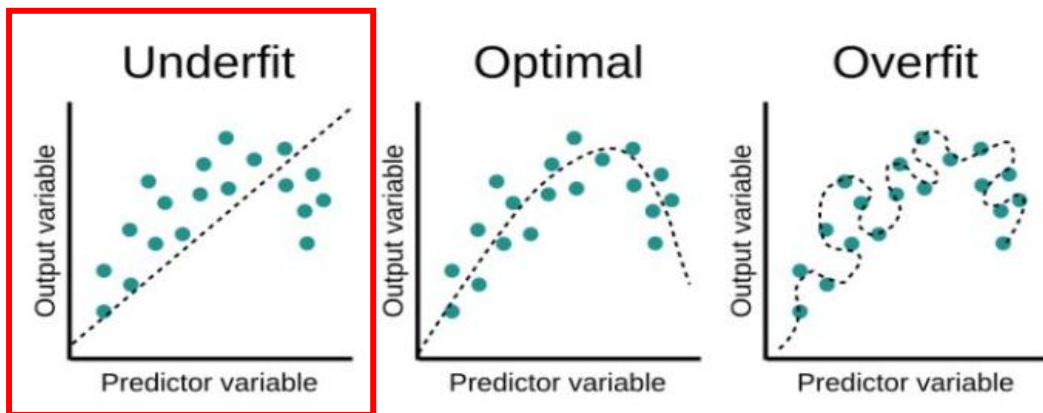
- 훈련 데이터와 실제 데이터의 성격이 다른 것
- 데이터를 재수집하거나 재학습을 진행해야 함
- 성능 저하의 원인은 “Overfitting” 뿐만 아니라 분포 변화로도 발생할 수 있음



✓ 과소적합(Underfitting)

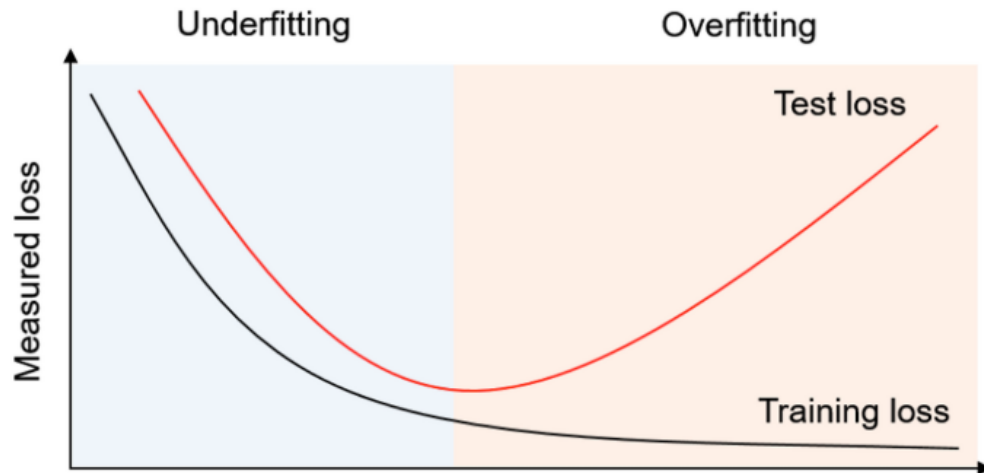
- 모델이 너무 단순해서 훈련 데이터의 핵심적인 패턴조차 제대로 학습하지 못한 상태
 - 학습이 적어서도 발생하지만, 모델 자체의 파라미터(특징)이 너무 적어서도 발생
- 모델의 성능은 훈련 데이터에서도 낮고, 테스트 데이터에서도 낮음
- 예를 들어 시험 공부를 아예 안해서, 기출문제조차 하나도 못 푸는 상태
- 더 복잡한 모델을 사용하거나 더 많은 파라미터(특징)를 추가하는 것이 해결책

➤ 그러면, 모델이 과적합(Overfitting) 혹은 과소적합(Underfitting) 됐는 지는 어떻게 알까?



✓ 데이터 분리

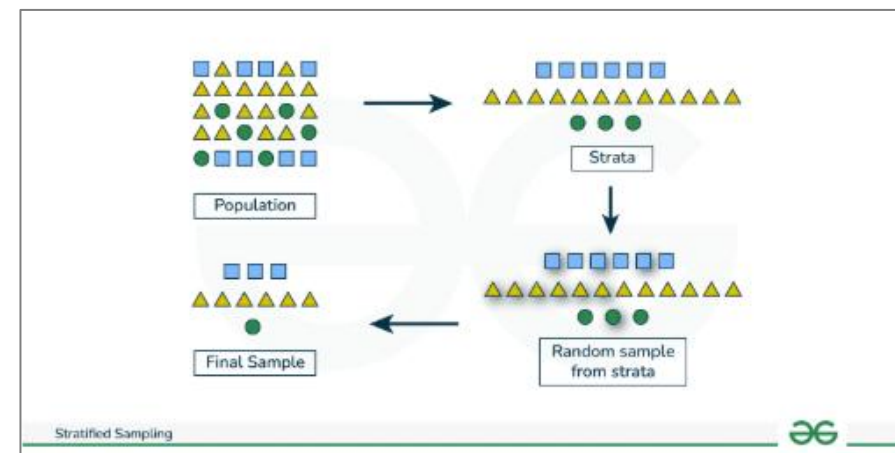
- 주어진 데이터를 훈련 데이터와 테스트 데이터로 분리
- 훈련 데이터: 모델을 학습시키는 용도로 활용
- 테스트 데이터: 모델이 얼마나 새로운 데이터를 잘 예측하는지 **일반화 성능을 최종 평가**하는 용도 (학습된 모델이 과적합(Overfitting) 됐는지 확인)
- 일반적으로 7:3, 8:2 (훈련:테스트) (데이터가 많으면 비율을 1%만 가져가기도 함)



✓ 데이터 분리 - 코드

- `train_test_split()`: 주어진 데이터를 훈련 데이터와 테스트 데이터로 분리하는 함수
- `test_size=0.3`: 전체 데이터 중 30%를 테스트용, 나머지 70%를 훈련용으로 나누겠다는 의미
- `random_state=42`: 이 숫자를 고정해놓으면 코드를 다시 시작해도 동일한 랜덤값이 배정 됨 (아무 값이나 가능)
- `stratify=y`
 - 원본 데이터의 정답 비율을 분할된 데이터셋에 동일한 비율로 나눠주는 옵션
 - 훈련 데이터에 '정상' 메일만 들어가고, 테스트 데이터에 '스팸' 메일만 들어가는 것을 방지할 수 있음
 - 분류 문제에서는 필수

```
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(X, y,
                                                    test_size=0.3,
                                                    random_state=42,
                                                    stratify=y)
```



✓ 데이터 분리를 했으니 이제 진짜 학습 시작??

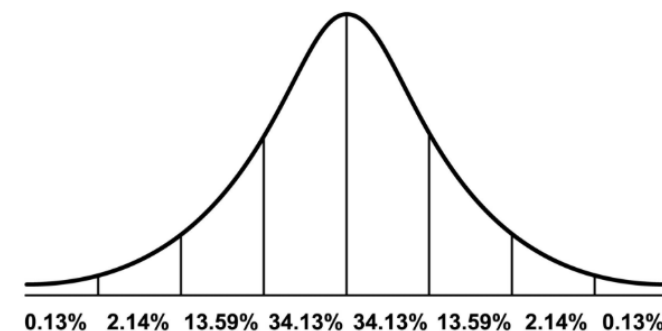
- 아니요
- 이제 데이터를 분리했을 뿐, 데이터 전처리를 아직 진행하지 않았음
- 결측치, 임베딩, **스케일링** 작업이 필요

✓ 스케일링

- 각 특성의 단위를 맞춰서 모델이 공평하게 학습하도록 만드는 과정
- 마치 키(180cm)와 몸무게(70kg)를 비슷한 범위의 값으로 조정
 - 조정하지 않으면 더 큰 값인 키가 모델에게 더 중요한 정보라고 오해받을 수 있음
- 대표적인 방법으로 **표준화**와 정규화가 있음

✓ [복습] 스케일링 - 표준화

- 모델에게 '나이'와 '연봉' 데이터를 그대로 주면 어떻게 될까?
 - 값의 범위가 훨씬 큰 '연봉'을 더 중요한 정보로 착각할 수 있음
- 제각각인 데이터의 단위를 공평하게 맞춰주는 작업
- 데이터의 평균을 0, 표준편차를 1로 변환
 - 평균 0 : 모든 데이터를 0을 기준으로 맞춤
 - 표준편차 1 : 각 데이터가 평균으로부터 얼마나 떨어져 있는지(표준편차)를 기준으로 데이터의 스케일을 조절
 - 각 데이터에서 평균을 빼고, 그것을 표준편차로 나누면 "표준화된 데이터" 생성
- 특징
 - 이상치에 상대적으로 덜 민감하고, 정규분포를 따를 때 효과적
(*이상치: 다른 값들에 비해 극단적으로 다른 값)
 - 모든 특성에 공평한 영향력을 부여
 - 경사하강법, SVM(서포트 벡터 머신, 분류/회귀), KNN(K-최근접 이웃, 분류/회귀)의 성능 향상



✓ 스케일링 - 표준화 - 코드

- 훈련 데이터의 표준화를 위한 평균과 표준편차를 구하고, 데이터 표준화 진행
- ❖ **주의**) 테스트 데이터의 표준화는 반드시 “훈련 데이터의 평균/표준편차”를 활용해야 함!!!!
 - 테스트 데이터는 아직 학습하지 않은 미래의 데이터
 - 테스트 데이터의 평균/표준편차를 scaler에 반영한다면, 정답을 미리 훑쳐본 것과 같아짐 => **데이터 누수(Data Leakage)**
 - 이렇게 유출된 정보로 학습한 모델은 실제 현실에서 제대로 된 성능이 나오지 않음

```
from sklearn.preprocessing import StandardScaler
scaler = StandardScaler()

# fit: 훈련 데이터의 평균과 표준편차 계산
# transform: 데이터를 표준화
scaler.fit(X_train)
X_train_norm = scaler.transform(X_train)

# or
# fit_transform: fit과 transform을 한 번에 수행
X_train_norm = scaler.fit_transform(X_train)

# 테스트 데이터는 훈련 데이터의 통계량(평균, 표준편차)을 사용하여 변환
X_test_norm = scaler.transform(X_test)
```

성능 평가

✓ 성능 평가

- 준비한 데이터를 기반으로 학습을 완료했다고 치자!(Gradient Descent 를 통한 가중치 최적화 진행)
- 선형 회귀 모델은 MSE(Mean Squared Error, 평균 제곱 오차)를 활용해서 모델을 평가
- 분류 모델은 어떻게 평가할까?
 - 모델의 예측 값과 실제 정답을 비교해서, 모델이 얼마나 정답을 잘 맞혔는 지 “객관적인 숫자”로 확인하는 과정
 - 평가 지표
 - 정확도(Accuracy)
 - 정밀도(Precision)
 - 재현율(Recall)
 - 조화 평균(F1-Score)

**혼동 행렬 활용
(Confusion Matrix)**

예측(y_pred)	정답(y_test)	결과
1	1	O
0	0	O
0	1	X
1	1	O

✓ 분류 모델 성능 평가 - 정확도(Accuracy) (1/2)

- (맞은 예측 개수) / (전체 예측 개수)
- 가장 이해하기 쉽고, 직관적인 성능 평가 지표
- 예시) 스팸 메일 분류
 - 100개의 이메일 중에서 실제 스팸 메일 10개와 정상 메일 90개가 있다.
 - 모델의 예측 결과
 - 스팸 10개 중 8개를 스팸으로 맞춤
 - 정상 90개 중 85개를 정상으로 맞춤
 - 모델이 올바르게 예측한 총 개수는 93개이므로, 정확도는 $93 / 100 = 93\%$

```
from sklearn.metrics import accuracy_score  
  
accuracy = accuracy_score(y_test, y_pred)
```

➤ 하지만, 정확도가 아무리 높아도 안심할 수 없음 why ?

✓ 분류 모델 성능 평가 - 정확도(Accuracy) (2/2)

- 하지만, 정확도가 아무리 높아도 안심할 수 없음
- 예) 암 진단
 - 100명의 환자 중 정상 99명, 암 환자 1명이 있다.
 - 모든 환자를 전부 '정상'이라고 예측하는 모델이라고 하면, 이 모델은 99명의 정상을 맞추고, 1명의 암 환자를 틀렸음.
 - 정확도는 99% 이지만, 정작 우리가 찾아야 할 1명의 암 환자를 놓쳤기 때문에
 - **이 모델은 아무 쓸모가 없음**
- 이처럼 불균형 데이터에서는 정확도를 성능 평가 지표로 사용하기 어려움

✓ 분류 모델 성능 평가 - 혼동 행렬(Confusion Matrix) (1/4)

- 분류 모델의 성능을 평가할 때 사용하는 가장 기본적이면서도 강력한 도구
- 어떤 실수를 어떻게 저질렀는지를 한눈에 보여주는 상세한 지표
- Positive: 우리가 찾으려는 대상(ex: 암, 스팸)
- Negative: 그 외의 대상(ex: 정상)
- 구성 요소
 - True Positive(TP)
 - True Negative(TN)
 - False Positive(FP)
 - False Negative(FN)

		예 측 클 래 스	
		Positive	Negative
실 제 클 래 스	Positive	TP (True Positive)	FN (False Negative)
	Negative	FP (False Positive)	TN (True Negative)

✓ 분류 모델 성능 평가 - 혼동 행렬(Confusion Matrix) (2/4)

- True Positive(TP)

- 실제 Positive인 것을 모델이 Positive로 정확히 예측한 것
- 암 환자(Positive)를 암(Positive)이라고 올바르게 진단
- 정답

- True Negative(TN)

- 실제 Negative인 것을 모델이 Negative로 정확히 예측한 것
- 정상인(Negative)을 정상(Negative)이라고 올바르게 진단
- 정답
- 실무 데이터에서는 정상 데이터가 압도적으로 많기 때문에 해당 능력도 중요

		예 측 클 래 스	
		Positive	Negative
실 제 클 래 스	Positive	TP (True Positive)	FN (False Negative)
	Negative	FP (False Positive)	TN (True Negative)

✓ 분류 모델 성능 평가 - 혼동 행렬(Confusion Matrix) (3/4)

- False Positive(FP)
 - 실제 Negative 인 것을 모델이 Positive 로 잘못 예측한 것
 - 정상인(Negative)을 암(Positive) 이라고 잘못 진단
 - 모델의 선부른 판단
- False Negative(FN)
 - 실제 Positive 인 것을 모델이 Negative 로 잘못 예측한 것
 - 암 환자(Positive)을 정상(Negative)이라고 잘못 진단
 - 모델이 놓침
- 모델이 놓치는 건, 선부른 판단보다 훨씬 위험!

		예 측 클 래 스	
		Positive	Negative
실 제 클 래 스	Positive	TP (True Positive)	FN (False Negative)
	Negative	FP (False Positive)	TN (True Negative)

✓ 분류 모델 성능 평가 - 혼동 행렬(Confusion Matrix) (4/4)

- 100명의 환자를 진단하는 암 예측 모델이 있다고 가정 (실제 암 환자: 10명, 실제 정상인: 90명)
- 모델의 예측 결과
 - 실제 암 환자 10명 중 8명을 암으로 예측(TP=8)
 - 실제 암 환자 10명 중 2명을 '정상'으로 놓침(FN=2)
 - 실제 정상인 90명 중 5명을 '암'으로 오진(FP=5)
 - 실제 정상인 90명 중 85명을 '정상'으로 예측(TN=85)

		예측 클래스	
		Positive	Negative
실제 클래스	Positive	TP (True Positive)	FN (False Negative)
	Negative	FP (False Positive)	TN (True Negative)

	예측: 암 (Positive)	예측: 정상 (Negative)	합계 (실제)
실제: 암	TP = 8	FN = 2	10
실제: 정상	FP = 5	TN = 85	90
합계 (예측)	13	87	100

✓ 분류 모델 성능 평가 - 정밀도(Precision)

- 모델이 “Positive”라고 예측한 것들 중에서 진짜 “Positive”한 것들의 비율
- 모델이 “암” 이라고 예측한 것들 중에서 진짜 “암”이었던 비율
- 모델의 예측이 **얼마나 정교한지**를 나타냄
- 정밀도 = $TP / (TP + FP) = 8 / (8 + 5) = 61.5\%$
- 모델의 예측을 “**진짜라고 믿을 수 있어야 할 때**” 유용

		예측 클래스	
		Positive	Negative
실제 클래스	Positive	TP (True Positive)	FN (False Negative)
	Negative	FP (False Positive)	TN (True Negative)

	예측: 암 (Positive)	예측: 정상 (Negative)	합계 (실제)
실제: 암	TP = 8	FN = 2	10
실제: 정상	FP = 5	TN = 85	90
합계 (예측)	13	87	100

✓ 분류 모델 성능 평가 - 재현율(Recall)

- 실제 “Positive” 중에서 모델이 “Positive”라고 맞춘 비율
- 실제 “암” 환자들 중에서 모델이 “암”이라고 맞춘 비율
- 모델이 **찾아야 할 것을 얼마나 빠짐없이 잘 찾아냈는지**를 나타냄
- 재현율 = $TP / (TP + FN) = 8 / (8 + 2) = 80\%$
- 모델이 **“하나라도 놓치면 안 될 때”** 유용

		예측 클래스	
		Positive	Negative
실제 클래스	Positive	TP (True Positive)	FN (False Negative)
	Negative	FP (False Positive)	TN (True Negative)

	예측: 암 (Positive)	예측: 정상 (Negative)	합계 (실제)
실제: 암	TP = 8	FN = 2	10
실제: 정상	FP = 5	TN = 85	90
합계 (예측)	13	87	100

✓ 분류 모델 성능 평가 - 조화 평균(F1-Score)

- 정밀도와 재현율을 모두 고려하고, 균형 잡힌 성능을 나타낼 때 사용
- 클래스 불균형이 심할 때, 정확도를 대체할 수 있는 훌륭한 성능 지표
- 일반적으로 정밀도를 높이면 재현율이 떨어지고, 재현율을 높이면 정밀도가 떨어지는 경향이 있음(Trade-off)
- 정밀도 = $TP / (TP + FP) = 8 / (8 + 5) = 61.5\%$
- 재현율 = $TP / (TP + FN) = 8 / (8 + 2) = 80\%$
- F1-score = 69.5%

$$F1 = \frac{2 \times Precision \times Recall}{Precision + Recall}$$

	예측: 암 (Positive)	예측: 정상 (Negative)	합계 (실제)
실제: 암	TP = 8	FN = 2	10
실제: 정상	FP = 5	TN = 85	90
합계 (예측)	13	87	100

성능 검증

✓ 평가한 점수가 정말 신뢰성이 있을까?

- 데이터를 어떻게 나눴는지 / 하이퍼파라미터를 어떻게 설정했는 지에 따라 점수가 달라질 수 있음
 - 하이퍼 파라미터: 사람이 외부에서 직접 설정하는 값 (ex: 학습률)
- 어떤 방식이 좋은 성적을 받을 수 있는 지를 검증하자!
 - 학습률을 0.1로 설정하고 검증, 학습률을 0.2로 설정하고 검증, ...
 - 제일 높은 성적을 받는 방식을 채택해서 성능 평가
- 검증을 테스트 데이터로 진행하면, “데이터 유출(Data Leakage)” 발생
 - 데이터 유출(Data Leakage): 모델의 설정값이 테스트 데이터의 특성에 맞춰지는 것. 즉, 정보가 세어나감
- 검증 방식 종류
 - 검증 데이터(단순 분할 방식)
 - 교차 검증(Cross-Validation)

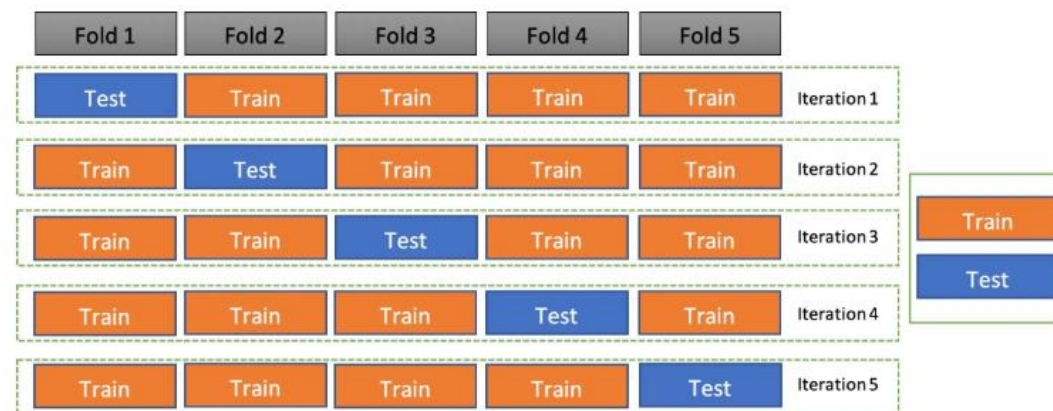
✓ 검증 데이터(Validation Data)

- 모델을 훈련시키는 과정에서, 어떤 훈련 방법이 가장 좋은지 확인하고, 최고의 모델을 선택하기 위해 사용되는 데이터
- 예를 들어,
 - 동일한 학습데이터에 대해서 모델이 A 방법으로 학습하고, 동일한 검증 데이터로 평가 => 85점
 - 동일한 학습데이터에 대해서 모델이 B 방법으로 학습하고, 동일한 검증 데이터로 평가 => 70점
 - 동일한 학습데이터에 대해서 모델이 C 방법으로 학습하고, 동일한 검증 데이터로 평가 => 65점
 - 모델은 가장 점수가 높은 “B 방법”으로 학습을 진행하고, 이후 테스트데이터로 평가 진행
- 즉, **최고의 학습 방법을 구분하기 위한** 중간 점검 데이터라고 생각하면 됨
- 검증 데이터는 학습 용도로서 활용되지 않으며, 학습 데이터에서 따로 분리
- 일반적으로 6:2:2 (Train : Val : Test) 비율을 많이 사용
- 검증 데이터가 없이 테스트 데이터로만 성능을 자꾸 확인하면, 테스트 데이터에만 잘 맞는 모델이 만들어짐
=> 정보 유출(Data Leakage) (이후에 학습 예정)

✓ 교차 검증(Cross-Validation)

- 훈련 데이터를 여러 개(K개)의 덩어리(Fold)로 나눈 뒤, 각 Fold가 돌아가면서 한 번씩 '검증 데이터'역할을 하는 것
- 검증 데이터를 일회성으로 사용했을 때의 불안정성을 극복하기 위해, 여러 개의 검증 세트를 체계적으로 만들어 평가하는 방법
- 마치 수능 전에 여러 번의 모의고사를 보는 것과 같고, 여러 번의 모의고사의 평균 점수는 진짜 실력일 확률이 높다!

구분	검증 데이터 (단순 분할)	교차 검증 (K-Fold)
목적	동일 (하이퍼파라미터 튜닝, 중간 성능 측정)	동일
'연습 시험지'	고정된 1개	여러 개(K개)가 돌아가며 역할 수행
신뢰도	데이터 분할에 따라 결과가 좌우될 수 있음	더 안정적이고 신뢰도 높은 성능 추정 가능
비용	계산이 빠름	모델을 여러 번 학습시켜 시간이 더 걸림



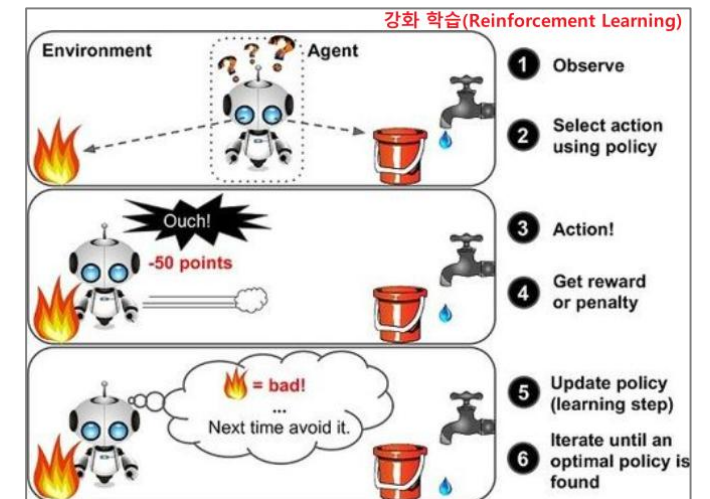
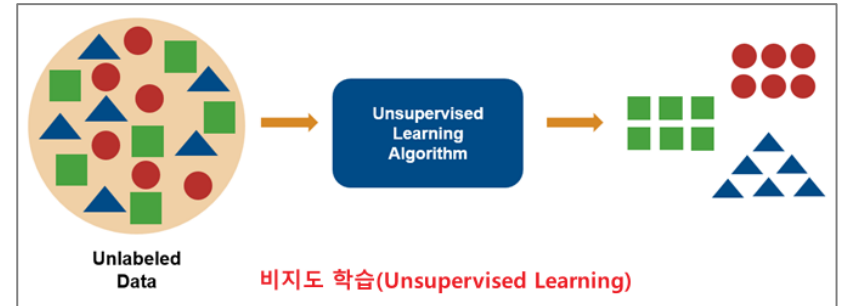
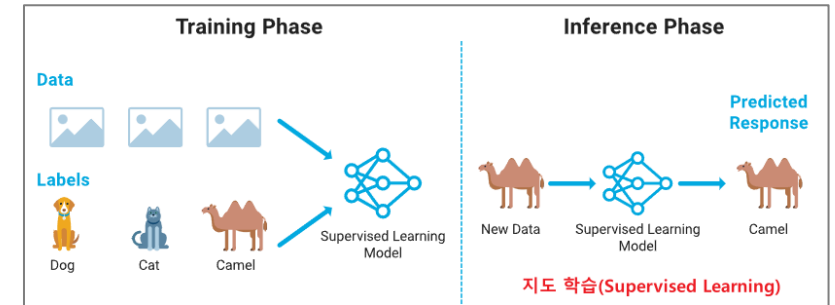
✓ 지도 학습 정리

- ✓ 정답이 있는 데이터를 가지고 모델을 학습시키고(경사하강법)
- ✓ 정답을 얼마나 잘 맞혔는지 평가(검증 데이터, 교차 검증)를 진행
- ✓ 근데 현실에는 “정답이 없는 데이터”가 더 많음.
- ✓ 쇼핑몰 고객들을 특징별로 묶고 싶은데, “이 사람은 A 그룹이다!” 라는 정답이 없으면 어떻게 할까?
- ✓ 이럴 때 필요한 게 바로 비지도 학습!

비지도 학습

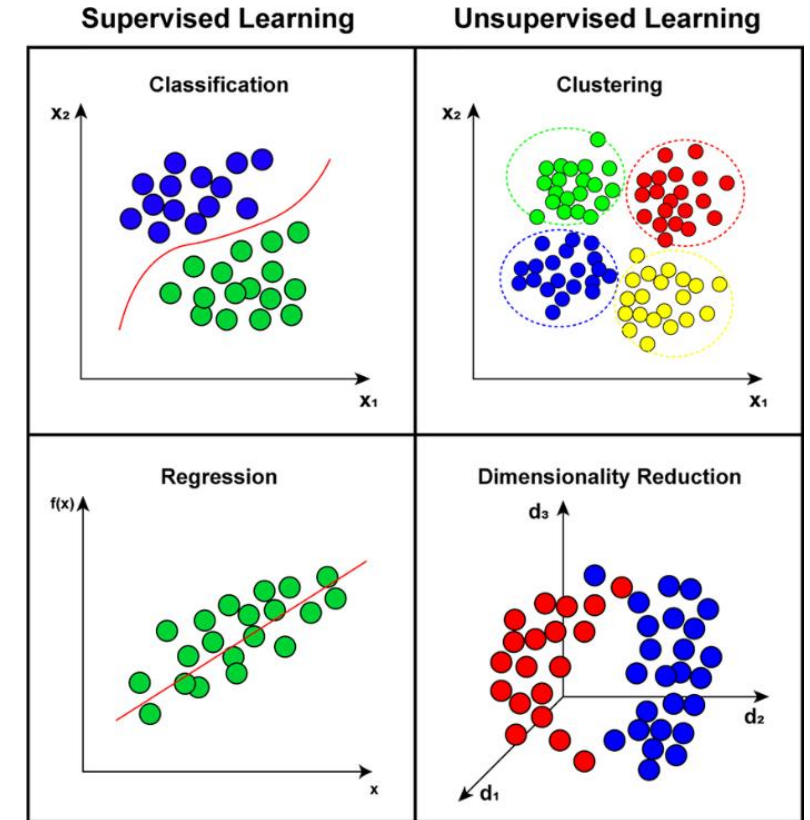
✓ 머신러닝

- 컴퓨터에게 데이터를 주고, 그 데이터 안에서 규칙(패턴)을 스스로 학습하게 하는 방식
- 학습 종류
 - 지도 학습(Supervised Learning)
 - 정답지가 있는 데이터를 학습시키는 방법
 - 의료 진단, 스팸 필터링, 주가/부동산 가격 예측 등에 활용
 - 비지도 학습(Unsupervised Learning)
 - 정답이 없는 데이터의 숨겨진 구조나 규칙을 찾는 방법
 - 군집화, 추천 시스템, 이상 거래 탐지, 고객 세분화 등에 활용
 - 강화 학습(reinforcement Learning)
 - '상' 과 '벌' 을 통해서 최적의 행동 방식을 학습하는 방법 (알파고)
 - 로봇 제어, 자율 주행, 게임 AI 등에 활용
- 일반적으로 비지도 -> 지도 -> 강화 학습으로 복합적인 학습으로 결과물 생성



✓ 비지도 학습(UnSupervised Learning)

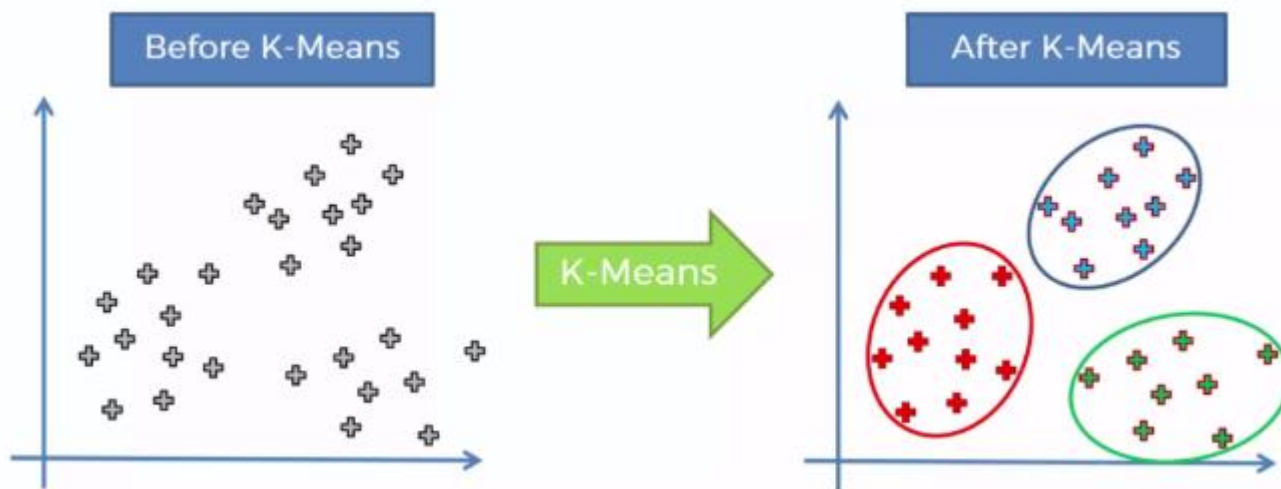
- 정답이 없는 데이터의 숨겨진 구조나 규칙을 찾는 방법
- 수백 장의 동물 사진을 주고 “비슷한 것끼리 묶어 봐” 라고 하는 것
- 비지도 학습 유형
 - 군집화(Clustering)
 - 비슷한 데이터끼리 그룹으로 묶는 것
 - 차원 축소(Dimensionality Reduction)
 - 데이터의 복잡성(차원)을 줄여 핵심만 남기는 것
 - ex) 3D 물체를 2D 그림자로 표현해도 그 형태를 알 수 있는 것
 - 연관(Association)
 - 데이터 안에서 특정 항목들이 얼마나 자주 함께 발생하는지를 분석하여 규칙이나 관계를 찾아내는 기법
 - ex) 마트에서 기저귀를 사는 사람은 맥주도 함께 살 확률이 높다.



군집화(Clustering)

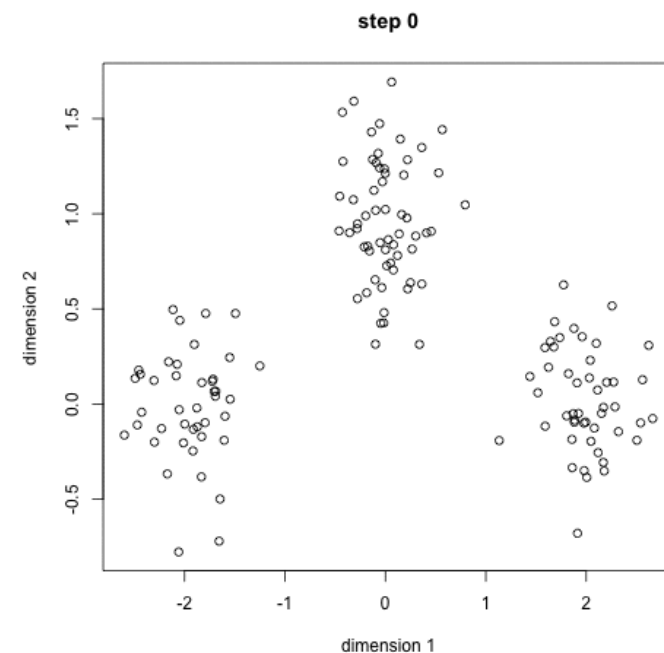
✓ 군집화(Clustering)

- 정답이 없는 데이터들을 유사(거리가 가까움)한 특징을 가진 것끼리 하나의 그룹(Cluster)으로 묶는 작업
 - 예) 구매 패턴이 비슷한 고객 나누기, 이미지 분할, 비슷한 주제의 문서를 분류
- 정답이 없어도 되기 때문에 레이블링 작업이 불필요
- 정답이 없기 때문에 사람이 파악하지 못한 숨겨진 패턴을 찾을 수 있음
- 군집화 알고리즘 종류
 - K-평균(K-Means)
 - 계층적 군집(Hierarchical)



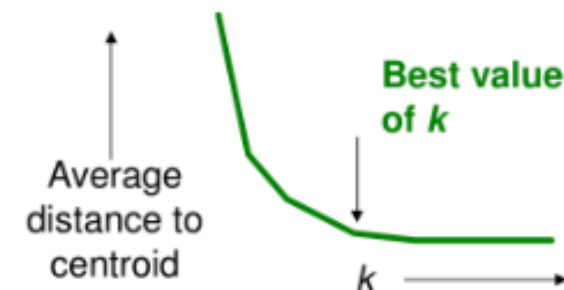
✓ K-Means (1/3)

- 데이터들이 가장 가까운 중심점(Centroid)을 기준으로 뭉치도록 그룹을 형성하도록 하는 알고리즘
- 동작 방식
 1. K개의 중심점을 데이터 공간에 랜덤으로 배치
 2. 모든 데이터는 **자신과 가장 가까운 중심점과 그룹을 형성** (K개의 그룹이 생성됨)
 3. 각 그룹의 평균을 계산하고, **그 지점을 다시 중심점**으로 설정
 4. 새로 생성된 중심점을 기준으로 2~3번 과정을 **“더 이상 변화가 없을 때까지”** 반복



✓ K-Means (2/3)

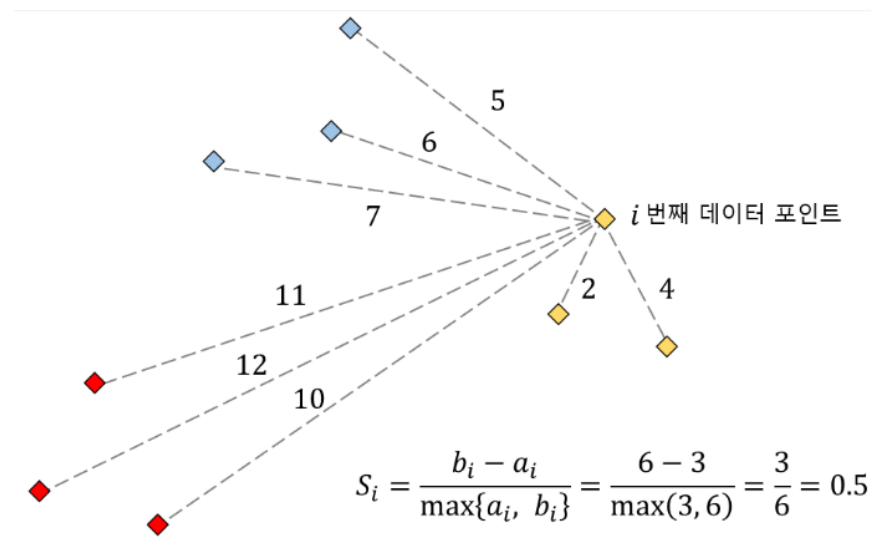
- 이상치에 민감하고, 초기 중심점의 위치에 따라 결과가 달라질 수 있음
- 군집의 개수(K)를 직접 지정해야 함
 - 너무작게잡으면, 다른 성격이 묶이고
 - 너무크게잡으면, 같은 그룹이 여러 개로 쪼개질 수 있음
- 엘보우 방법(Elbow Method)
 - 적절한K 값을 선택하는 방법
 - K를 1부터 10까지 점차 늘려가면서 “이너셔”를 계산
 - 이너셔
 - 각데이터 포인트가 자신이 속한 클러스터의 중심점에서 얼마나 떨어져 있는지, 그 거리의 총합
 - 이너셔가 작을수록 데이터들이 잘 뭉쳐있음을 나타냄
 - 이너셔가 급격하게 감소하다가 완만해지는 지점이 “최적의K”가 될 확률이 가장 높은 후보



[엘보우 기법]

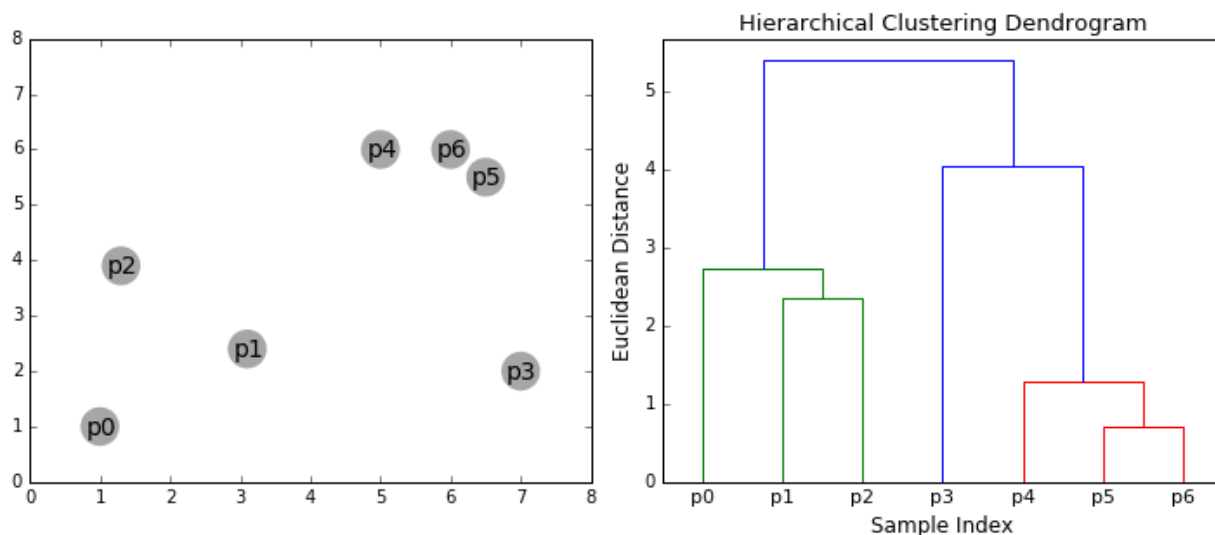
✓ K-Means (3/3)

- 정답이 없는데 군집이 잘 뭉쳤는지는 어떻게 평가할까?
- 실루엣 스코어(Silhouette Coefficient)
 - 각 데이터들이 다른 그룹과의 가까움에 따라서 점수를 매김
 - a_i : 내가 속한 그룹 안의 데이터들과의 거리(응집도, 작을수록 좋음)
 - b_i : 내가 속하지 않은 가장 가까운 그룹과의 거리(분리도, 클수록 좋음)
 - 결과값은 항상 -1에서 1사이로 나옴
 - 1에 가까움: 군집화가 아주 잘 됨
 - 0 근처: 군집 간의 경계가 모호함. 서로 겹쳐 있을 가능성이 높음
 - 음수: 데이터가 아예 잘못된 그룹에 들어가 있음



✓ 계층적 군집(Hierarchical Clustering) (1/2)

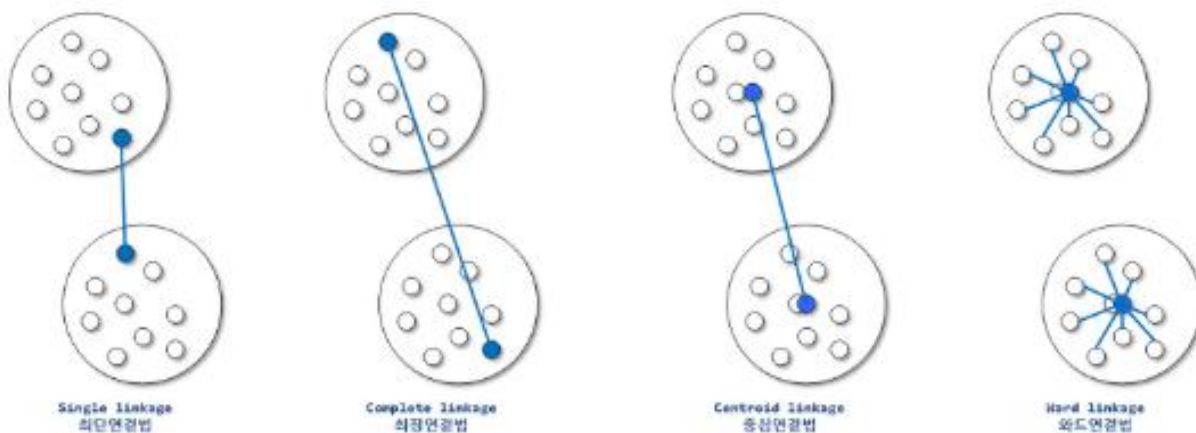
- 데이터 간의 거리를 바탕으로 가까운 것부터 순서대로 묶어 나가면서 나무 형태의 구조를 만들어가는 방식
- K-Means 와 달리 K 값을 미리 정하지 않아도 되는 군집
- y축의 높이는 군집 간의 거리를 의미하므로, 높이가 높을수록 서로 많이 다른 성격임을 뜻함
- 나무를 어느 높이에서 자르느냐에 따라 군집의 개수가 결정됨 (결과를 보고 최적의 K를 고를 수 있음)
- 모든 점 사이의 거리를 계산해야 하므로 계산량이 매우 많음



덴드로그램(Dendrogram)

✓ 계층적 군집(Hierarchical Clustering) (2/2)

- 이미 만들어진 군집 사이의 거리를 어떤 기준으로 잴 것인가?
 - 최단연결: 두 군집의 점들 중 가장 가까운 점끼리의 거리 $\Rightarrow$ 사슬 현상이 나타날 수 있음(점들이 짐검다리처럼 등성등성 있으면 합쳐지는 현상)
 - 최장연결: 두 군집의 점들 중 가장 먼 점끼리의 거리 $\Rightarrow$ 이상치에 매우 민감
 - 중심연결: 두 군집의 중심 간의 거리 $\Rightarrow$ 모양이 꼬일 수 있음
 - 평균연결: 두 군집의 모든 점 조합에 대한 거리의 평균 $\Rightarrow$ 시간이 엄청 걸림
 - 와드 연결: 군집 내의 오차제곱합에 기초하여 군집을 수행 $\Rightarrow$ 가장 많이 쓰임



차원 축소

✓ 데이터를 분석할 때, 데이터의 특징이 매우 많다면 모델이 더 똑똑해질까?

- No! “차원의 저주(Curse of Dimensionality)”
- 차원의 저주
 - 데이터의 차원(특징의 개수)이 증가할수록, 데이터를 분석하는 데 필요한 공간의 부피가 기하급수적으로 커져 발생하는 문제
 - 즉, 공간은 광활해지는데 데이터 개수는 그대로라면, 데이터 사이의 거리가 멀어져서 아무 패턴도 찾지 못하는 현상
 - 문제점
 - 불필요한 정보(노이즈)가 많아져 모델의 예측 성능이 떨어질 수 있음
 - 모델이 너무 복잡해져 과적합(Overfitting)이 될 가능성이 높아짐
 - 계산할 것이 너무 많아져 모델 학습에 많은 시간이 걸림
 - 3차원을 넘어서면서 시각화하기 어려움

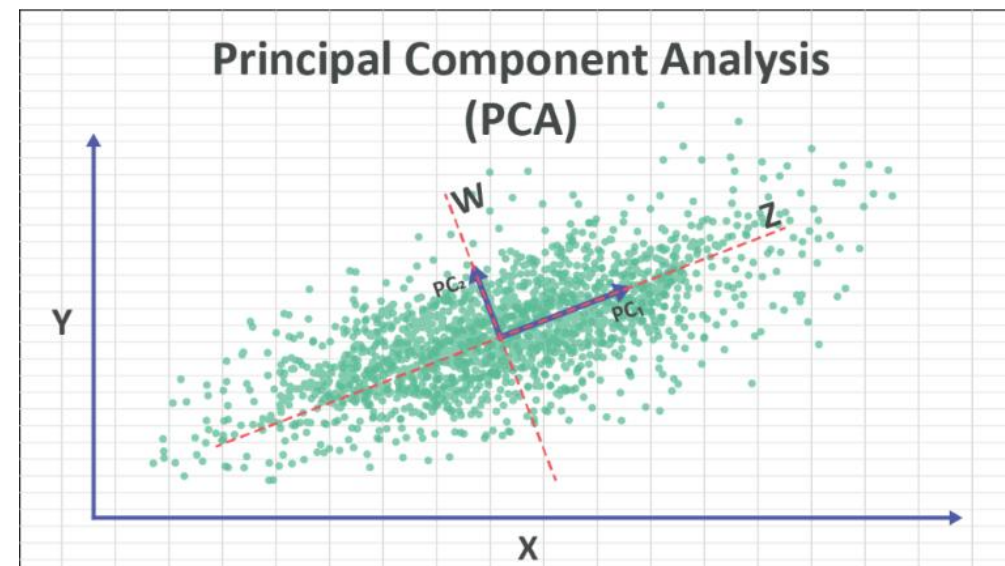
➤ 데이터의 중요한 정보는 최대한 유지하면서, 불필요한 특징을 제거하자
⇒ “차원 축소(Dimensionality Reduction)”

✓ 차원 축소(Dimensionality Reduction)

- 데이터의 중요한 정보는 최대한 유지하면서, 서로 관련이 높거나 불필요한 특징들을 제거하거나 압축하여 데이터의 차원을 줄이는 기술
- 두 가지 방법
 - 특성 선택
 - 여러 특징 중에서 가장 중요하다고 생각되는 몇 개의 특징만 골라내는 방식
 - ex) 100개 특징 중 상관관계가 높은 10개만 선택
 - **특성 추출**
 - 기존 특징들을 새로운 축으로 투영하여, 더 적은 수의 새로운 특징으로 압축하는 방식
 - **PCA(주성분 분석)**

✓ PCA(Principal Component Analysis)

- 데이터가 가장 넓게 퍼져있는(분산이 가장 큰) 방향으로 새로운 축을 찾는 기술
 - 최대한 넓게 퍼진 축이 데이터 사이의 차이점(정보)을 가장 잘 보여줌
- 새로운 축이 바로 데이터의 정보를 가장 잘 설명하는 **제1 주성분**(Principal Component)
- 예를 들어) 럭비공을 2D로 표현한다면?
 - 럭비공이 가장 길어보이는 방향에서 찍어야 함
 - 주성분은 이런 데이터를 정보를 가장 많이 포함하는 첫 번째 축
- 주성분과 직각을 이루는 축이 **제2 주성분**
- PCA를 하기 전에는 모든 특징의 단위를 똑같이 맞춰주는 StandardScaler가 필수!
그렇지 않으면, 큰 값을 가진 특징이 주성분을 독점함



4. MLP

• INDEX

- 딥러닝
 - 퍼셉트론
 - Nerual Network
- MLP
- MLP 성능 향상
 - 활성화함수(ReLU)
 - 옵티마이저(Adam)
 - 규제(Dropout)

딥러닝

✓ 머신러닝과 딥러닝

- 컴퓨터에게 데이터를 주고, 그 데이터 안에서 규칙(패턴)을 스스로 학습하게 하는 방식

• 학습 종류

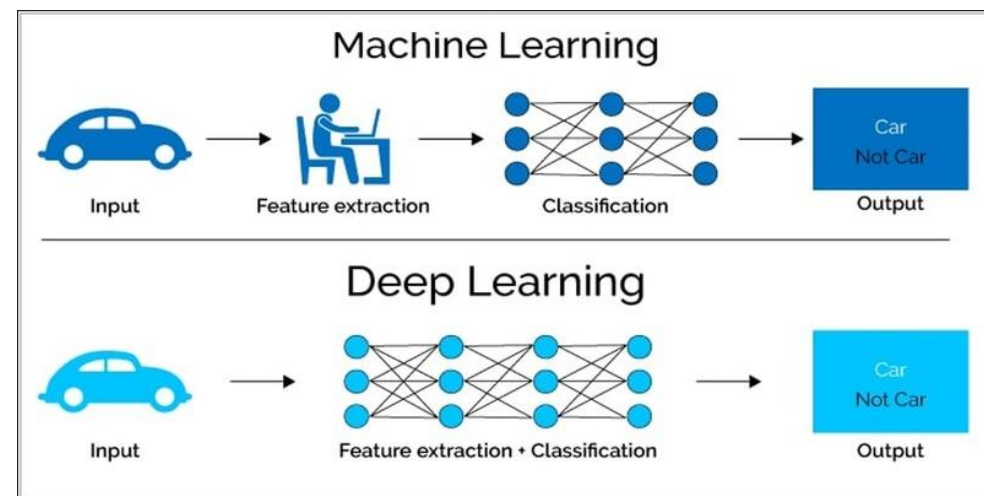
- 지도 학습(Supervised Learning) => 선형/로지스틱 회귀
- 비지도 학습(Unsupervised Learning) => K-means, PCA
- 강화 학습(reinforcement Learning)

고전적인 머신러닝!
(안 좋은 의미가 아닌 Classical)

• 고전적인 머신러닝의 한계

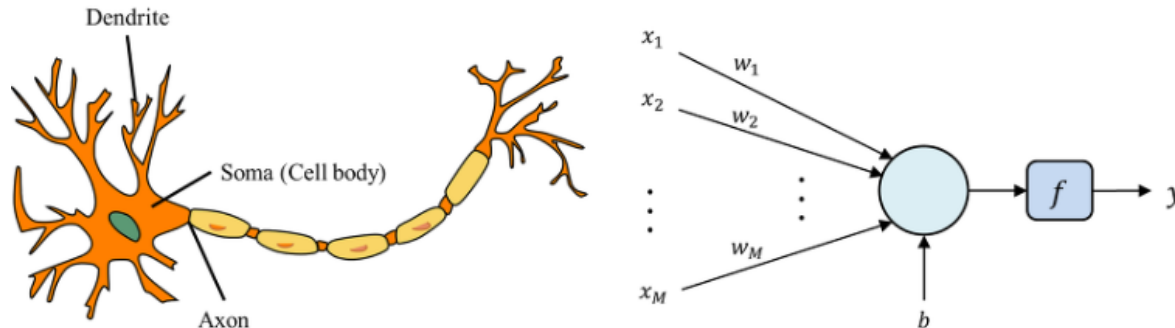
- 모델이 잘 학습할 수 있도록 사람이 직접 데이터의 중요한 부분을 알려주고, 가공해서 먹여줘야 함
- 데이터의 양이 일정 수준을 넘어서면, 복잡한 패턴을 추가로 학습하지 못하고 성능이 정체 됨
- 기본적으로 행과 열이 정해진 정형 데이터 처리에 적합하여, 이미지나 텍스트, 음성과 같은 “비정형 데이터” 처리에 비효율적

- 위 한계를 극복하기 위해서 등장한 방법론이 “딥러닝”



✓ 딥러닝(Deep Learning) 아이디어

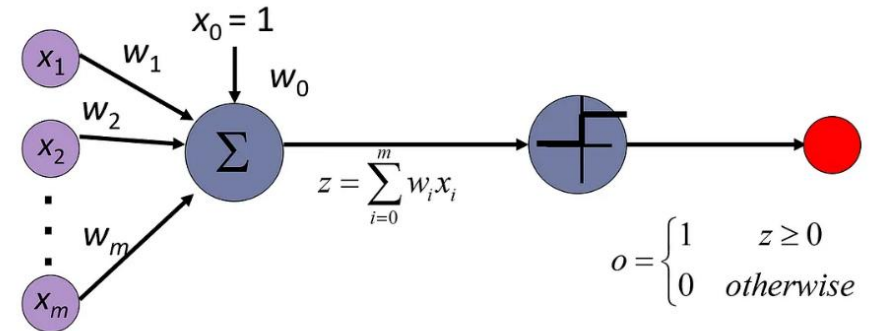
- 인간의 뇌에서 영감을 받은 인공신경망을 사용하여 복잡한 패턴을 학습하는 머신러닝 학습 방법론
- 핵심 아이디어
 - 뇌의 신경망 구조와 동작 방식을 컴퓨터로 모방
 - 뇌가 수십억 개의 신경세포(뉴런)들이 서로 신호를 주고 받으며 학습하고 판단하는 것처럼
 - 컴퓨터 안에 인공뉴런을 만들고, 이들을 연결하여 네트워크를 구축
- 인공 신경망(ANN, Artificial **Neural Network**)
 - 컴퓨터 안에 인공뉴런(**퍼셉트론**)을 만들고, 이들을 연결하여 네트워크를 구축한 것



퍼셉트론

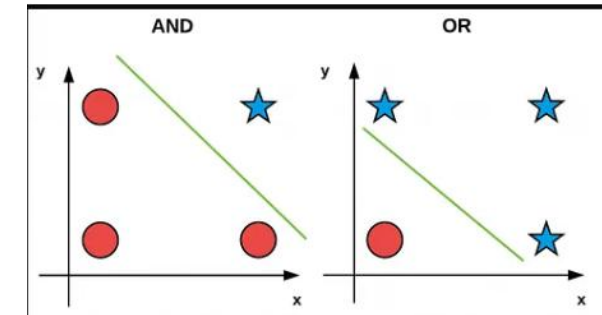
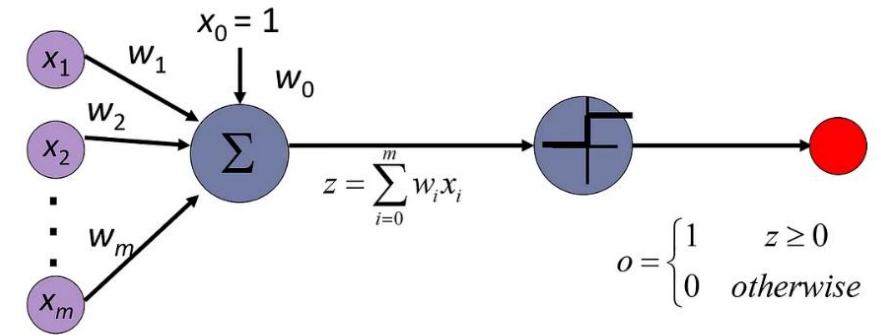
✓ 퍼셉트론(Perceptron) (1/3)

- 여러 정보를 받아서 최종적으로 참(1) 또는 거짓(0)과 같은 결정을 내리는 간단한 결정 모델
- 인공 신경망(ANN)의 가장 기본적인 단위로, 사람의 뇌를 구성하는 뉴런의 작동 원리를 모방한 알고리즘
- 구성 요소
 - 입력(inputs, x)
 - 가중치(weights, w)
 - 가중합(Weighted Sum)
 - 모든 입력에 각각의 가중치를 곱한 뒤, 모두 더한 값 + 편향
 - $(x_1 * w_1) + (x_2 * w_2) + \dots + b$ 와 같이 “선형 회귀”와 유사
 - 활성화 함수(Activation Function)
 - 입력된 신호를 받아, 그 신호를 처리하여 출력 신호를 결정하는 함수
 - 즉, “활성화”시켜 전달할지 말지를 결정하는 스위치



✓ 퍼셉트론(Perceptron) (2/3)

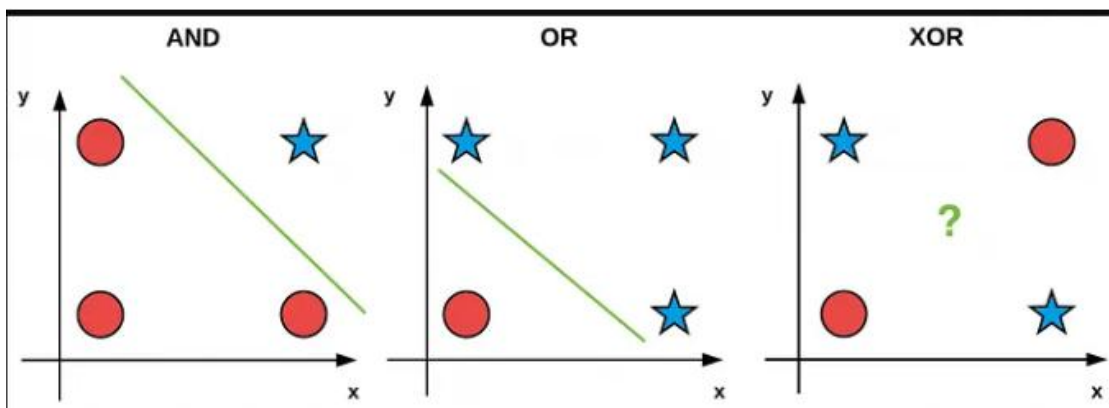
- 퍼셉트론은 결국 데이터 사이에 “직선” 하나를 그어서 두 그룹을 나누는 것과 같음
 - 가중합(z) = $(x_1 * w_1) + (x_2 * w_2) + \dots + b$
 - z 가 0보다 크면 1, 0보다 작으면 0으로 분류
 - 즉 $z = (x_1 * w_1) + (x_2 * w_2) + \dots + b$ 가 되는데, 수학적 이유로 1차 함수를 의미하게 됨
- 그리고 퍼셉트론은 “선형 분리 가능한 문제”를 잘 해결했음
 - 선형 분리 가능한 문제
 - 데이터들을 하나의 직선으로 완벽하게 나눌 수 있는 문제
 - 예1) 논리 게이트(AND, OR)
 - 예2) 간단한 분류 문제(스팸 메일 분류, 암 진단)



- 기계가 데이터를 보고 스스로 경계선(규칙)을 찾아낼 수 있다는 가능성을 보여준 엄청난 알고리즘

✓ 퍼셉트론(Perceptron) (3/3)

- “XOR(배타적 논리합) 문제”를 해결할 수 없는 치명적인 문제를 가지고 있음
 - XOR: 두 입력이 서로 다를 때만 1(참), 같으면 0(거짓)
 - "가격은 싼데(T) 리뷰는 좋거나(T), 가격은 비싼데(F) 리뷰는 안 좋은 곳(F)은 숨은 맛집이 아니다(F)"

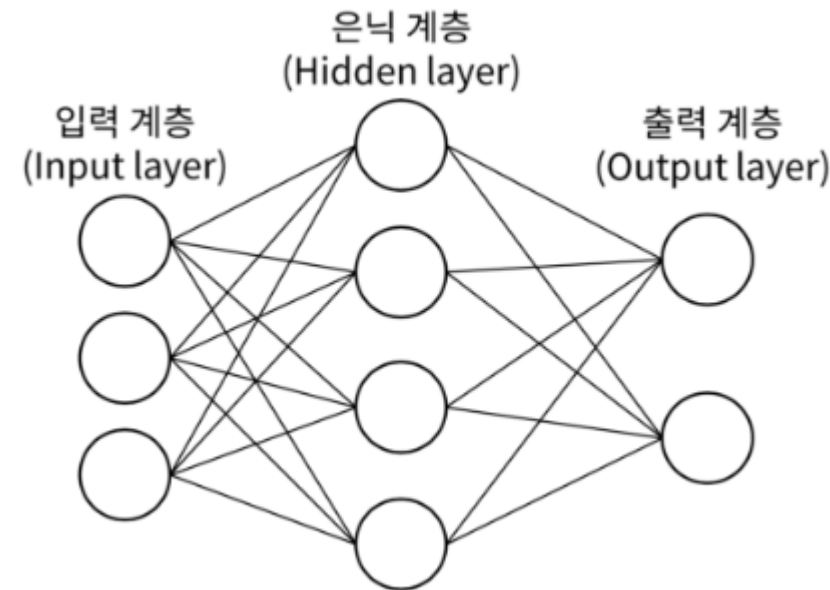


- XOR 문제 때문에 퍼셉트론은 근본적인 한계가 있다는 비판과 함께 **AI의 겨울이 도래...**
- 하지만, 퍼셉트론을 여러 층으로 쌓아 직선이 하나가 아닌, 여러 개의 직선을 조합하여 XOR 문제를 해결!!!!
 - **인공 신경망(Deep Neural Network)**의 시작

Neural Network

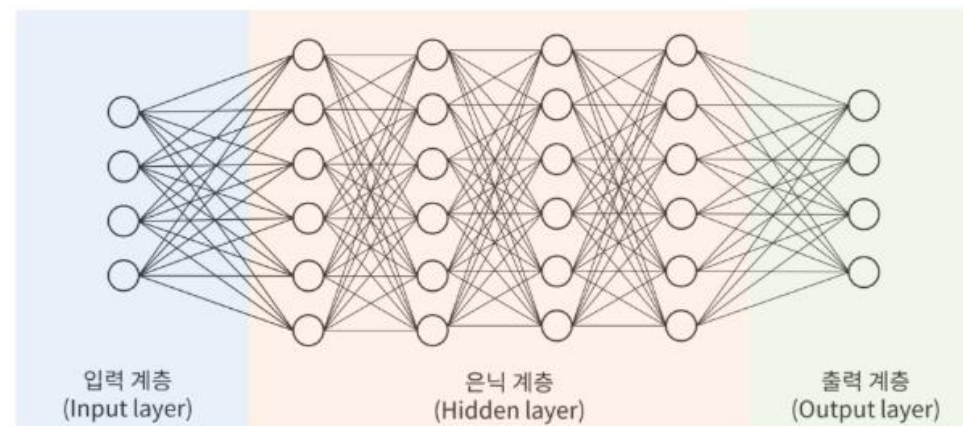
✓ 얇은 신경망(Shallow Neural Network, SNN)

- 초기 인공 신경망의 형태
- 얇은 신경망(Shallow Neural Network)
 - 입력/은닉/출력의 3가지 계층으로 이루어짐
 - 은닉층(Hidden Layer)가 딱 하나만 있는 구조
 - 만능근사이론에 의해서 이론적으로는 모든 문제를 해결할 수 있음
- 만능근사이론
 - **이론적으로** 은닉층에 뉴런이 충분히 많다면
단 하나의 은닉층만으로도 세상의 거의 모든 연속적인 함수를 흉내낼 수 있음
 - 하지만, 은닉층을 터무니 없이 넓게 만들어야했고(계산량 매우 큼), 단계별/조합적 학습이 불가능
- 그래서 뉴런을 넓게 만드는 것이 아니라, 깊게 쌓아올리는 아이디어가 “**심층 신경망**”



✓ 심층 신경망(Deep Neural Network, DNN) ≈ 딥러닝

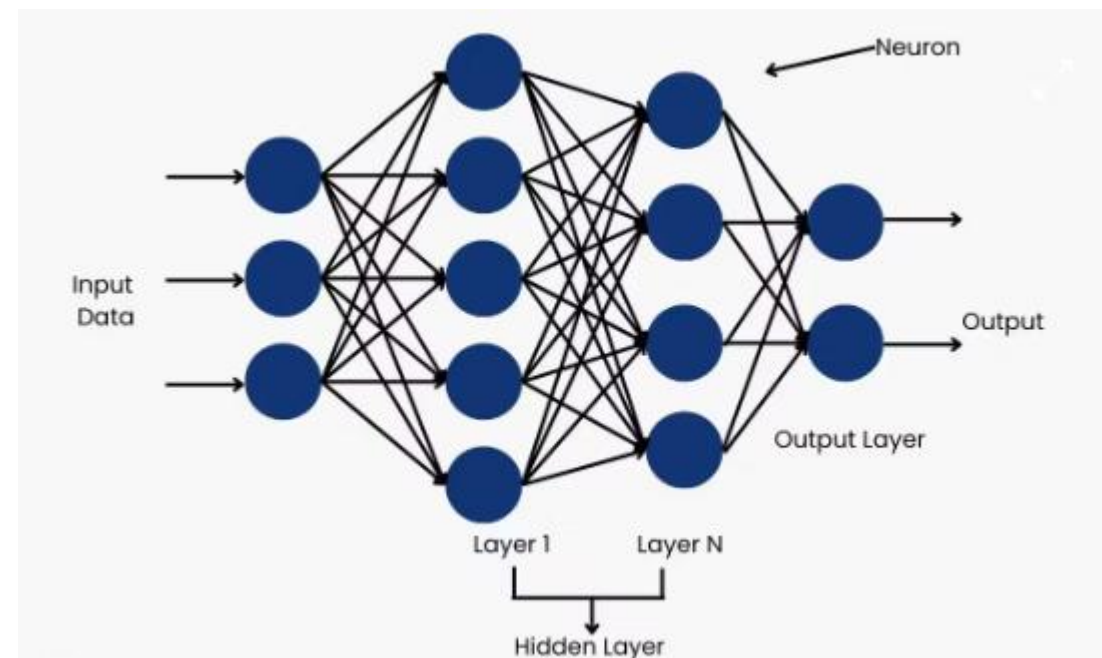
- 신경망을 얹으려가 아닌, 깊게(Deep) 쌓는 방식
- 여러 개의 은닉층을 가진 인공 신경망
- 얇은 신경망과 비교
 - 얇은 신경망: 뉴런 12개 $\Rightarrow$ 조합 12개(재사용불가능)
 - 깊은 신경망: 뉴런 6개 * 6개 $\Rightarrow$ 조합 36개
- 더 복잡하고 고차원적인 특징을 학습 가능
- 사람이 특징을 알려주지 않아도, 복잡한 패턴을 스스로 발견
- 정리하자면, **심층 신경망**이라는 모델을 사용하여 **딥러닝**을 진행!



MLP

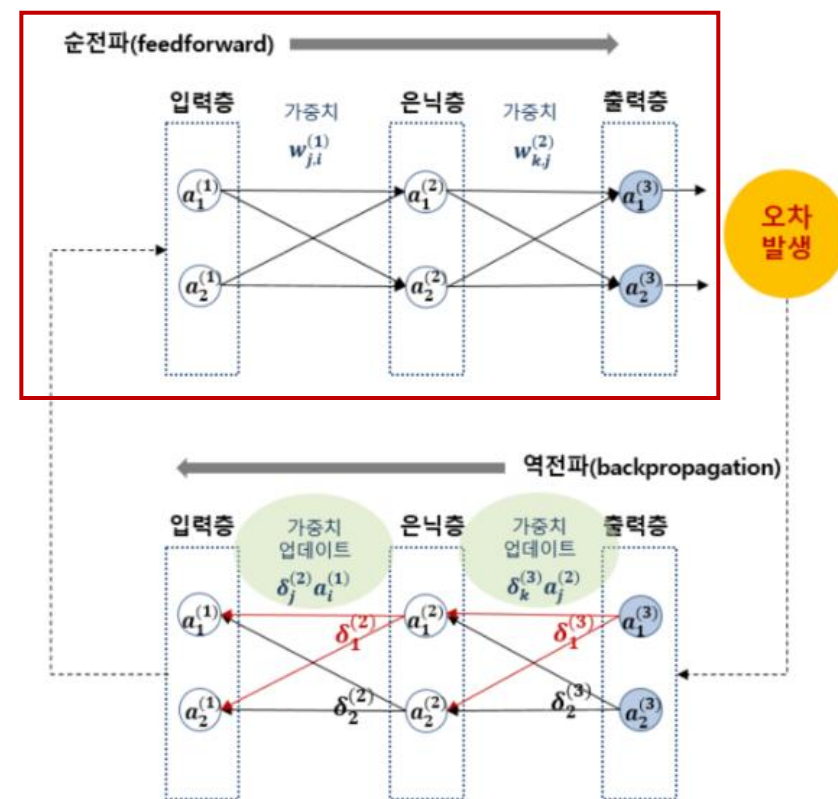
✓ 다중 퍼셉트론(Multi-Layer Perceptron, MLP)

- 인공 신경망(Neural Network) 의 한 종류로, 퍼셉트론을 여러 층으로 쌓은 모델
- 입력층과 출력층 사이에 1개 이상의 은닉층을 가짐
- MLP의 학습 과정
 1. 순전파(Forward Propagation)
 2. 손실 함수 계산(Loss Function)
 3. 역전파(Back Propagation) 및 가중치 업데이트 (Optimization)
 4. 1~3번 과정을 오차가 최소화될 때까지 반복 후 종료 (Training 완료)



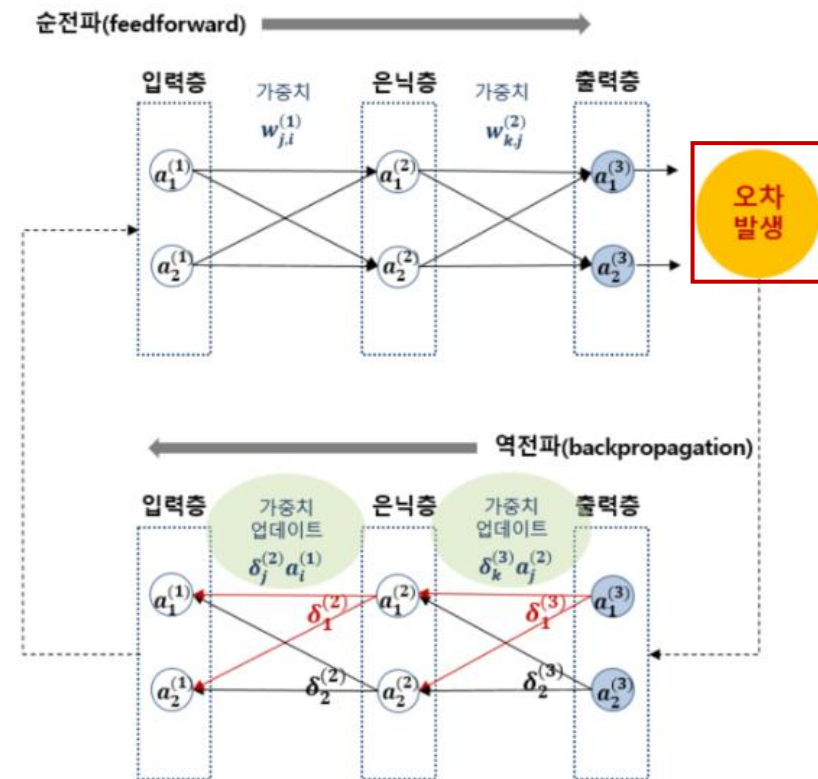
✓ MLP 학습 과정 - 순전파(Forward Propagation)

- 데이터가 입력층에서 시작해서 여러 은닉층을 거쳐 출력층까지, 앞에서 뒤로 흘러가면서 계산되는 과정
- 순전파 과정
 1. 초기에 각 뉴런(퍼셉트론)들은 무작위 가중치를 갖음
 2. 데이터(특징 벡터)가 연결된 입력층으로 들어감
 3. 각 뉴런은 입력값과 무작위로 초기화된 가중치와 편향을 이용해 계산하고, 활성화 함수(RELU)를 통과시켜 결과를 다음 층으로 전달
 4. 마지막 출력층까지 반복되어 최종 예측값을 출력
- 초기에는 가중치가 랜덤이기 때문에, 매우 엉터리 결과가 나옴



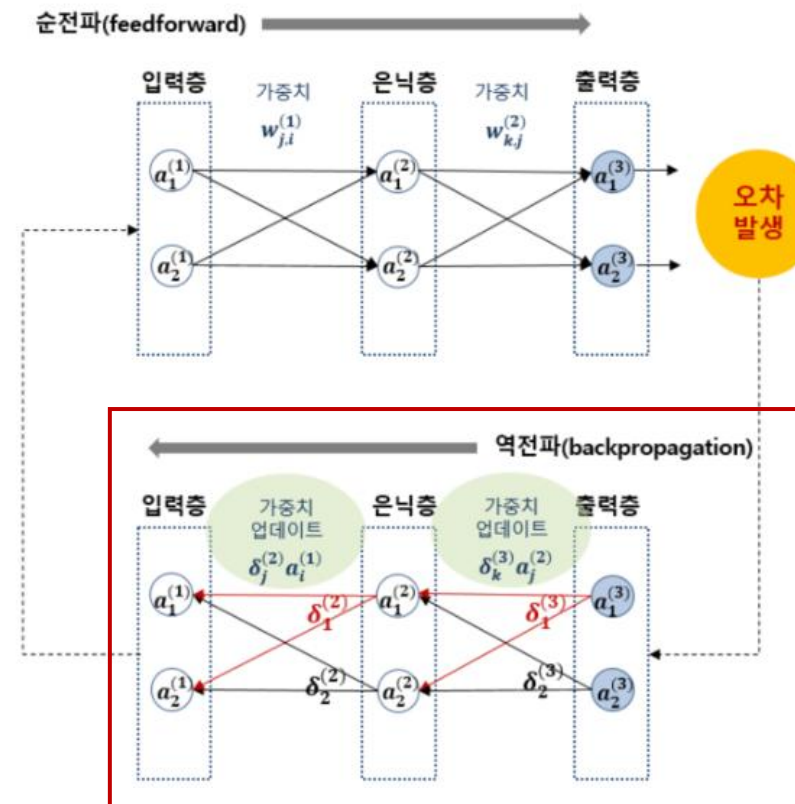
✓ MLP 학습 과정 - 손실 함수 계산(Loss Function)

- 순전파 과정에서 나온 결과를 실제 정답과 비교해서 손실(Loss)/오차(Error)를 구함
- 구하는 방법
 - 회귀 (평균 제곱 오차, 각 오차의 제곱의 합)
 - 분류 (교차 엔트로피, 모델이 강하게 확신하고 틀리면 강한 벌점)
 - 비지도(클러스터링 후 떨어진 거리의 총합을 줄이기, 심플 버전)



✓ MLP 학습 과정 - 역전파(Back Propagation)

- 손실 값을 가지고, 출력층에서 거꾸로 계산하면서 가중치를 업데이트하는 과정
- 역전파 과정(쉬움버전)
 - 발생한 오차를 가지고, 이전 층으로 거꾸로 추적하면서 각 가중치가 오차에 준 영향을 구함
 - 계산된 영향력을 바탕으로 오차를 줄이는 방향으로 각 가중치를 조금씩 업데이트
 - 첫 은닉층까지 반복 후 다시 순전파 반복
- 이해가 어려우면 아래 링크를 참고하는 것도 추천
 - <https://www.youtube.com/watch?v=tkH7KgLZcOE>



✓ [심화] 역전파 진행 과정 (1/2)

책임 신호: 뉴런의 기울기=영향력

1. 예측값과 정답 차이에서 하나의 "최종 오차(L)"가 발생
2. "최종 오차(L)"가 뒤로 전파되면서, 각 뉴런은 최종 오차(L)에 대한 각기 다른 '책임 신호'를 전달받는다.
 - 책임 신호는 뉴런의 출력값이 최종 오차(L)에 얼마나 영향을 미쳤는지를 나타냄
 - 수학적으로는 "최종 오차(L)"를 "각 뉴런의 출력"으로 편미분
3. 뉴런 내부의 각 (가중치(w) $\times$ 입력데이터(x))은 책임 신호에 각기 다른 영향을 미쳤고, 이 영향력에 맞춰서 가중치를 수정해야 한다.
 - 영향력이 크면 가중치 수정도 더 많이 해야 한다.

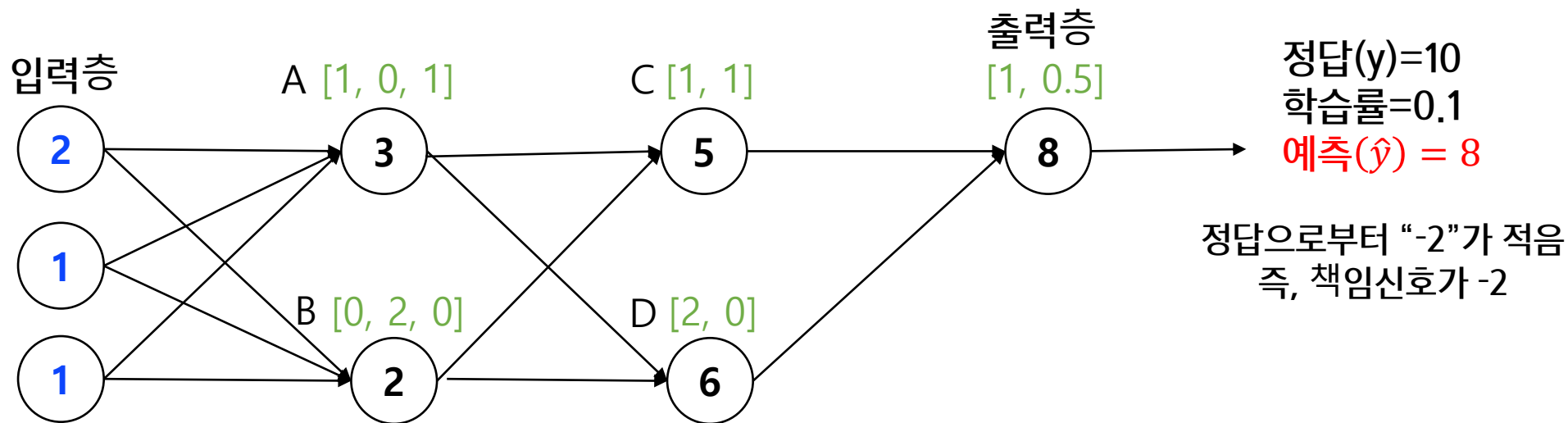
✓ [심화] 역전파 진행 과정 (2/2)

책임 신호: 뉴런의 기울기=영향력

최종 책임 신호: 가중치의 기울기

3. 뉴런 내부의 각 (가중치(w) $\times$ 입력데이터(x))은 **책임 신호**에 각기 다른 영향을 미쳤고, 이 영향력에 맞춰서 가중치를 수정해야 한다.
 - 영향력이 크면 가중치 수정도 더 많이 해야 한다.
4. (가중치(w)*입력데이터(x))의 영향력에서 실질적으로 **책임 신호**에 영향을 주는 부분은 "입력데이터(x)"이다.
 - 우리가 궁금한 건 “가중치를 조절했을 때의 결과가 얼마나 변할까? 이고, 이 가중치의 영향력을 결정하는 것은 입력데이터의 몫
 - 예를 들어) 방정식 $b = 7 * a$ 가 있을 때, a 의 영향력을 보고 싶으면 결국 a 를 고정하고 7 을 쳐다봐야 한다.
 - 그리고 이것이 미분(편미분)이다...
5. 그렇기에 기존에 있던 가중치(w)는 신경쓰지 말고, 입력데이터(x)과 **책임 신호**를 곱해서 **최종 책임 신호**(가중치의 기울기)를 계산 후 학습률과 곱해서 각 가중치 값을 업데이트 한다.
 - 입력데이터가 $\uparrow \rightarrow$ 영향력이 $\uparrow \rightarrow$ 가중치의 업데이트 변화량도 커야 하고 $\rightarrow$ 학습률이란 곱해질 **최종 책임 신호** 값도 커야 한다.

✓ [그림] 역전파 진행 과정 - 순전파



$$A = (2 \times 1) + (1 \times 0) + (1 \times 1) = 3$$

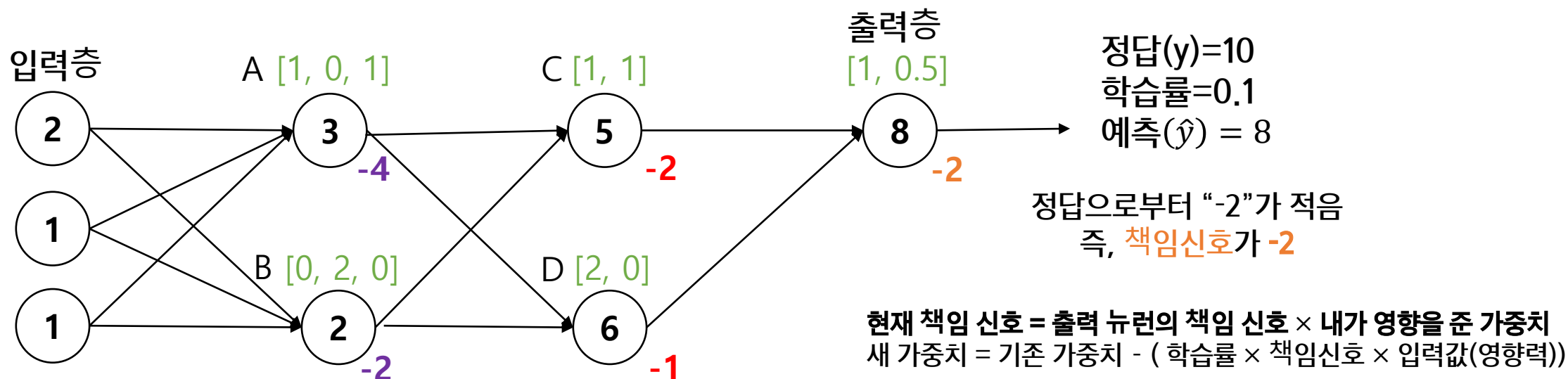
$$B = (2 \times 0) + (1 \times 2) + (1 \times 0) = 2$$

$$C = (3 \times 1) + (2 \times 1) = 5$$

$$D = (3 \times 2) + (2 \times 0) = 6$$

$$\text{출력층} = (5 \times 1) + (6 \times 0.5) = 8$$

✓ [그림] 역전파 진행 과정 - 역전파 (1/3)



$$\text{출력층 책임신호} = -2$$

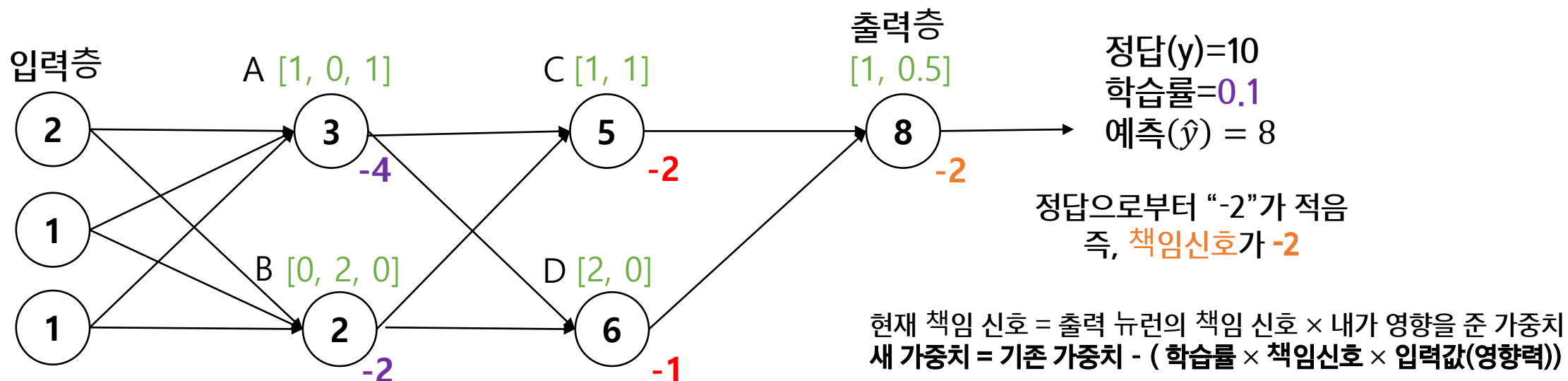
$$\text{C 책임신호} = (-2 \times 1) = -2$$

$$\text{D 책임신호} = (-2 \times 0.5) = -1$$

$$\text{A 책임신호} = (-2 \times 1) + (-2 \times 1) = -4$$

$$\text{B 책임신호} = (-1 \times 2) + (-1 \times 0) = -2$$

✓ [그림] 역전파 진행 과정 - 역전파 (2/3)



출력층 가중치 업데이트

$$w_1 = 1 - (0.1 \times (-2) \times 5) = 2$$

$$w_2 = 0.5 - (0.1 \times (-2) \times 6) = 1.7$$

C 뉴런 가중치 업데이트

$$w_1 = 1 - (0.1 \times (-2) \times 3) = 1.6$$

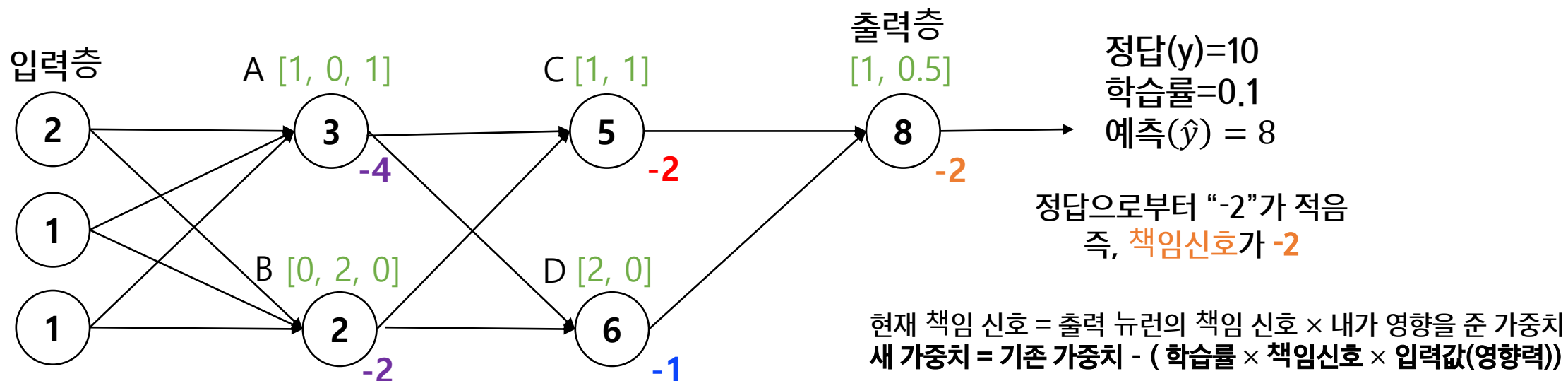
$$w_2 = 1 - (0.1 \times (-2) \times 2) = 1.4$$

D 뉴런 가중치 업데이트

$$w_1 = 2 - (0.1 \times (-1) \times 3) = 2.3$$

$$w_2 = 0 - (0.1 \times (-1) \times 2) = 0.2$$

✓ [그림] 역전파 진행 과정 - 역전파 (3/3)



A 뉴런 가중치 업데이트

$$w_1 = 1 - (0.1 \times (-4) \times 2) = 1.8$$

$$w_2 = 0 - (0.1 \times (-4) \times 1) = 0.4$$

$$w_3 = 1 - (0.1 \times (-4) \times 1) = 1.4$$

B 뉴런 가중치 업데이트

$$w_1 = 0 - (0.1 \times (-2) \times 2) = 0.4$$

$$w_2 = 2 - (0.1 \times (-2) \times 1) = 2.2$$

$$w_3 = 0 - (0.1 \times (-2) \times 1) = 0.2$$

✓ MLP 학습 과정 - Training 완료

- 순전파 → 오차 계산 → 역전파 과정을 언제까지 진행할까?
- 1. 정해진 학습 횟수(Epoch)에 도달
 - 언제 끝날지 예측이 가능하고, 구현이 간단
 - 너무 적게 반복하면 과소적합, 너무 많이 반복하면 과적합
- 2. 조기 종료(Early Stopping) - 가장 많이 쓰이는 방법
 1. 훈련/검증/테스트 데이터로 나눈 후, 훈련 데이터로 가중치를 업데이트
 2. 1Epoch가 끝날 때마다, 검증 데이터로 오차를 확인
 3. 훈련이 지속될수록 오차가 줄어들다가, 어느 순간 과적합으로 인해 오차가 떨어지기 시작
 4. 여기서 연속으로 N번 이상 오차의 변화가 없거나 떨어지면 학습을 중단
 - 모델이 가장 성능이 좋았던 '최적의 순간'에 학습을 멈추게 해줌

MLP 성능 향상

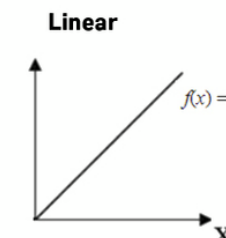
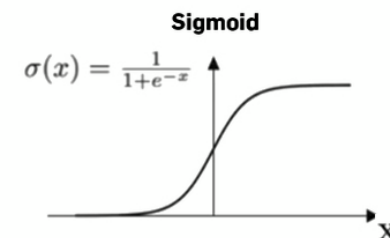
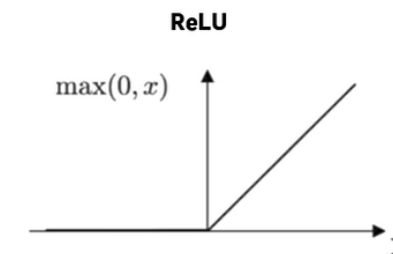
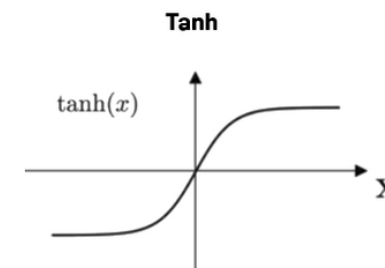
✓ MLP 성능 향상시키는 기술들

- 우리가 배운 기본 MLP은 실제 복잡한 환경에서 제대로 동작하지 않음
- 대표적인 성능향상 기술
 1. 더 좋은 활성화 함수(ReLU)
 2. 더 효율적인 학습을 위한 옵티마이저(Optimizer)
 3. 과적합(Overfitting)을 막기 위한 규제(Regularization)

활성화 함수(ReLU)

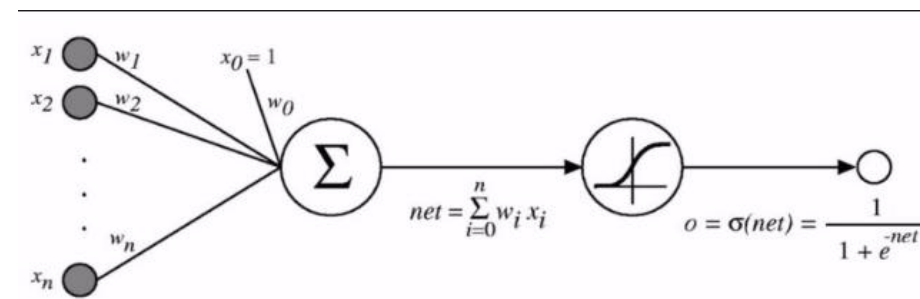
✓ 활성화 함수

- 입력된 신호를 받아, 그 신호를 처리하여 출력 신호를 결정하는 함수
- 즉, “활성화”시켜 전달할지 말지를 결정하는 스위치
- 활성화 함수가 필요한 이유
 - 활성화 함수가 없다면, 신경망은 여러 층을 아무리 깊게 쌓아도 결국 거대한 선형 함수와 다를 바가 없음
 - 활성화 함수는 이런 선형 함수를 훨씬 복잡한 패턴을 학습할 수 있게 함
- 대표적인 활성화 함수
 - Linear : 입력값을 그대로 출력
 - Sigmoid : 0 ~ 1 사이 값으로 출력
 - ReLU : 0보다 작은 값은 0, 0보다 크면 그대로 출력



✓ MLP의 초기 활성화 함수

- 초기 신경망은 시그모이드(Sigmoid) 함수를 활성화 함수로 사용
- 값을 0~1사이로 만들어주니 좋아보였으나, 역전파 과정에서 “**기울기 소실 문제**(Vanishing Gradient Problem)”가 발생
- **기울기 소실 문제**(Vanishing Gradient Problem)
 - 시그모이드 함수는 역전파 과정에서 ‘오차신호’를 뒤로 전달할 때마다 미분하는 과정에서 신호의 세기를 아주 약하게 만듦
 - 층을 거꾸로 거슬러 올라갈 때마다, 이 작은 값들이 계속 곱해짐
 - 결국 입력층 근처로 갈수록 0에 가깝게 사라져 버림(Vanishing)
 - 앞쪽 층에 있는 뉴런들은 오차에 대한 피드백을 전혀 받지 못해 학습이 멈춰버림



- 기울기 소실 문제로 인해 수십년 전에 등장한 MLP이지만, 빛을 보지 못함
 - 하지만, 이때 등장한 것이 **ReLU 활성화 함수**

✓ ReLU 활성화 함수

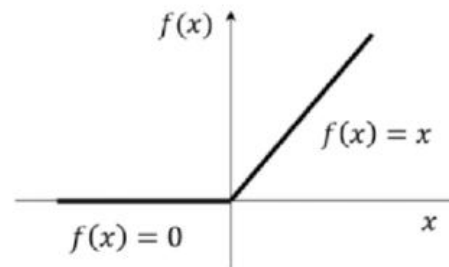
- Rectified Linear Unit
- 입력값이 0보다 작으면 0, 0보다 크면 값을 그대로 출력
- ReLU가 MLP의 성능을 향상 시킨 이유

1. 기울기 소실 문제 해결

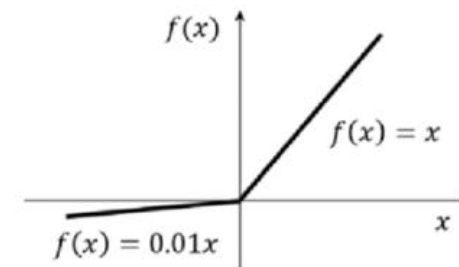
- 입력값이 0보다 클 때, ReLU의 기울기는 무조건 1
- 역전파 과정에서 신호의 세기가 줄어들지 않고 앞쪽 층까지 온전히 전달됨
- but 지속적인 음수만 들어오는 경우 생기는 DeadLeLU 문제가 있으며, 이를 해결하기 위해 LeakyReLU를 활용
 - Leaky ReLU: 음수 구간을 0이 아니라 아주 완만한 경사 (0.01)로 바꿈

2. 매우 빠른 계산 속도

- 시그모이드 함수와 같이 복잡한 지수 함수 계산이 필요없음



ReLU activation function



LeakyReLU activation function

옴티마이저(Adam)

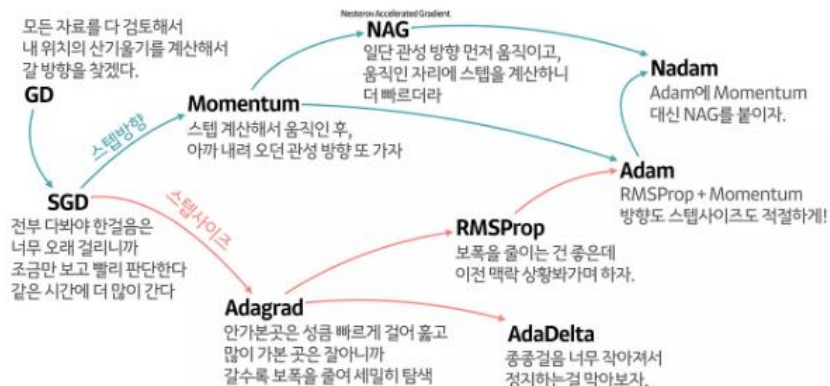
✓ 옵티마이저(Optimizer)

- 머신러닝에서 손실 함수의 값이 최소가 되는 최적의 가중치를 찾기 위해 사용되는 알고리즘
- 가장 기본적인 옵티마이저에는 “경사하강법(Gradient Descent, GD)”
- 경사 하강법의 치명적인 단점
 - 학습률(Learning Rate)에 따른 다양한 문제 발생
 - 학습률이 높은 경우: 손실 함수 값이 줄어들지 않고 오히려 커지는 “오버슈팅(overshooting)” 발생
 - 학습률이 낮은 경우: 학습이 너무 느리고, 지역 최저점(Local Minima)에 빠지면 학습을 멈추거나 매우 느려짐

이런 경사하강법의 단점을 해결한 방법이

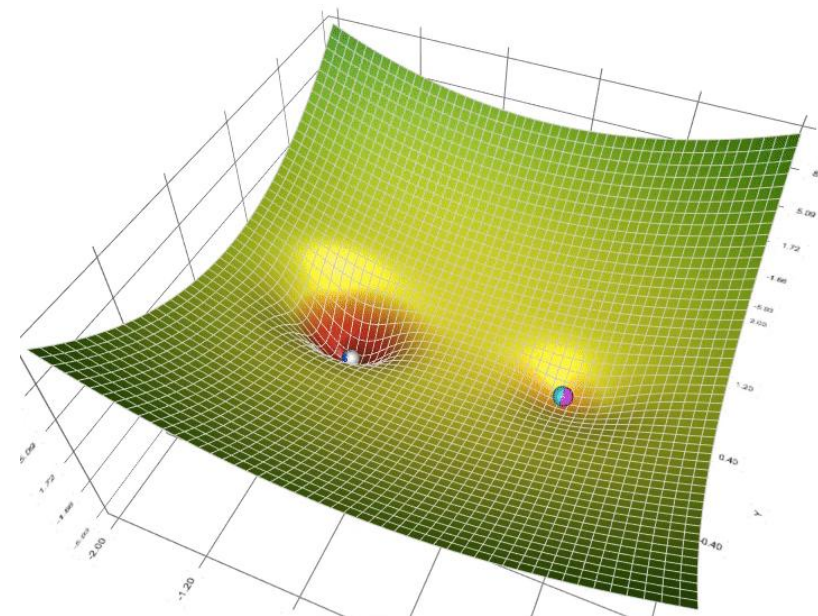
➤ Adam(Adaptive Moment Estimation)

산 내려오는 작은 오솔길 찾기(Optimizer)의 발달 계보



✓ Adam(Adaptive Moment Estimation) (1/3)

- 현재 딥러닝 모델의 80% 이상이 사용하는 사실상 표준 옵티마이저
- 이름처럼 “적응적(Adaptive)으로 학습률(Learning Rate)을 조절하는 것이 핵심”
- Adam의 아이디어
 1. 관성(Momentum) 도입
 2. 적응적 학습률(Adaptive Learning Rate) 도입



Animation of 5 gradient descent methods on a surface: gradient descent (cyan), momentum (magenta), AdaGrad (white), RMSProp (green), Adam (blue). Left well is the global minimum; right well is a local minimum.

✓ Adam(Adaptive Moment Estimation) (2/3)

1. 관성(Momentum) 도입

- 현재 위치의 기울기만 보고 움직이는 경사하강법과는 다르게, 이전까지 이동해온 '속도와 방향(관성)'을 기억
- 양쪽 벽으로 요동치는 움직임을 상쇄하고, 관성을 이용해 부드럽게 내려가게 만듦
- 잠시 '지역 최저점'에 빠지더라도, 관성의 힘으로 지역 최저점을 빠져나올 수 있음

2. 적응적 학습률(Adaptive Learning Rate) 도입

- 가중치마다 "고유한 학습률"을 부여 하여, 최적의 학습 속도를 유지하여 효율적으로 낮은 손실 지점에 도달
- 많이 움직였던 가중치에 대해서는 학습률을 작게 만들어서, 적게 움직이도록 함
- 적게 움직였던 가중치에 대해서는 학습률을 크게 만들어서, 빠르게 학습을 진행

✓ Adam(Adaptive Moment Estimation) (3/3)

1. 하이퍼파라미터 튜닝이 거의 필요 없음

- 기본 설정값(학습률: 0.001)만 써도 웬만한 문제는 거의 다 해결

2. 매우 빠름

- 관성과 적응형 학습률을 동시에 사용해서 훨씬 빠르게 골짜기에도착

3. 안정성

- 데이터가 들쭉날쭉하거나 노이즈가 많아도 흔들리지 않음

- 웬만하면 Adam 부터 시도해보자!

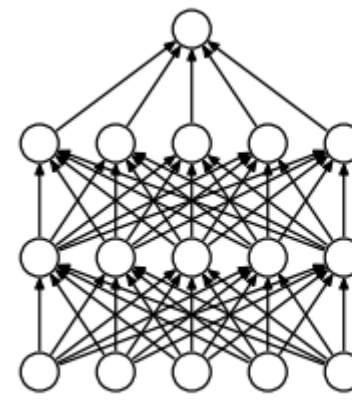
규제(Dropout)

✓ 규제(Regularization)

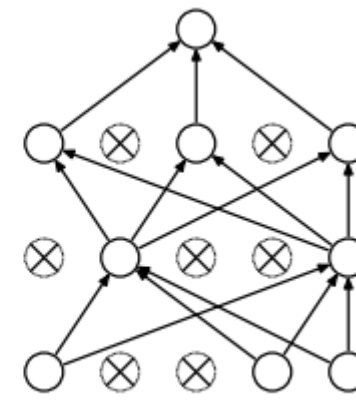
- 모델의 과적합(overfitting)을 막기 위해 제약을 거는 것
- 대표적인 규제 방법
 1. L1 규제 (Lasso)
 - 일부 가중치를 0으로 만들어 제거함으로써 복잡도를 감소시며, 불필요한 특성이 많은 경우 사용
 - ex) 시험에 안 나올 것 같은 단원은 쿨하게 버린다..
 2. L2 규제 (Ridge) $\approx$ 가중치 감쇠 (Weight Decay)
 - 가중치를 0에 가깝게 만들지만 완전히 0으로 만들지는 않음
 - 어느 한 과목만 100점 맞는 것이 아닌, 모든 과목을 골고루 80점 이상 맞도록 힘을 분산하는 것
 3. **드롭아웃** (Dropout)
 - 학습 시 각 뉴런을 랜덤하게 일시적으로 학습시키지 않는 것

✓ 드롭아웃(Dropout)

- 학습 시 각 뉴런을 랜덤하게 일시적으로 OFF 시키는 것
- 훈련할 때, 매번 순전파를 진행할 때마다 각 은닉층의 뉴런을 일정 확률(보통 0.5)로 랜덤하게 선택하여 일시적으로 OFF
- 장점
 - 특정 뉴런이 모든 것을 학습하거나(편향), 다른 뉴런에게 지나치게 의존하는 것을 방지
 - 매번 다른 네트워크(약간씩 다른 구조)를 훈련시키는 것과 같은 효과를 냄 (양상블 효과)
- 테스트 시에는 모든 뉴런을 사용해서 테스트를 진행함



(a) Standard Neural Net



(b) After applying dropout.

5. CNN

• INDEX

- CNN의 등장
- CNN
- 다양한 CNN 모델

CNN의 등장

✓ 이미지 데이터의 특징

- 이미지는 픽셀들이 2차원 공간 구조를 가진 데이터 (이미지 추가)
- 컴퓨터는 이미지를 픽셀값의 집합으로 받아들임(이미지 추가)

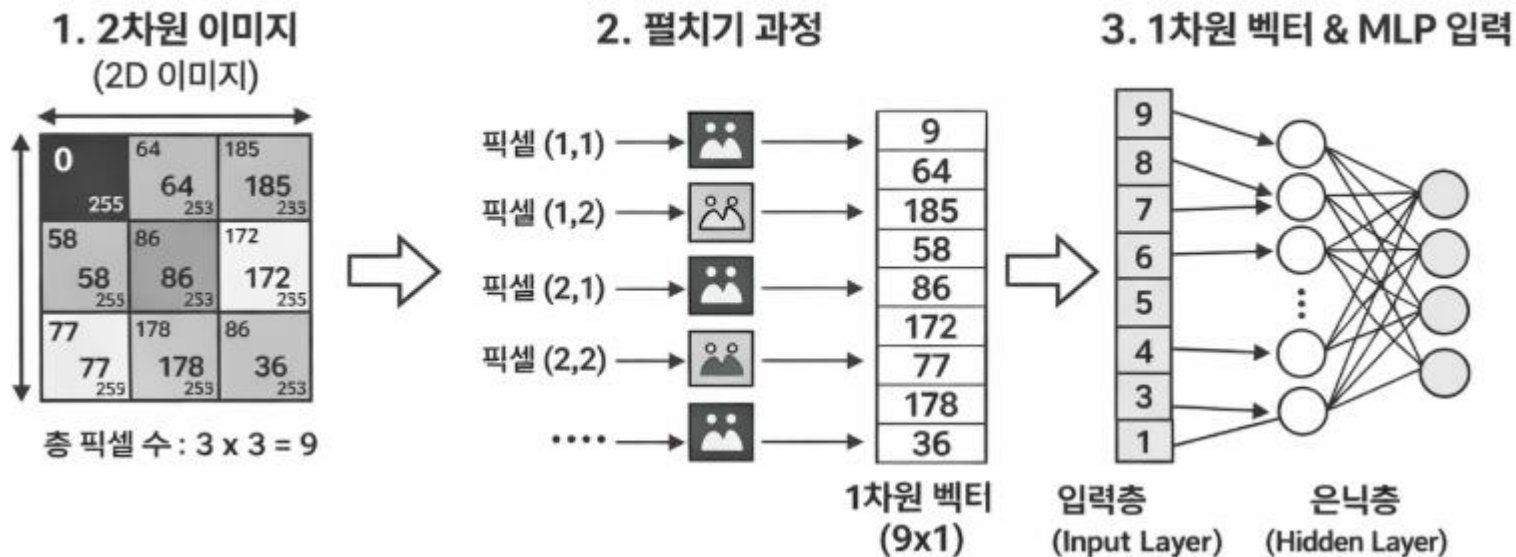


컴퓨터는

131	152	152	179	133	179
128	160	179	178	184	185
113	151	171	138	179	188

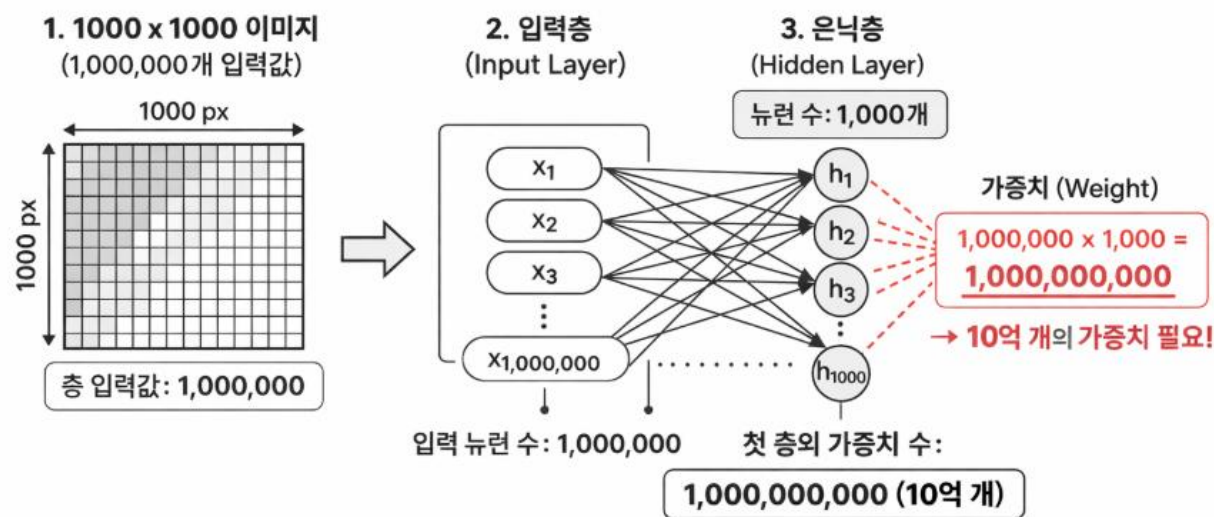
✓ MLP의 한계 (1/2)

- MLP는 데이터를 1차원 배열로 입력 받음
- 이미지를 MLP로 학습 시, 2차원 배열 형태의 숫자로 변환된 이미지를 1차원 벡터로 펼치는 Flatten 과정을 거쳐야 함
- 이로 인해, 인접한 픽셀끼리 밀접한 관계(선, 면, 형상)가 모두 사라져서 특징을 읽기 어려워짐



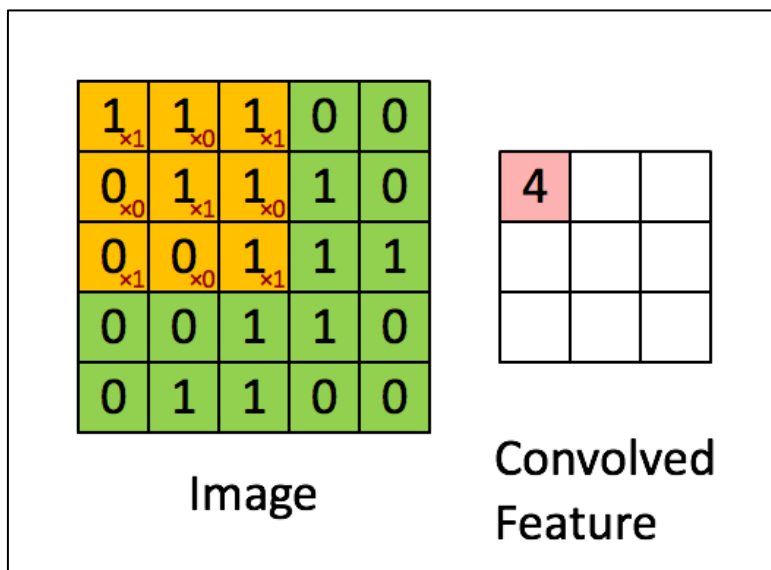
✓ MLP의 한계 (2/2)

- MLP는 이전 층의 모든 뉴런이 다음 층의 모든 뉴런과 연결되는 완전 연결 구조임
- 그래서 고해상도의 이미지를 입력하게 되면 많은 가중치로 인해 학습 속도를 느리게 하고, 과적합 위험이 있음
- 예를 들어,
 - 1000 * 1000 해상도의 이미지는 100만개의 입력값을 가짐
 - 은닉층 뉴런이 1,000 개라면 총 100만 * 1,000 으로 10억개의 가중치가 필요함



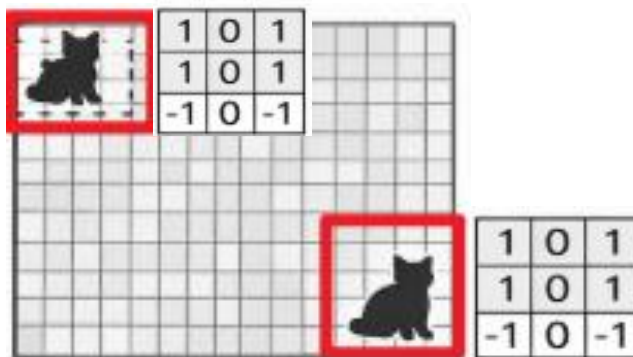
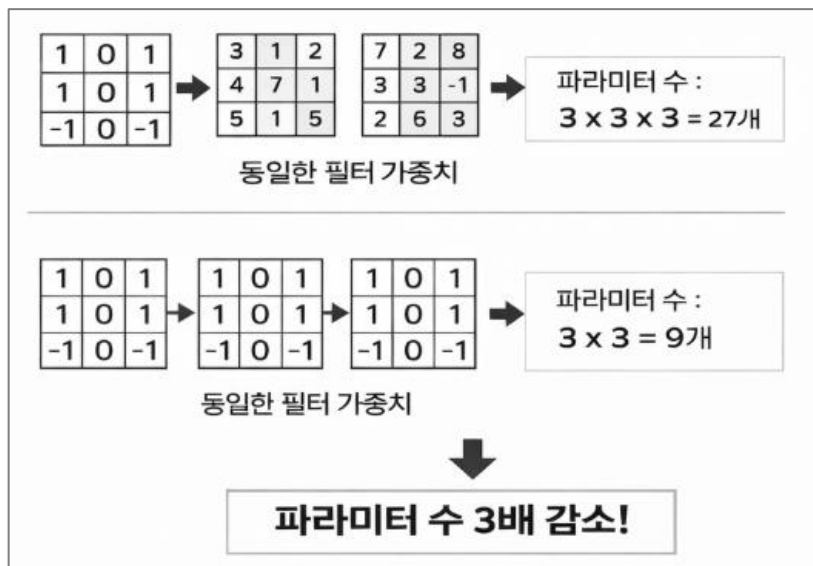
✓ CNN의 등장 (1/3)

- MLP의 한계
 - 2차원 배열 형태의 숫자로 변환된 이미지를 1차원 벡터로 펼치는 Flatten 과정
- CNN의 극복 방안
 - 이미지를 펼치는 대신, 작은 필터(Filter/Kernel)를 사용해 **이미지를 훑으며 특징을 찾아냄**



✓ CNN의 등장 (2/3)

- MLP의 한계
 - MLP는 이전 층의 모든 뉴런이 다음 층의 모든 뉴런과 연결되는 완전 연결 구조
- CNN의 극복 방안
 - 이미지 전체에 **동일한 필터 가중치**를 적용해 파라미터 수를 줄임
 - 동일한 가중치의 필터가 이미지 전체를 훑기 때문에, 특징이 어디에 있든 똑같이 추출할 수 있음



✓ CNN의 등장 (3/3)

- CNN은 이미지의 구조를 유지하면서 효율적으로 학습하기 위해서 등장
- 이미지를 학습한다는건 ?
 - 단순히 픽셀값을 외우는 것이 아님
 - 데이터를 요약하여 대상을 설명할 수 있는 특징을 추출하는 것이 목표
 - 저수준(선, 점, 색상) → 고수준(눈, 코, 창문과 같은 형태) → 최종 (특징을 조합해 고유의 특징맵을 만드는 것)
 - 우리는 어떤 사진이 들어와도 그 대상의 특징을 가장 잘 끄집어낼 수 있는 “필터”를 찾아내는 것이 목표!

CNN

✓ CNN의 핵심 아이디어

- 작은 필터로 국소 영역을 본다
- 같은 필터를 이미지 전체에 반복 적용한다.
- 특징을 단계적으로 추출한다.

0	0	0	0	0	0	0
0	60	113	56	139	85	0
0	73	121	54	84	128	0
0	131	99	70	129	127	0
0	80	57	115	69	134	0
0	104	126	123	95	130	0
0	0	0	0	0	0	0

Kernel

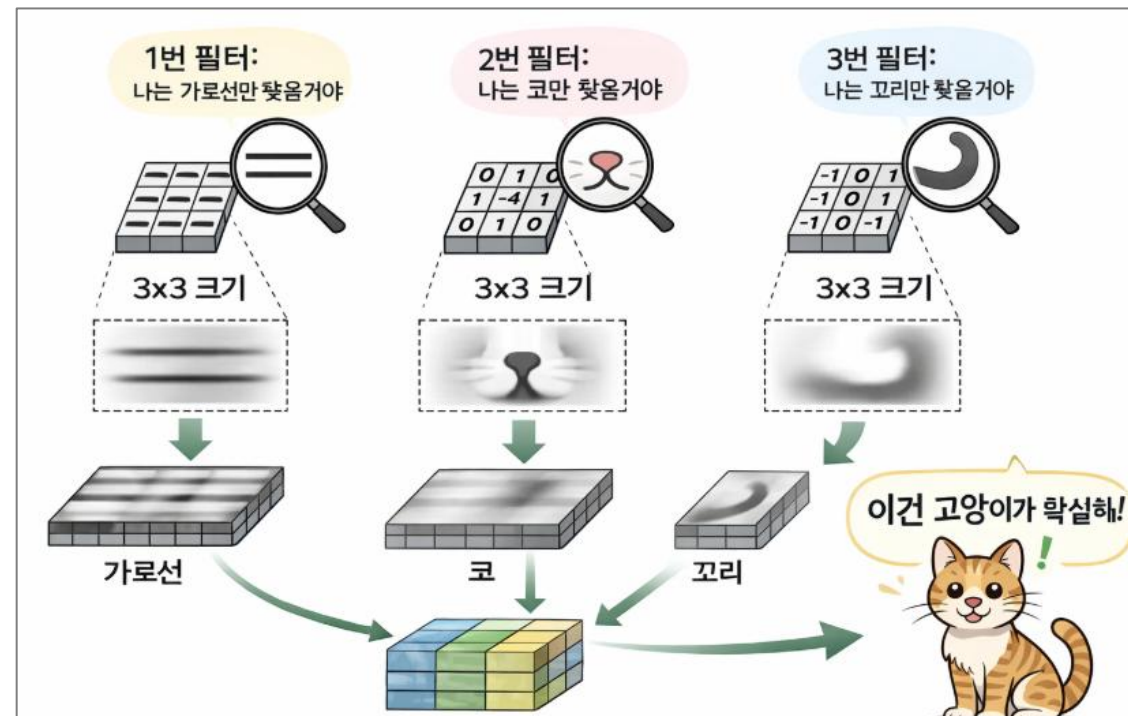
0	-1	0
-1	5	-1
0	-1	0

114				

✓ 커널/필터 (1/2)

- 이미지의 특징(세로선, 색상 변화 등)을 찾아내기 위한 작은 행렬
- 각 셀은 가중치로 채워져 있음
- 필터마다 찾고자 하는 특징이 다름
- 예를 들어)

- 1번 필터: 가로선만 찾을거야,
- 2번 필터: 코만 찾을거야
- 3번 필터: 꼬리만 찾을거야
- 이런 필터들이 층이 중첩되면서 정보들이 조합되고
“이건 고양이가 확실해”라고 판단하게 되는 것

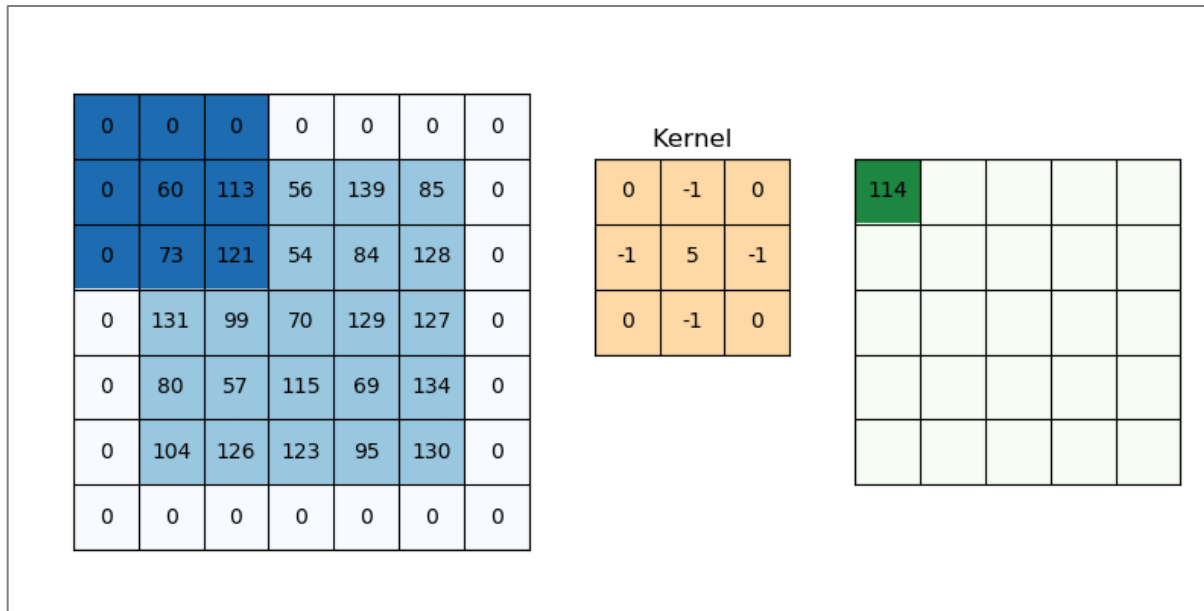


✓ 커널/필터 (2/2)

- 정확히는 커널과 필터는 다른 용어
 - 커널:(Kemel): 가장 작은 단위의 2차원 가중치 행렬이며, 입력 데이터의 “단일 채널”에 적용되어 특징을 추출
 - 필터:(Filter) : 여러 개의 커널이 모여 있는 3차원 텐서 묶음이며, 입력 데이터의 “모든 채널”을 동시에 처리
 - RGB 컬러 이미지(3채널)을 처리하기 위해서는 빨강, 초록, 파랑을 각각 담당하는 3개의 커널을 가져야 함
 - 왜 혼용해서 사용하나요? 입문 자료에서는 설명을 단순화하기 위해 채널이 1개인 흑백 이미지를 예로 들기 때문
- CNN은 커널/필터가 이미지 위를 일정한 간격으로 겹쳐 지나가며 수학적 연산을 수행하여 특징을 추출!
- 이 수학적 연산이 **“합성곱 연산”**

✓ 합성곱(Convolution) 연산 (1/2)

- 필터를 이미지 위에 겹쳐 놓고 곱한 뒤 더해 하나의 값을 만드는 연산
- 필터가 원하는 특징이 어디에 있는지 찾는 과정
- 그리고 이러한 수학적 연산의 결과값이 모여서 새로운 2차원 배열이 생기는데 이것이 “**특징맵**”



합성곱 연산 과정

✓ 합성곱(Convolution) 연산 (2/2)

- PyTorch에서는 `nn.Conv2d`를 사용하여 2D 이미지에 대한 합성곱 계층을 정의
- `nn.Conv2d(in_channels=3, out_channels=6, kernel_size=5)`의 의미:
 - `in_channels=3`: 입력 이미지의 채널 수 (RGB)
 - `out_channels=6`: 사용할 필터의 개수. 즉, 생성될 특징 맵의 개수
 - `kernel_size=5`: 필터의 크기 (5x5)
- 실습 코드에서는...
 - 첫 번째 합성곱 층에서는 3채널 이미지를 입력받아 6개의 특징 맵을,
두 번째 층에서는 6개의 특징 맵을 입력받아 16개의 더 복잡한 특징 맵을 추출

```
self.conv1 = nn.Conv2d(3, 6, 5)
self.conv2 = nn.Conv2d(6, 16, 5)
```

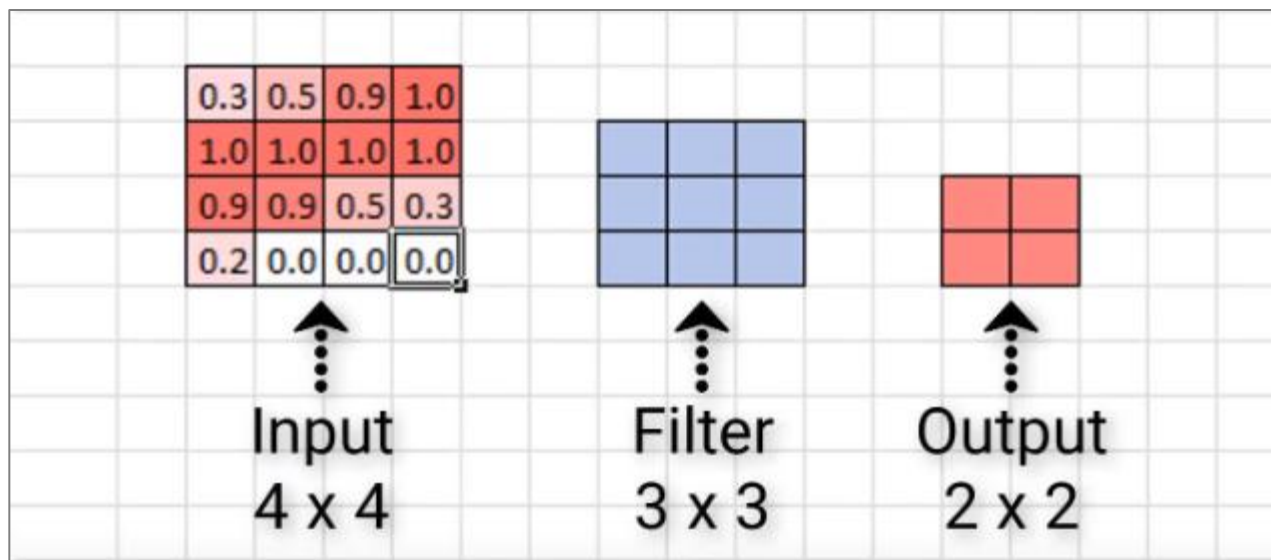
✓ 특징맵(Feature Map)

- 합성곱 연산의 결과값이 모여서 만들어진 2차원 배열
- 필터가 찾고자 하는 특징을 수치로 적어놓은 것이라고 이해하면 됨
- 높은 숫자(활성화): 필터가 찾고자 하는 특징이 해당 위치에서 강하게 나타날 때
- 낮은 숫자: 특징이 없거나 약한 경우 0에 가까운 값이 기록



✓ 패딩(Padding) (1/2)

- 합성곱 연산을 수행하기 전, 입력 데이터의 외곽에 특정 값(주로 0)을 채워 넣는 것을 의미
- 왜 패딩이 필요한가요?
 - 합성곱 연산을 거치면서 **특징맵의 크기가 줄어드는데** 이렇게 되면 층을 깊게 쌓을 수 없음
 - 패딩을 통해 입력 크기를 출력에서도 그대로 유지할 수 있게 하여, 신경망을 깊게 설계할 수 있도록 하기 위해서



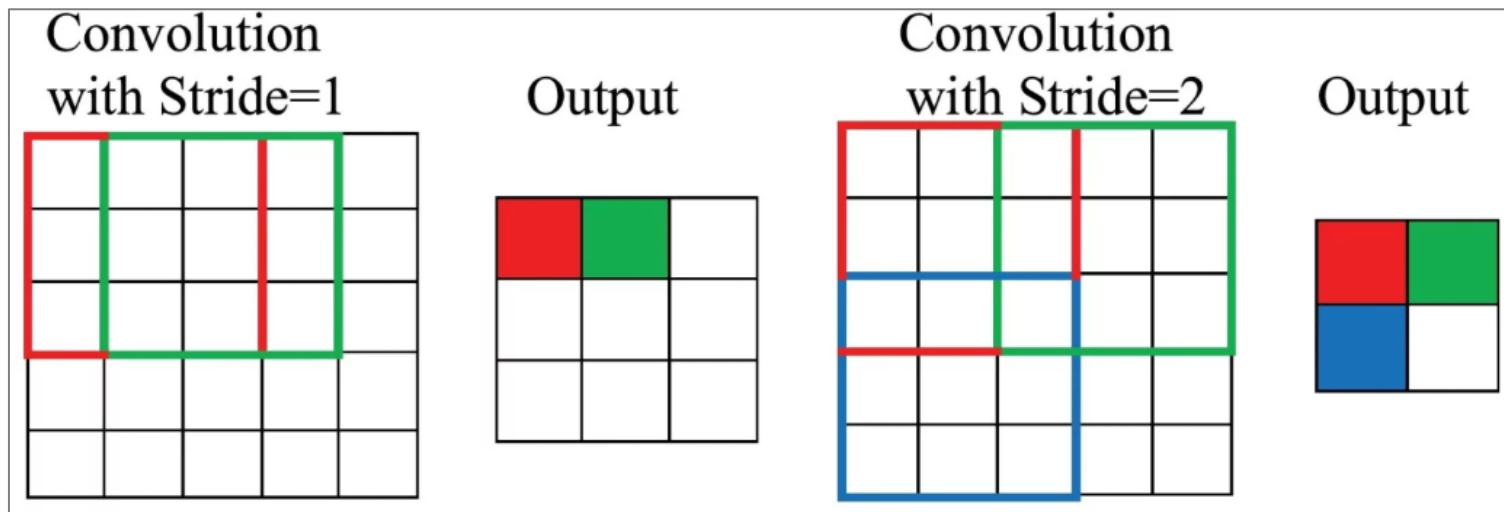
✓ 패딩(Padding) (2/2)

- 합성곱 연산을 수행하기 전, 입력 데이터의 외곽에 특정 값(주로 0)을 채워 넣는 것을 의미
- 왜 패딩이 필요한가요?
 - 이미지의 중심부 픽셀은 필터가 여러 번 훑으며 연산에 참여하지만, 외곽 픽셀은 연산에 참여하는 횟수가 적음
 - 패딩을 주면 외곽 데이터도 연산의 중심에 올 수 있게 되어 중요한 특징을 골고루 추출할 수 있음
 - 패딩이 없다면 1번만 연산에 참여했지만, 패딩으로 인해 여러 번 참여할 수 있게 됨



✓ 스트라이드(Stride) (1/2)

- 필터를 입력 데이터 위에서 이동시킬 때의 간격을 의미
- 보통 1의 값을 가지며, 필터를 한 픽셀씩 옆으로 옮기며 연산한다는 의미
- 스트라이드를 조절하는 이유1
 - 스트라이드 값이 커질수록 필터가 건너뛰는 칸수가 많아지므로, 특징맵의 크기가 급격히 작아짐
 - 이는 데이터를 압축하는 효과를 줌



✓ 스트라이드(Stride) (2/2)

- 스트라이드를 조절하는 이유 2
 - 필터가 연산하는 횟수 자체가 줄어들기 때문에 전체적인 연산량이 감소
- 스트라이드를 조절하는 이유 3
 - 큼직하게 건너뛰며 볼 때, 필터가 이미지의 더 넓은 영역을 한 번에 훑는 효과가 생김
 - 전체적인 구도를 파악하는 데 유리할 수 있음
- 가장 중요한 기능은 “데이터의 핵심적인 특징만 남겨서 압축”

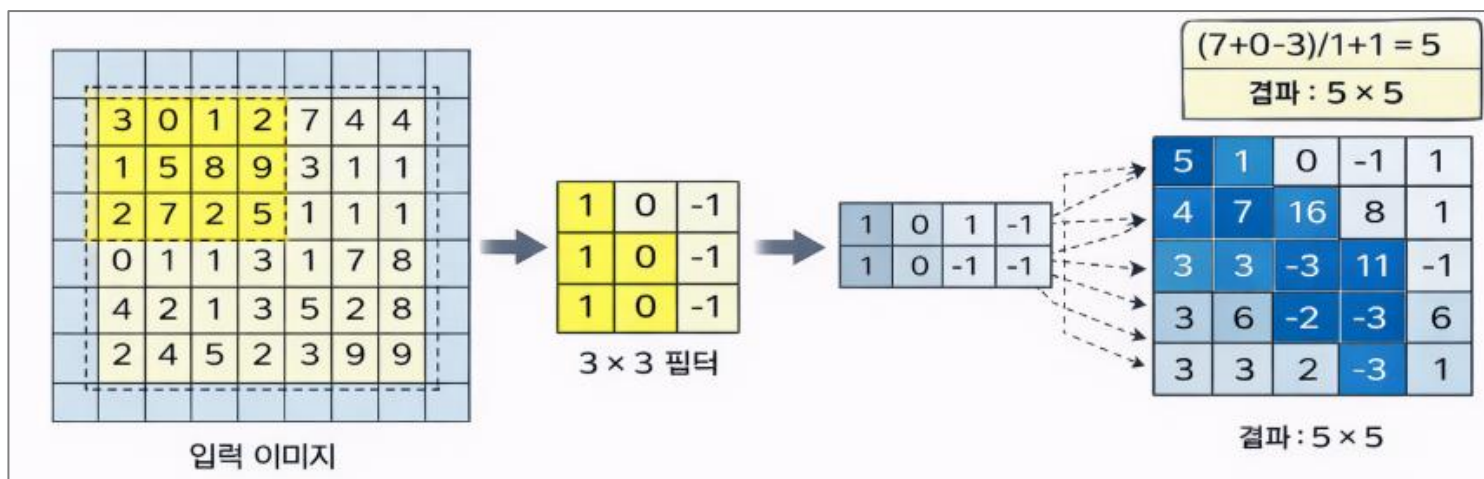
✓ 지금까지 배운 내용을 기반으로 특징맵을 계산해보자 (1/2)

- 입력 이미지: $7 * 7$
- 필터: $3 * 3$
- 패딩 :0
- 스트라이드:1
- 특징맵의 크기는??

✓ 지금까지 배운 내용을 기반으로 특징맵을 계산해보자 (2/2)

- 입력 이미지(I): 7×7
- 필터(F): 3×3
- 패딩(P): 0
- 스트라이드(S): 1
- 특징맵(O)의 크기는?? 5×5

$$O = \frac{I - F + 2P}{S} + 1$$



✓ 활성화 함수: ReLU

- 합성곱 연산이 끝나고 나온 특징맵에 대해서 활성화 함수(ReLU)에 통과시킴
- 활성화 함수에 통과시키는 이유
 - 합성곱 연산은 '선형 연산'이며, 아무리 계속해도 선형 함수일 뿐이라서 복잡한 이미지를 학습할 수 없음
 - 복잡한 패턴을 학습하려면 '비선형성'을 추가해야하고, 이를 위해 활성화 함수를 활용
- 현대 CNN에서는 거의 무조건 ReLU를 사용
 - ReLU: 입력값이 0보다 크면 그대로 내보내고, 0보다 작으면 0으로 만들어버림
- 기울기 소실 문제 해결 + 계산의 단순함 + 중요하지 않은 정보를 0으로 처리

5	0	0	8	1
10	7	16	8	1
0	3	-3	11	-1
3	6	-2	-3	6
3	2	3	-3	3

입력 특징맵

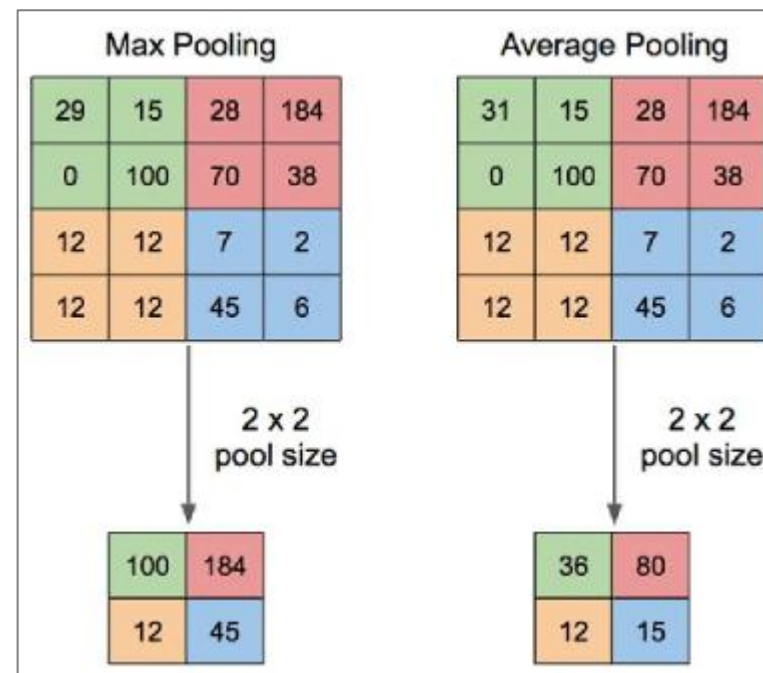
→

5	0	0	8	1
0	7	16	8	1
0	3	0	11	0
3	6	0	0	0
3	2	3	0	0

ReLU 적용 후

✓ 풀링(Pooling) (1/2)

- 특징맵의 크기를 줄여서 계산량을 줄이고, 주요 특징을 추출(압축)하는 단계
- 합성곱 연산과는 달리 가중치가 전혀 없고, 정해진 규칙에 따라 계산만 진행
- 풀링의 종류 (계산하는 방식에 따라 나뉨)
 - Max Pooling (많이 사용)
 - 설정한 영역에서 가장 큰 값만 뽑아냄
 - 해당 영역에서 특징이 가장 강하게 나타난 부분만 대표로 활용
 - 노이즈를 제거하고 가장 선명한 특징만 강조하는 효과가 있음
 - Average Pooling
 - 설정한 영역 안의 값들의 평균을 내어 전달
 - 전체적인 분위기를 전달하는 용도로 활용
 - 핵심 특징이 희석될 수 있어서 마지막 layer 외에는 잘 사용하지 않음



✓ 폴링을 하는 이유

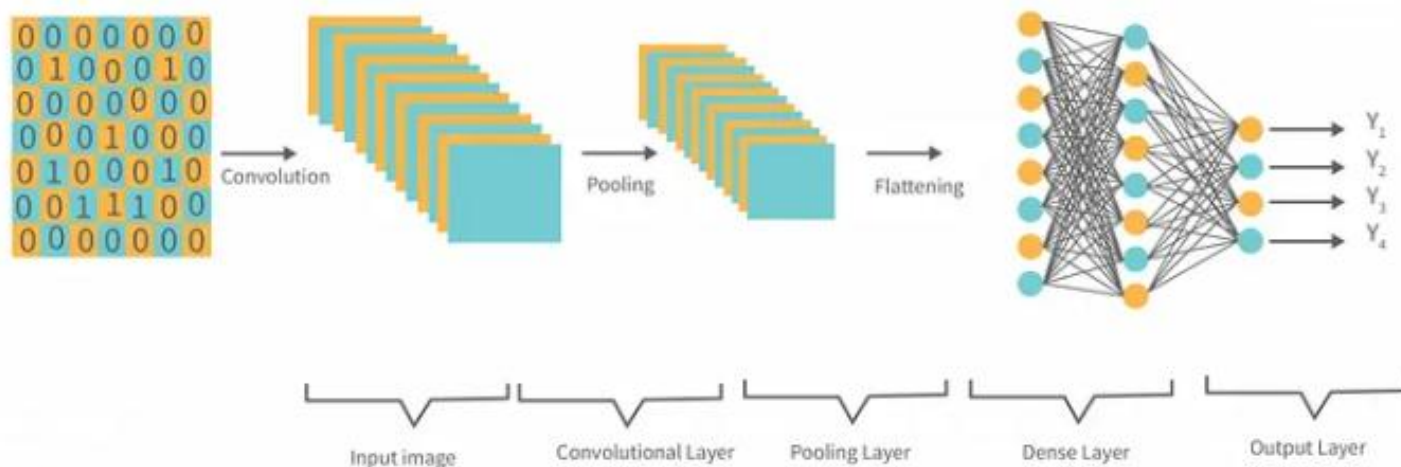
1. 이미지 크기를 줄여서 학습 연산의 부담을 줄이기 위해서
2. 너무 세세한 정보까지 학습하지 않게 데이터를 단순화시키기 위해서 (과적합 방지)
3. 합성곱은 채널을 합치지만, 폴링은 각 채널별로 따로 작동하여 채널 수가 변하지 않음
 - 합성곱: 빨강, 초록, 파랑 물감을 섞어서 “새로운 색”을 만들어내는 과정
 - 폴링: 빨간 종이, 초록 종이, 파랑 종이를 각각 똑같은 크기로 접어서 다시 쌓아두는 과정 (특성 보존)
4. 대상이 이미지 안에서 살짝 움직이더라도, 폴링 영역 안에서 결과는 동일할 수 있음 (위치에 덜 민감함)

✓ 지금까지 배운 내용 정리

- CNN은 추출한 특징을 여러 겹으로 쌓는 과정
 1. 합성곱(Convolution) + (Padding/Stride) : 필터로 이미지의 특징을 찾아내서 특징맵을 생성
 2. ReLU: 찾아낸 특징 중 의미있는 데이터만 남김 (음수는 0으로)
 3. Pooling: 데이터를 압축한다.
 4. 위 과정을 반복하여 점/선 -> 눈/코 -> 얼굴 과 같이 고차원 특징을 완성한다.

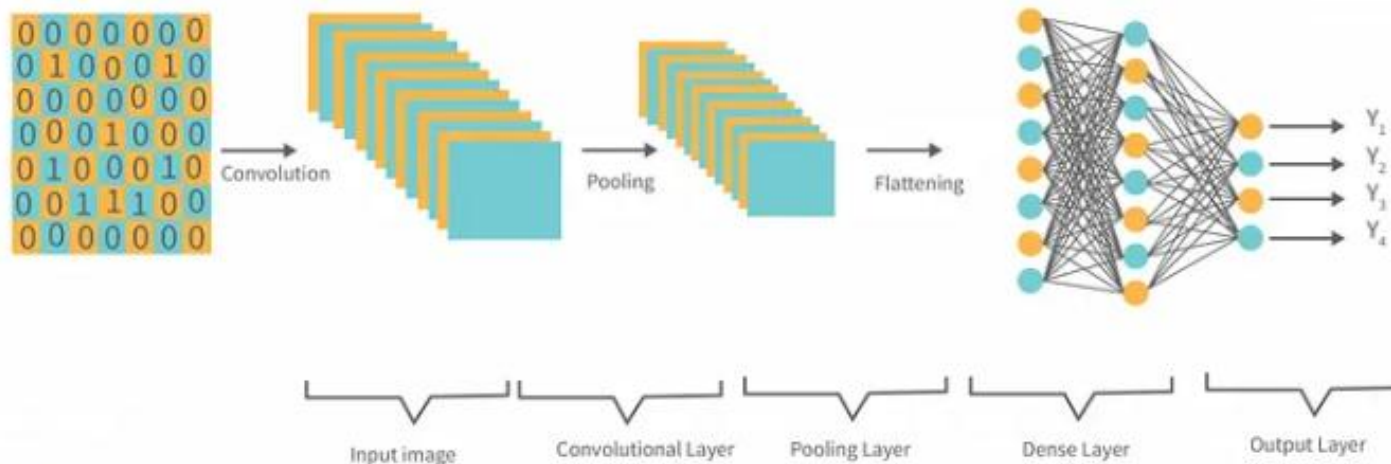
✓ Flatten (1/2)

- 지금까지는 이미지의 '특징'을 추출했고, 이 정보는 가로, 세로, 채널이 있는 3차원 정육면체(Tensor)의 형태
- 하지만 우리가 분류 모델로 사용하는 일반적인 MLP은 1차원 벡터만 입력으로 받을 수 있음
- 3차원의 특징맵을 한 줄(1차원)로 길게 늘어뜨리는 작업
- Flatten을 진행하면 공간 정보가 사라지나요?



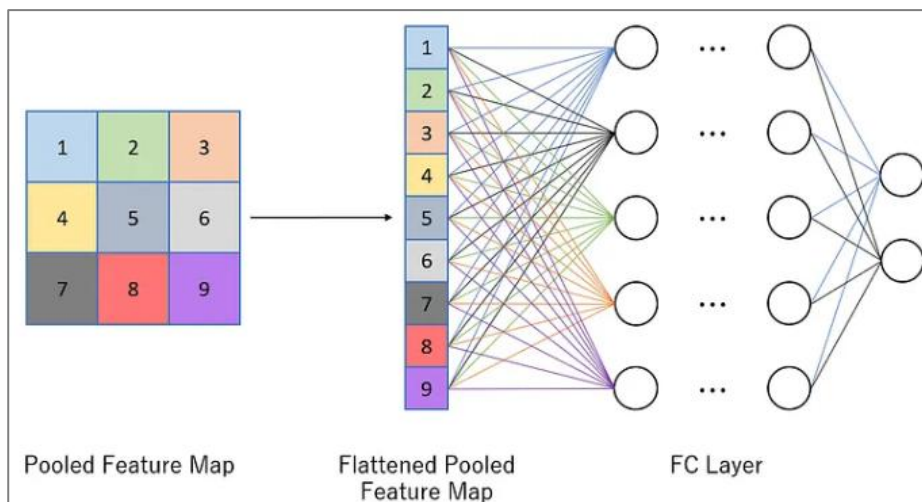
✓ Flatten (2/2)

- 지금까지는 이미지의 '특징'을 추출했고, 이 정보는 가로, 세로, 채널이 있는 3차원 정육면체(Tensor)의 형태
- 하지만 우리가 분류 모델로 사용하는 일반적인 MLP은 1차원 벡터만 입력으로 받을 수 있음
- 3차원의 특징맵을 한 줄(1차원)로 길게 늘어뜨리는 작업
- Flatten을 진행하면 공간 정보가 사라지나요? **아니요!** 하지만 합성곱/풀링에서 특징을 다 뽑아냈기 때문에 괜찮!



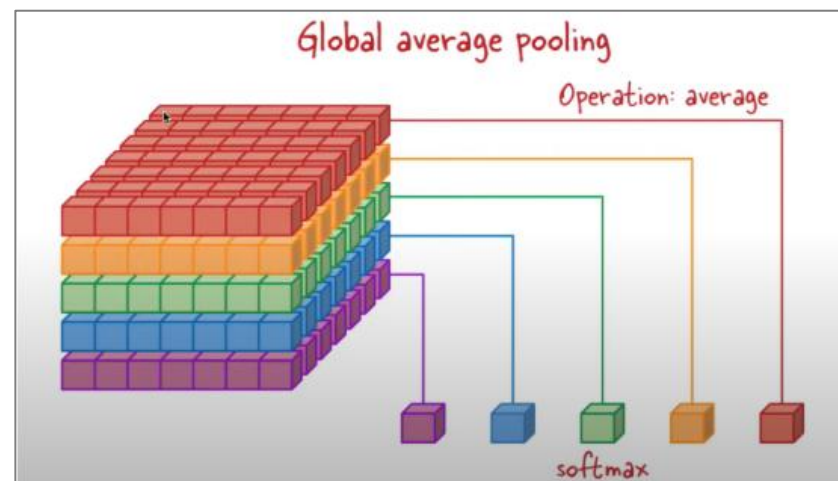
✓ Fully Connected Layer (FC Layer)

- 펼쳐진 벡터를 입력으로 받아 특징들을 하나로 조합하여 이미지가 어떤 클래스에 속하는 지 분류하는 역할
- 다중 클래스 분류를 위해 “소프트맥스 활성화 함수”를 활용함
- 4장에서 학습한 MLP의 동작과정과 동일하게 진행되어 분류 작업을 진행함
- 하지만! 여전히 파라미터(가중치) 수가 너무 많아 메모리를 많이 차지하고 과적합 위험이 있음
 - 그래서 최근에는 Flatten 대신에 **GAP(Global Average Pooling)** 기법을 사용



✓ GAP(Global Average Pooling)

- 특징맵 전체를 대상으로 평균을 내는 풀링 기법
- Flatten으로 펼쳐진 수천 개의 노드와 다음 층을 연결하기 위해서 수많은 가중치를 줄이기 위해 사용
- 작동 방식
 - 각 채널의 가로 * 세로 크기에 상관없이, 한 채널의 모든 원소값을 평균내어 하나의 숫자로 만듦
 - ex) $7 * 7 * 512$ 크기의 데이터가 있다면, GAP을 거치면 바로 $1 * 1 * 512$ 가 됨
 - $\Rightarrow 1 * 1$ 에는 해당 채널 값들의 평균이 들어감
- 각 채널별로 평균을 따로 구하는 것
 - 채널은 각각 서로 다른 특징을 의미
 - 이는 각 특징들의 요약을 구하는 것을 의미



✓ CNN 핵심 아키텍처 요약

- 1. 특징 추출 (Feature Extraction)
 - Conv (합성곱): 필터로 이미지의 특징(선, 면, 패턴)을 찾아냄 (Padding/Stride로 크기 제어)
 - ReLU (활성화): 음수 데이터를 0으로 처리하여 '의미 있는 특징'만 활성화
 - Pooling (압축): 특징의 위치에 상관없이 핵심 정보만 남기고 데이터 크기 축소
- 2. 데이터 브릿지 (Bridge)
 - Flatten: 3D 특징맵을 1차원 벡터로 길게 펼치기 (전통적 방식)
 - GAP (Global Average Pooling): 채널별 평균값만 뽑아 핵심 요약 (과적합 방지)
- 3. 최종 분류 (Classification)
 - FC Layer (전결합층): 추출된 특징들을 가중치로 조합하여 클래스별 점수 계산 (MLP 구조)
 - Softmax (최종 출력): 점수를 0~1 사이의 확률로 변환 (모든 합 = 100%)

다양한 CNN 모델

✓ 다양한 CNN 모델과 현재의 이미지 처리 방식 (1/2)

- AlexNet (2012): 딥러닝 패러다임의 전환
 - ReLU 활성화 함수를 사용하여 기울기 소실 문제를 완화하고 학습 속도를 높임
 - Dropout을 통해 과적합을 방지하며, GPU 연산을 본격적으로 도입하여 대규모 데이터를 처리
- VGGNet (2014): 구조의 단순화와 깊이의 증명
 - 모든 층에서 3*3 크기의 작은 커널만을 사용해서, 파라미터 수를 줄이고 깊게 쌓아 성능을 향상
 - 아키텍처 설계의 표준화를 제시
- ResNet (2015): 잔차 학습을 통한 층의 수직적 확장
 - 네트워크가 일정 수준 이상 깊어지면 오히려 성능이 떨어지는 '망의 퇴화' 문제를 해결한 모델
 - 입력 데이터를 출력에 직접 더해주는 방식을 통해 기울기가 소실되지 않고 신경망의 하위 층까지 잘 전달되도록 설계
=> “잔차 학습(Residual Learning)”

✓ 다양한 CNN 모델과 현재의 이미지 처리 방식 (2/2)

- ViT (Vision Transformer): 새로운 패러다임의 제시
 - 자연어 처리에서 성공을 거둔 Transformer 구조를 이미지 처리에 도입한 모델
 - 이미지를 작은 패치 단위로 분할하여 문장 내 단어처럼 취급하고, Self-Attention 매커니즘을 적용함
 - CNN의 본질적인 제약인 “지역적 연산”에서 벗어나 이미지 전체의 문맥을 한 번에 평가
 - 지역적 연산: 이미지를 처리할 때, 전체 이미지가 아닌 주변의 인접한 픽셀들만 고려하여 연산하는 방식
 - 거대 데이터셋 학습 시 CNN의 성능 한계를 뛰어넘으며, 현재 다양한 기술이 개발되고 있지만 근본으로 자리잡은 기술

6. 전이 학습

• INDEX

- 전이 학습(Transfer Learning)
 - Linear Probing
 - Fine-tuning
- 모델 성능 향상
 - Data Augmentation
 - Learning Rate Scheduler

전이학습

- ✓ 지금까지 배운 내용을 바탕으로 나만의 모델을 만들어보자!
- ✓ 나의 목표는 “고양이를 완벽하게 구분하는 모델”
- ✓ 학습을 위해 필요한 준비물
 - 강력한 GPU
 - 최신 CNN 알고리즘
 - 시간
 - 학습 데이터에 해당하는 100만장의 고품질도 이미지

➤ 자, 이제 시작해볼까요?

✓ 모델을 처음부터 학습시키는 건 현실적인 어려움이 있음

1. 방대한 양의 데이터가 필요
 - 기초부터 스스로 깨우치게 하려면 수만 장 이상의 데이터가 필요
2. 막대한 컴퓨팅 자원과 시간
 - 모든 파라미터를 업데이트하며 학습하는 시간은 고성능 GPU를 장시간 점유해야 함 + 우리의 소중한 시간
3. 느린 수렴과 초기 성능 저하
 - 랜덤 초기화 모델의 첫 에포크 정확도는 보통 10% 내외
 - local minimum에 빠질 확률이 너무 높음
4. 과적합 위험
 - 데이터가 부족하다면 학습 데이터에만 너무 맞춰진 나머지, 테스트 데이터에 대한 성능이 낮아질 수 있음

➤ 그러면 어떻게 해요???????

✓ 전이학습: 거인의 어깨에 올라타자!

- 대규모 데이터셋(예: ImageNet)으로 이미 학습된 모델의 가중치를 새로운 모델에 활용하자.
 - ImageNet: 세계에서 가장 크고 유명한 이미지 데이터베이스 (압도적 규모, 온갖 다양한 사물, 딥러닝 혁명의 시발점)
- 이미 학습된 지식을 빌려와도 문제가 없나요?
 - CNN이 학습하는 저수준 특징들(엣지, 색상, 선 등)은 매우 범용적인 공통 지식이라 활용 가능
 - 학습된 지식은 건드리지 않고, 이를 출력하는 부분만 용도에 맞게 고쳐서 활용할 수 있음



✓ 전이학습을 위한 CNN 분석

- 이미지를 분류하는 CNN은 크게 두 부분으로 나뉨

1. 특징 추출

- 합성곱과 풀링을 통해 이미지의 선, 색상, 형태, 질감 등 시각적 패턴을 찾아내는 부분
- 특징을 찾아내는 능력은 어떤 이미지에서든 공통적으로 쓰이기 때문에 **그대로 사용**

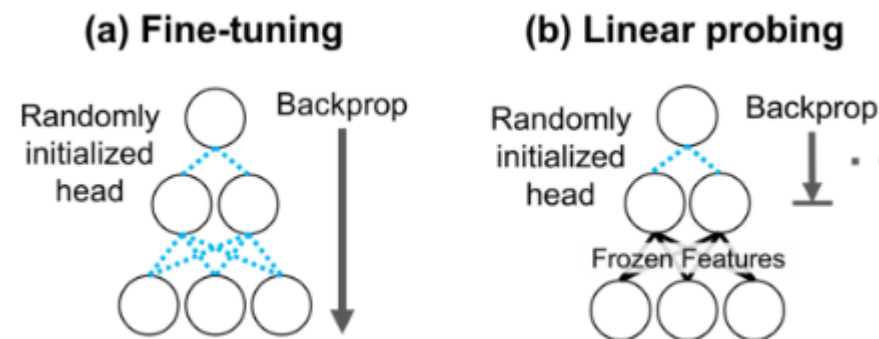
2. 분류

- 추출된 특징들을 종합하여 최종 판단을 내리는 Fully Connected Layer
- 기존 모델과 다른 목적의 분류가 필요하다면, 마지막 층만 떼어내고 우리의 목적에 맞는 **새로운 층을 달기**

- 위 특징을 이해하고, 어떻게 학습시킬 지 고민해보자!

✓ 전이학습 종류

- Linear Probing
 - 특징 추출부의 가중치를 고정(Freeze)하고, 새로 만든 분류층만 학습
 - 학습할 양이 매우 적어 압도적으로 빠름
 - 학습 데이터가 아주 적어서 빌려온 모델을 망가뜨릴 것 같을 때!
- Fine-tuning
 - 고정했던 특징 추출부까지 포함하여 모델 전체를 다시 학습
 - 내 데이터에 완전히 최적화되므로 최고의 성능을 발휘
 - 기존 지식을 잊어버리면 안되므로, 조심스럽게 학습을 진행해야 함



Linear Probing

✓ Linear Probing

- 사전 학습된 모델의 “특징 추출부” 가중치를 고정(Freeze)하고, 마지막에 새로 추가한 분류층만 학습시키는 방법
- 동작 방식
 - 가중치 동결(Weight Freezing)
 - 모델의 합성곱(Convolution) 층들의 파라미터가 업데이트되지 않도록 설정
 - 이미 ImageNet 같은 데이터로 최적화된 “특징 추출 능력”을 훼손하지 않기 위해서
 - code: `param.requires_grad = False`
 - 분류층 교체(Head Replacement)
 - 기존 출력 노드를 제거하고, 우리의 목적에 맞는 출력 노드를 가진 새로운 층을 붙임
 - 이 층은 초기에 랜덤한 값으로 초기화되어 있으므로, 새로운 데이터를 학습시켜야 함

✓ Linear Probing을 하는 이유

- 압도적인 학습 속도
 - 전체 파라미터 중 약 1% 미만(분류층)만 학습하면 되기 때문에 GPU 자원을 적게 점유
- 적은 데이터로도 가능
 - 데이터가 부족할 때 모델 전체를 학습하면 과적합이 될 수 있지만, 분류층만 학습하면 되기 때문에 훨씬 안정적
- 성능의 베이스라인
 - “사전 학습 모델이 얼마나 괜찮은가?”를 확인하는 객관적인 지표가 됨
 - 예를 들어, ResNet과 ViT의 성능을 비교하고 싶다면 둘 다 똑같이 얼려놓고 Linear Probing 하면 성능을 비교할 수 있음

✓ Linear Probing Code

```
# 1. 마지막 분류층(Fully Connected Layer) 교체
# ResNet-18의 마지막 층을 수행해주세요.
# 기존 ResNet-18의 마지막 층(model.fc)은 ImageNet의 1000개 클래스를 분류합니다.
# 우리는 CIFAR-10의 10개 클래스를 분류하도록 새로운 `nn.Linear` 레이어로 교체해야 합니다.
model.fc = nn.Linear(model.fc.in_features, 10)

# 2. 선형 프로빙을 위해 ResNet-18을 1에서 선언한 마지막 층을 제외하고 업데이트가 되지 않도록 가중치를 동결하세요.
# `param.requires_grad = False`로 설정하면 해당 파라미터는 학습 중에 업데이트되지 않습니다.
for name, param in model.named_parameters():
    if "fc" not in name: # fc 층이 아닌 경우
        param.requires_grad = False
```

Fine-tuning

✓ Fine-tuning

- 동결 했던 “특징 추출부”도 포함해서 모델의 전체 파라미터를 우리 데이터셋에 맞춰 다시 학습시키는 방법
- 전체 레이어의 동결을 해제해야 함
- 가져온 사전 학습 모델은 매우 정교하기 때문에 학습률을 매우 낮춰서 학습을 진행해야 함
 - 학습률이 높으면 기존 가중치를 순식간에 파괴하여 성능이 급락할 수 있음

```
# 1. 전체 레이어의 동결을 해제해주세요
for param in model.parameters():
    param.requires_grad = True

# 2. 새로운 `optimizer`를 선언해주세요. 학습률은 낮은 0.0005로 설정해주세요.
optimizer = optim.SGD(model.parameters(), lr=0.0005)
```

- 기존 성능이 이미 좋지만 Fine-tuning을 사용하는 이유
 - 우리 데이터만의 특수한 질감이나 형태에 더 예민하게 반응하도록 하기 위해서

✓ Linear Probing vs Fine-tuning

구분	Linear Probing	Fine-tuning
정의	특징 추출 부분(Backbone)을 동결하고 분류층(Head)만 학습	전체 또는 일부 모델을 새로운 데이터로 추가 학습
학습 파라미터	분류층만 학습 (전체의 약 1% 미만)	모델 전체 학습 (100%)
학습 속도	매우 빠름 (분 단위)	상대적으로 느림 (시간 단위)
학습률 (LR)	상대적으로 큼 (0.001 ~ 0.01)	상대적으로 작음 (0.0001 ~ 0.001)
성능	기본 성능 확보	최고 성능 달성 가능
과적합 위험	낮음	높음 (특히 데이터가 적을 때)
적용 시나리오	데이터가 적거나 빠른 프로토타이핑이 필요할 때	충분한 데이터와 컴퓨팅 자원이 있을 때

모델 성능 향상

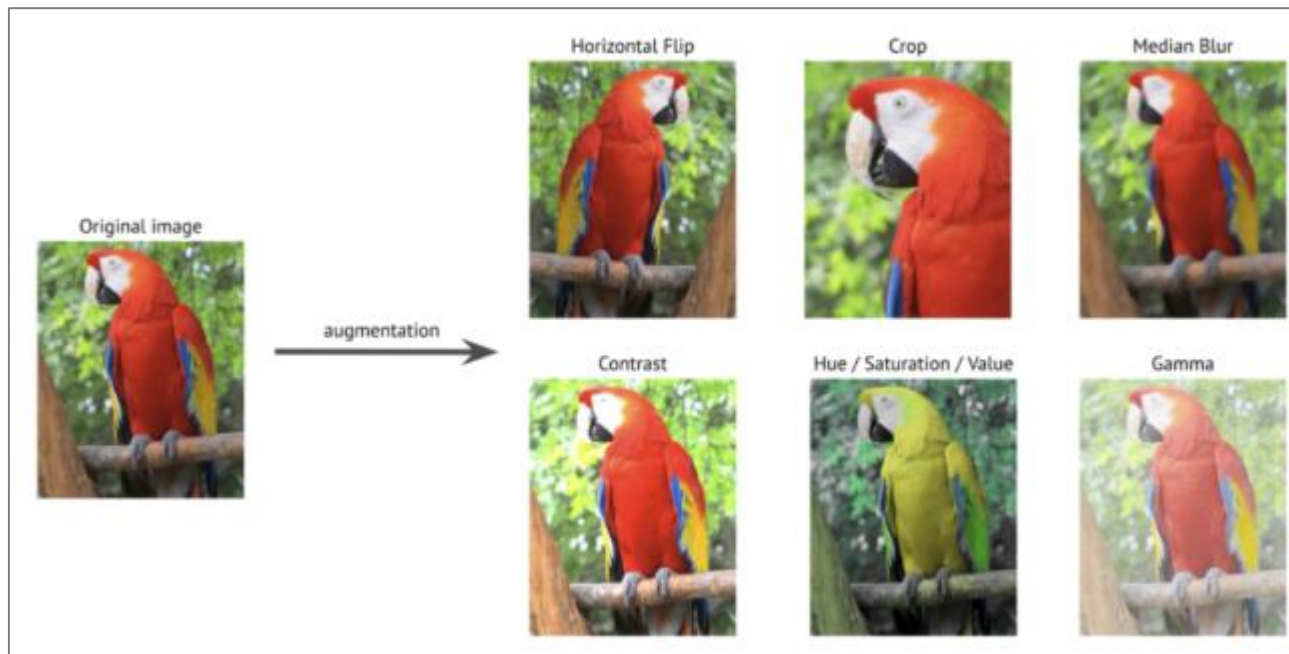
✓ 앞서 배운 전이학습을 토대로 다시 학습을 시작해보자!?

- 현실은 전체 모델을 학습시키기 전 두 가지 큰 문제에 직면하게 됨
 - 과적합(Overfitting): 모델이 데이터를 암기해서 테스트에서는 성능을 발휘하지 못하는 것
 - 최적화(Optimization): 너무 빠르거나 느리게 학습하여 최적의 성능을 놓치는 것
- 성능 향상을 위한 두 가지 기술
 - 데이터 증강(Data Augmentation)
 - 학습률 스케줄러(LR Scheduler)

데이터 증강

✓ 데이터 증강(Data Augmentation)

- 학습 데이터에 무작위적인 변환(회전, 자르기, 뒤집기 등)을 가하여 데이터의 다양성을 인위적으로 높이는 기법
- 모델은 매번 조금씩 다른 이미지를 보게 되므로, 사물의 본질적인 특징을 학습하게 됨



✓ 데이터 증강 특징

- [장점] 과적합(Overfitting) 방지
 - 데이터에 노이즈를 주어 암기를 방해하고 이해를 유도함
- [장점] 일반화 성능 향상
 - 실제 세상의 사물은 회전/자르기/뒤집기 등 다양한 노이즈가 있는데, 증강은 다양한 상황에서 모델이 유연하게 대처할 수 있게 함
- [장점] 부족한 데이터 극복
 - 부족한 데이터 수를 증강을 통해 무한에 가까운 조합으로 변경시켜 학습 효과를 극대화할 수 있음
- [단점] 데이터의 본질을 훼손
 - 이미지의 모양을 바꿀 수 있지만, 사물의 정체성 또한 바뀔 수 있음 (ex: 6을 뒤집으면 9)
- [단점] 학습 난이도가 늘어나고, 학습 시간이 늘어남
 - 학습이 매우 느리게 진행되어 손실 값이 잘 떨어지지 않을 수 있음

✓ 데이터 증강 코드

```
train_transform_aug = transforms.Compose([
    # 두 가지 증강기법을 추가해봅시다. 기존의 `train_transform`의 구성에 추가해주세요.
    # 32x32 이미지 주변에 4픽셀의 패딩(padding)을 추가한 뒤, 무작위로 32x32 영역을 잘라냄
    transforms.RandomCrop(32, padding=4),
    # 50% 확률로 이미지를 좌우로 뒤집음
    transforms.RandomHorizontalFlip(p=0.5),
    transforms.Resize((224, 224)),
    transforms.ToTensor(),
    transforms.Normalize(mean=mean, std=std)
])
```

학습룰 스케줄러

✓ 학습률 스케줄러(Learning Rate Scheduler, LR Scheduler)

- 특정 시간이나 Epoch에 따라 학습률(Learning Rate, 보폭)을 동적으로 조정하는 기법
- 처음에는 크게 움직이다가, 정답 근처에 가서는 조심스럽게 작게 움직이는 것이 핵심
- 학습 초반에는 “학습률을 크게” 설정하여 넓은 영역을 빠르게 학습
- 학습 후반에는 “학습률을 작게” 설정하여 천천히 학습(빠르면 정답을 지나치기 때문)
- 주요 스케줄러 종류
 - StepLR: 일정한 에포크마다 학습률을 고정된 비율로 감소시킴 (구현이 쉽고, 직관적)
 - CosineAnnealingLR
 - ReduceLROnPlateau

✓ 학습률 스케줄러(LR Scheduler) 특징

- [장점] 세밀한 손실 감소
 - 학습률이 줄어드는 시점마다 손실값이 확 떨어지는 걸 볼 수 있음
- [장점] 성능 극대화와 안정적인 학습
- [단점] 또 하나의 파라미터 지옥
 - 스케줄러를 도입하는 순간, 우리가 관리해야 할 파라미터가 하나 더 추가됨
 - 이 수치가 조금만 어긋나도 모델이 충분히 학습되지 않거나, 너무 늦게 수렴되어 시간만 낭비하게 됨
- [단점] 성급한 학습 종료
 - 모델이 더 학습해야 되는 상태이지만 스케줄러가 학습률을 줄여서 정답 근처에도 가지 못하는 경우가 생김
- [단점] 범용성 부족
 - “5에포크마다 0.1배씩 줄이는 게 정석” 과 같은 일반화 케이스가 없음
 - 즉, 본인이 실험을 통해서 적절한 파라미터 값을 찾아야 함

✓ Optimizer vs LR Scheduler (1/2)

- Optimizer(Gradient Descent, Adam, ...)
 - 매 순간(Batch)마다 들어오는 데이터의 기울기를 보고, 개별 파라미터들을 조정
- LR Scheduler(StepLR, ...)
 - 특정 시간(Epoch)가 지나거나 성능이 정체되면, 기본 학습률 자체를 조정

구분	옵티마이저 (Optimizer)	학습률 스케줄러 (LR Scheduler)
핵심 질문	"어느 방향으로, 어떻게 움직일까?"	"지금 속도를 줄여야 할 때인가?"
관리 대상	모델의 가중치(Weights) 하나하나	옵티마이저가 가진 기본 학습률(LR)
실행 단위	매 배치(Batch)마다 (실시간)	매 에포크(Epoch)마다 (주기적)
실습 코드	<code>optimizer.step()</code>	<code>scheduler.step()</code>

✓ Optimizer vs LR Scheduler (2/2)

- 예를 들어, **기준 학습률**이 0.01로 설정되어있다고 가정
- Optimizer(Adam)의 작업
 - 가중치별로 보폭을 다르게 가져감
 - 가중치 W_1 (기울기가 가파름): $0.01(\text{기준}) * 0.5(\text{Adam 계수})$
 - 가중치 W_2 (기울기가 완만함): $0.01(\text{기준}) * 2.0(\text{Adam 계수})$
- LR Scheduler(StepLR)의 작업
 - 초기 세팅: `step_size=5, gamma=0.1`
 - **기준 학습률**이 0.01에서 0.001로 바뀜

7. RNN과 LSTM

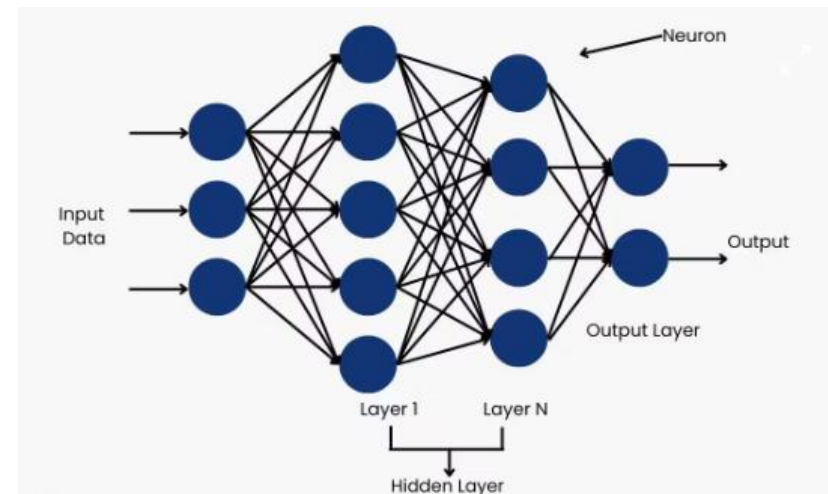
• INDEX

- RNN
- LSTM

RNN

✓ 다중 퍼셉트론(Multi-Layer Perceptron, MLP) (1/2)

- 인공 신경망(Neural Network) 의 한 종류로, 퍼셉트론을 여러 층으로 쌓은 모델
- 입력층과 출력층 사이에 1개 이상의 은닉층을 가짐
- MLP의 학습 과정
 1. 순전파(Forward Propagation)
 2. 손실 함수 계산(Loss Function)
 3. 역전파(Back Propagation) 및 가중치 업데이트 (Optimization)
 4. 1~3번 반복 후 종료 (Training 완료)



✓ 다중 퍼셉트론(Multi-Layer Perceptron, MLP) (2/2)

- 은닉층과 비선형 활성화 함수를 사용하여 복잡한 데이터 패턴을 학습할 수 있음
- 완전 연결 구조로써 각 층의 뉴런은 이전 층의 모든 뉴런과 연결되고, 이로 인해 특징 조합을 학습할 수 있음
- 분류와 회귀 등 다양한 문제에 적용할 수 있는 매우 유연한 모델
- **입력 데이터의 순서를 고려하지 못함**
 - 예1) I go to school and School go I to 는 동일한 정보로 처리하여, 단어의 순서에 담긴 의미를 학습할 수 없음
 - 예2) 어제 / 오늘 / 내일의 날씨를 예측할 때, 각 날짜를 개별적인 사건으로 볼 뿐, 시간의 흐름에 따른 연속적인 관계를 학습하지 못함
- **고정된 입출력 크기**
 - 모델을 설계할 때 입력과 출력의 크기(차원)이 고정되어야 함
 - 문자열의 길이나 순차적 데이터의 길이가 달라지면 처리하기 힘들

➤ 위의 한계를 극복하기 위해 등장한 것이 “RNN”

✓ [참고] 순차적 데이터(Sequential Data)

- 순서에 의미가 있는 모든 종류의 데이터
- 데이터의 순서가 바뀌면 그 의미나 정보가 완전히 달라지는 데이터

• 특징

- 순서가 중요
- 장기 의존성(Long-term dependency)
 - 과거의 정보가 현재 / 미래에 영향을 준다.
- 가변 길이
 - 데이터의 길이가 고정되어 있지 않은 경우가 많음
 - ex) 문장의 길이, 음악의 길이

• 대표적인 예시

- 자연어(텍스트), 시계열 데이터(날씨 데이터, 주식 가격), 오디오 / 비디오 데이터

• 오디오



• 텍스트

Natural | Language | Process

• 비디오

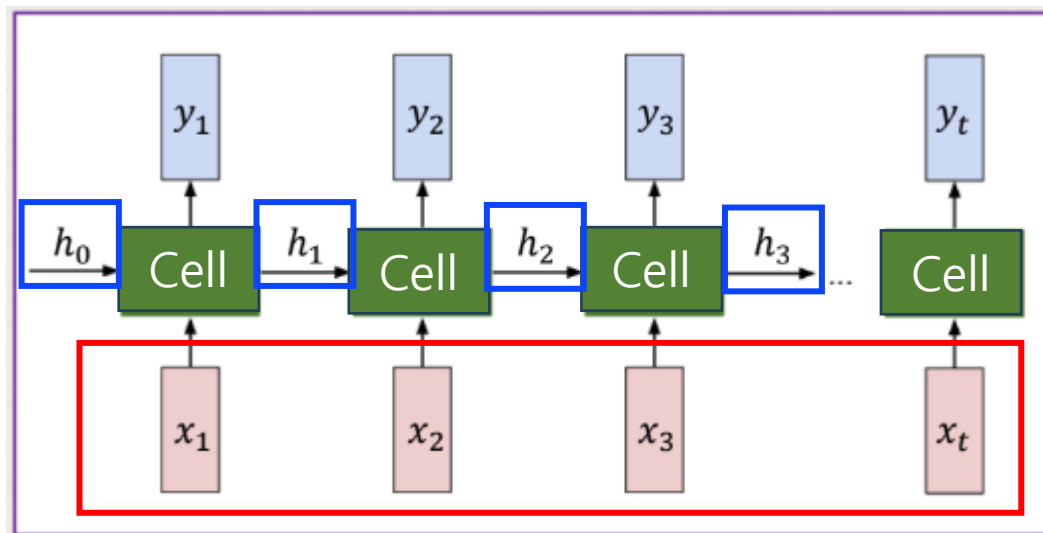


✓ RNN(Recurrent Neural Network)

- 순환 신경망
- 순서가 있는 데이터(Sequential Data)를 처리하기 위해 고안된 인공 신경망(Neural Network)
- 과거의 정보를 기억하고, 기억한 정보를 활용해 현재의 데이터를 해석
- “Cell” 이 **입력 데이터**와 **과거의 데이터(은닉 상태)** 를 섞어서 “새로운 데이터”를 만들어 냄

과거의 데이터(은닉 상태)

- 이전 시점의 데이터
- 초기에는 $[0, 0, 0, 0]$
(현재 벡터는 예시)

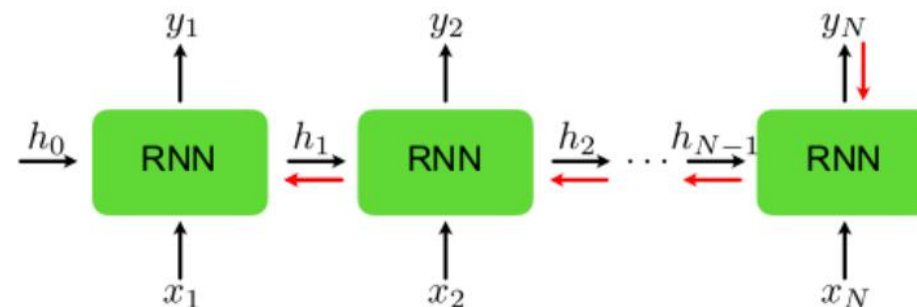


입력 데이터(순차 데이터)

- 인코딩/임베딩이 진행된 벡터가 들어감
- ex) $1 \Rightarrow [1, 0, 0, 0]$

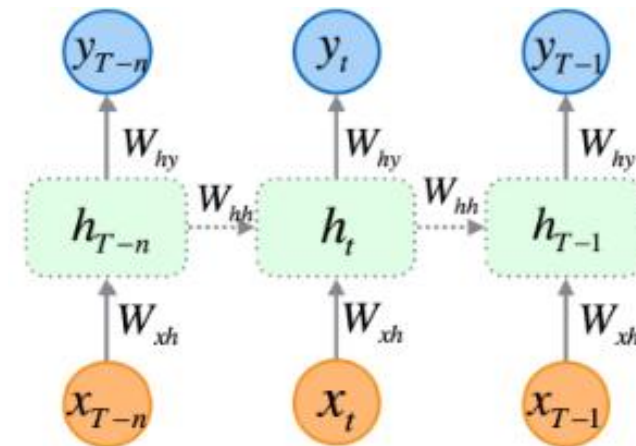
✓ RNN - Cell (1/4)

- 과거의 정보와 현재의 입력 데이터를 섞어서 새로운 데이터를 만들어내는 곳 (like 퍼셉트론)
- 과거의 정보를 담아두는 곳이 바로 “**은닉 상태(Hidden State)**”
- 동작 방식
 1. 셀은 두 가지 데이터를 입력 받음
 - 현재 데이터(x_t)
 - 이전 셀의 **은닉 상태(h_{t-1})**
 2. 셀 내부에서 두 가지 데이터를 **조합**하여 새로운 **은닉 상태**를 만듦(h_t)
 3. 새로운 **은닉 상태**는 두 곳으로 전달 됨
 - 출력 값(y_t): 현재 시험의 예측 결과 (필요한 경우에 사용)
 - 다음 셀: 다음 순서의 데이터를 처리할 때 참고할 ‘기억’으로 전달



✓ RNN - Cell (2/4)

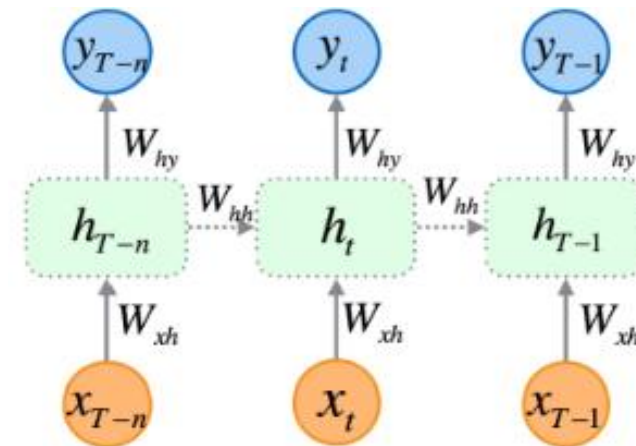
- 입력 데이터와 은닉 상태를 조합하는 방법에 대해서 살펴보자.
- 각 입력데이터와 은닉 상태의 영향력을 나타내는 가중치가 존재
 - 입력 가중치 (W_{xh}) : 입력 데이터의 영향력
 - 순환 가중치 (W_{hh}) : 이전 기억의 영향력
 - 출력 가중치 (W_{hy}) : 출력 데이터의 영향력
 - 초기에 해당 값은 랜덤으로 설정 됨
- 이후에 다시 언급하겠지만
 - 모든 입력, 순환, 출력 가중치는 같은 값
 - 입력 가중치와 순환 가중치, 출력 가중치가 같다는 말은 아님!!



✓ RNN - Cell (3/4)

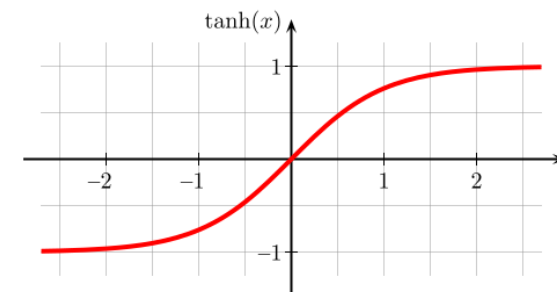
• 입력 데이터와 은닉 상태 조합의 동작 방식

1. 셀은 두 가지 데이터를 그대로 받지 않고, 각자의 가중치를 곱해 중요도를 반영
 - 가중치가 반영된 **현재 정보** = 현재 데이터(x_t) * 입력가중치(W_{xh})
 - 가중치가 반영된 **과거 기억** = 이전 기억(h_{t-1}) * 순환가중치(W_{hh})
2. 두 정보를 합침
 - 조합된 정보 = **현재 정보** + **과거 기억**
 - 실제로는 편향(default 값)을 추가로 더하지만, 단순화하기 위해 생략
 - 각 데이터는 벡터(특징)로 표현되며, 더하는 것만으로도 새로운 특징을 만들어 냄
3. 합쳐진 정보를 활성화 함수(tanh)에 넣어 압축
4. 새로운 은닉 상태(h_t)를 다음 셀로 전달!



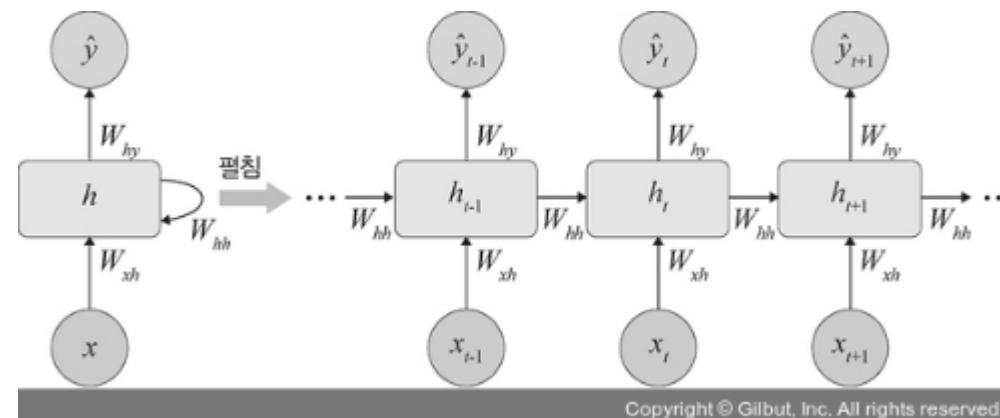
✓ [참고] 활성화 함수 - tanh

- 입력받은 값을 -1과 1 사이의 값으로 압축시켜주는 역할
- 장점
 1. 출력값을 1과 -1 사이로 제한하여, 기울기 폭발 방지
 - 이전 단계의 결과에 계속해서 같은 가중치를 곱하는 구조이기 때문에 기울기 폭발 가능성이 있음
 - -1이 강한 부정을 의미한다는 것을 표현할 수 있음
 2. 0을 중심으로 대칭이어서, 더 빠르고 안정적인 학습이 가능
 - 학습 과정에서 가중치를 업데이트할 때, 모든 가중치가 같은 방향(모두 증가/모두 감소)으로 움직이는 편향이 줄어듦
- 여전히 기울기 소실이라는 한계가 있음
- RNN에서 다른 활성화 함수를 쓰지 않는 이유
 - ReLU: 양수를 그대로 출력하기 때문에 기울기 폭발이 발생
 - 시그모이드(Sigmoid): 0과 1 사이로 바꿔주면서, 0을 중심으로 하지 않아서 학습이 느리고 장기 기억을 못함



✓ RNN - Cell (4/4)

- 사실 “셀(cell)은 하나다!”
- 이전에 말한 각각의 입력 데이터, 은닉 상태 가중치가 같은 이유
- 하나의 셀을 재사용하는 이유(가중치 공유)
 - 압도적인 효율성
 - 입력 데이터가 아무리 길어도, 학습시킬 가중치 세트가 하나임
 - 모델이 매우 가볍고 효율적
 - 좋은 일반화 성능
 - 각 셀이 정해진 순서에 따른 데이터를 학습하는 것이 아님!! (ex: 첫 번째 단어를 처리하는 방법, 세 번째 단어를 처리하는 방법, ...)
 - 어떤 위치의 단어든 이전 기억과 조합하는 일반적인 방법을 학습
 - 훈련 때 학습하지 않은 길이의 문장에도 유연하게 대처할 수 있음



$$h_t = \tanh(W_{hh}h_{t-1} + W_{xh}x_t)$$

✓ RNN의 학습 방식 (1/2)

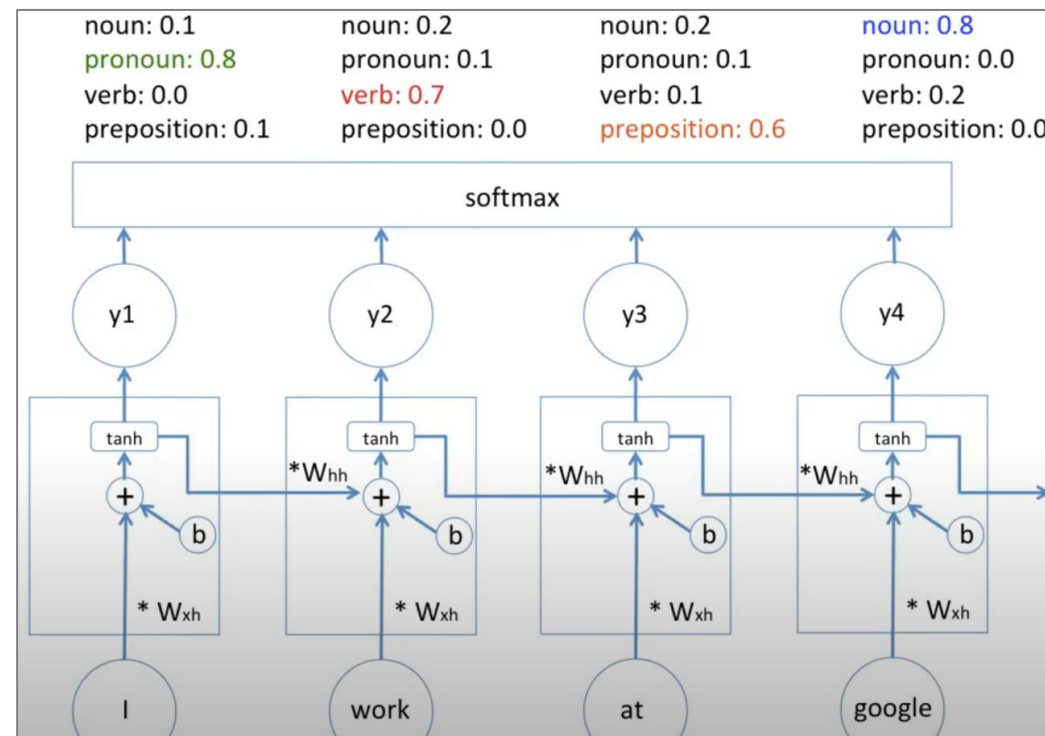
- RNN이 학습하는 과정(오차를 최소로 하는 가중치를 찾아가는 과정)

1. 순전파(Forward Propagation)

- 우리가 배웠던 과정을 그대로 진행
- 랜덤하게 초기화된 가중치(W_{xh} , W_{hh} , W_{yh})를 사용
- 단어를 순서대로 셀을 통과시키면서 은닉 상태를 업데이트
- 각 시점에서, 혹은 마지막 시점에서 최종 예측값을 출력

2. 오차 계산

- 각 문제에 맞는 채점 기준에 맞춰 오차를 계산
- 예시) 품사분류기 (지도학습)
- softmax: 클래스 총합이 1이 되도록, 각 확률을 구함



- 학습 과정에서는 softmax를 통과 후 나온 결과 중 가장 높은 값과 정답을 비교해서 오차를 구함

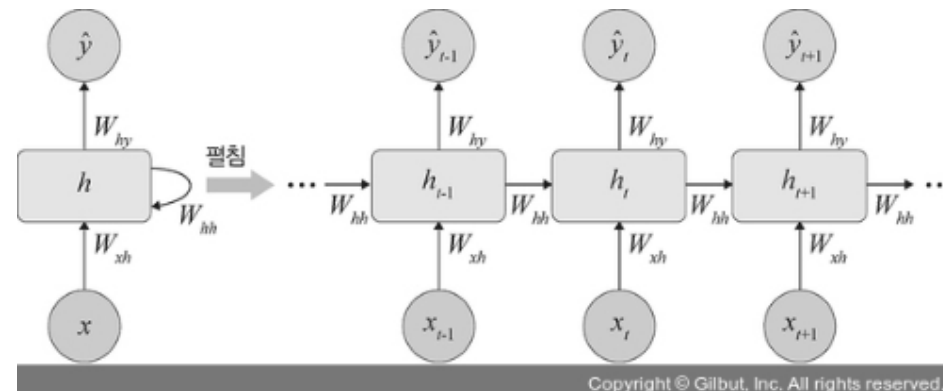
✓ RNN의 학습 방식 (2/2)

- RNN이 학습하는 과정(오차를 최소로 하는 가중치를 찾아가는 과정)

1. 역전파(Backpropagation)

- 정확히는 BPTT(Backpropagation Through Time)

- 최종 오차의 영향력을 역순으로 추적, 가중치가 오차에 미친 영향력을 구함
- 셀에는 **은닉 상태**와 **입력 데이터**의 가중치가 존재, 해당 시점의 “**입력데이터의 가중치**”가 오차에 미친 영향력을 구함
 - 우리가 원하는건 가중치를 업데이트하는 것이다.
 - 기존 역전파와 마찬가지로 “입력 데이터”에 따라 영향력(기울기)를 구함
- 해당 시점에서 최종 오차가 “**은닉 상태 가중치**”와 “**입력 데이터 가중치**”로 적절하게 나뉘면, “**은닉 상태 가중치의 오차**”를 이전 셀로 넘긴다.
- 2번~3번 과정을 첫 번째 시점에 도달할 때까지 반복한다.
- RNN은 모든 시점에서 동일한 가중치를 사용하기 때문에 모든 시점의 가중치들을 합산하고, 이를 학습률과 결합해 오차가 줄어드는 방향으로 가중치를 업데이트



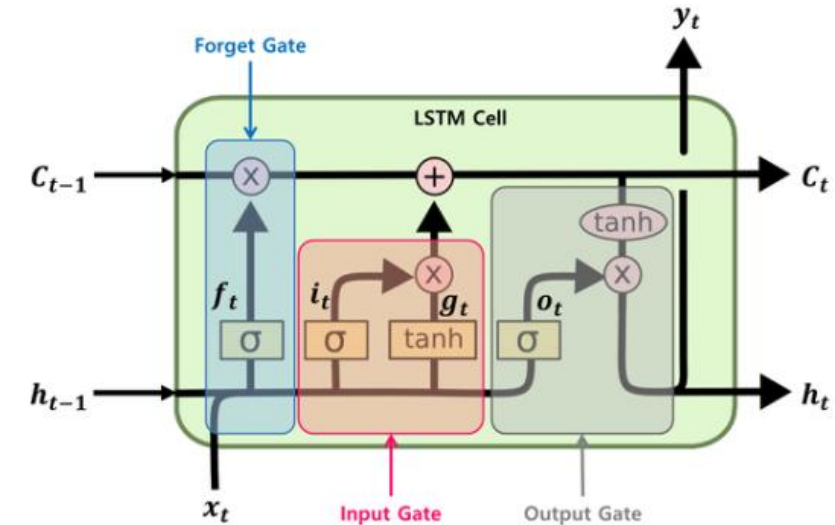
✓ RNN 의 한계

- 기울기 소실 문제(Vanishing Gradient Problem)
 - BPTT(역전파) 과정에서 “이전 정보의 책임” + “현재 시점의 책임”으로 오차가 나뉘지면서, 뒤로 갈수록 오차 정보가 거의 전달되지 않음
 - 결국 오래된 정보에 대해서 학습이 제대로 이루어지지 않음
 - 장기 의존성 문제(Long-Term Dependency Problem)
- 그리고 이와 같은 장기 의존성 문제를 해결하기 위해서 등장한 것이 “LSTM(Long Short-Term Memory)”

LSTM

✓ LSTM (Logn Shrot-Term Memory)

- 이름 그대로 “장단기 기억”
- RNN의 한 종류로, 기존 RNN의 장기 의존성 문제를 해결하기 위해 설계 됨
- “**셀 상태(Cell State)**와 **게이트(Gate)**” 라는 독자적인 구조를 도입하여 문제를 해결!
- 기존 RNN
 - 은닉 상태(h_t) 하나만으로 정보를 전달
- LSTM
 - 은닉 상태(h_t) + **셀 상태(c_t)** 로 2가지 정보 전달 통로를 활용
 - **셀 상태(c_t)** 는 장기 기억을 전달하는 통로
 - **셀 상태(c_t)** 는 **게이트(Gate)** 를 활용하여 장기 기억을 전달할지, 삭제할지를 결정



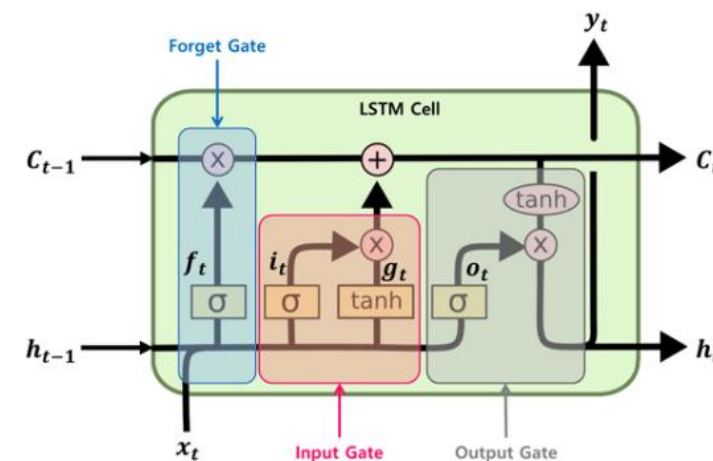
✓ LSTM 구성요소

• 셀 상태(Cell State)

- 장기 기억 데이터를 전송하는 통로
- 기존 RNN의 은닉 상태와는 다르게 덮어쓰기를 하지 않고, $\tanh$ 함수를 통과하는 등의 복잡한 변환이 일어나지 않음
- 단순하고 직접적인 경로로써 기울기가 소실되지 않고 시간을 거슬러 올라갈 수 있게 함

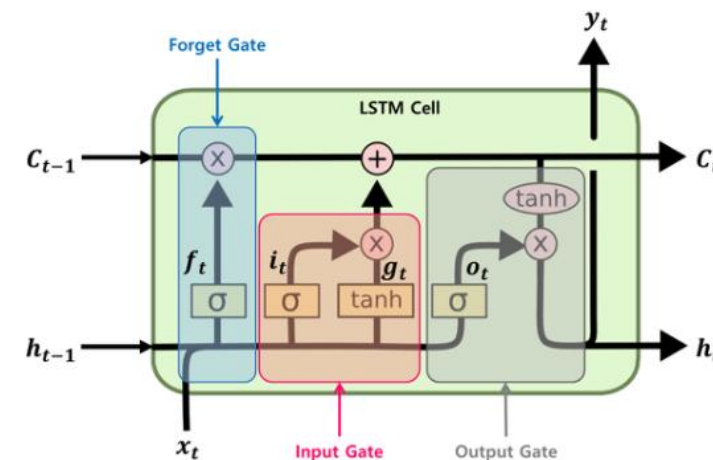
• 게이트(Gate)

- 장기 기억 통로일 뿐인 “**셀 상태(Cell State)**”를 실질적으로 컨트롤하는 중요한 역할
- 어떤 정보를 기억/망각/사용할지를 ‘스스로 학습’하여 결정
- 각기 다른 역할을 하는 3개의 게이트가 존재
 - 망각 게이트(Forget Gate)
 - 입력 게이트(Input Gate)
 - 출력 게이트(Output Gate)
 - 모든 게이트는 각각의 가중치(W_f, W_i, W_o)를 가지고 있음



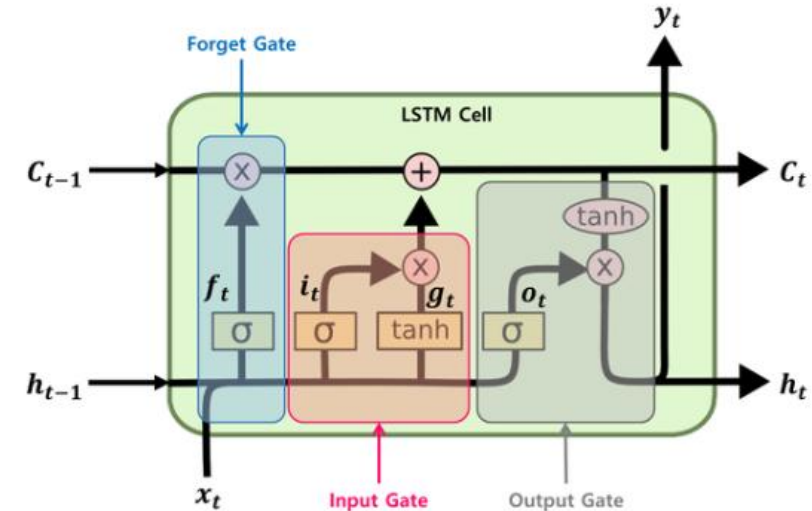
✓ LSTM - 게이트(Gate)

- 장기 기억 통로일 뿐인 “셀 상태(Cell State)”를 실질적으로 컨트롤하는 중요한 역할
- 어떤 정보를 기억/망각/사용할지를 ‘스스로 학습’하여 결정
- 게이트의 작동 원리
 - 모든 게이트는 시그모이드 함수(Sigmoid Function)를 활용 (0 ~ 1 사이의 값으로 바꾸기)
 - 0: 게이트를 완전히 닫아 정보를 차단
 - 1: 게이트를 완전히 열어 모든 정보를 통과
 - 0.5: 게이트를 절반만 열어 정보의 50%만 통과 (0.5 외의 0 ~ 1 사이의 모든 값 가능)
 - 시그모이드를 활용하는 이유: “흐름을 조절하기에 적합한 범위를 출력”
 - (이전 시점의 단기 기억(h_{t-1}) + 현재 시점의 새로운 정보(x_t)) * 각 게이트의 가중치의 값으로 게이트를 얼마나 열지를 결정



✓ LSTM - 게이트(Gate) - 망각 게이트(Forget Gate)

- “과거의 기억(장기 기억)에서 어떤 걸 지울까?” 를 결정하는 게이트
- 불필요한 장기 기억을 제거하는 역할
- 작동 방식
 1. 이전 시점의 은닉 상태 (h_{t-1}) 과 현재 시점의 입력 (x_t) 를 하나의 벡터로 합침
 2. 망각 게이트의 가중치(W_f) 와 생성한 벡터를 곱함
 - "현재 상황을 고려할 때, 장기 기억이 얼마나 중요한가?"를 평가하는 과정
 3. 시그모이드 함수를 통과시키면서 “망각 벡터”가 생성
 4. 그리고 이 값을 셀 상태(장기 기억) * 망각 벡터 을 통해 셀 상태(장기 기억)을 선택적으로 지움



✓ LSTM - 게이트(Gate) - 입력 게이트(Input Gate)

- “새로운 정보 중 어떤 것을 셀 상태(장기 기억)에 저장할까?” 를 결정하는 게이트
- 중요하다고 판단되는 정보만 선별하여 기억을 업데이트하는 역할
- 작동 방식

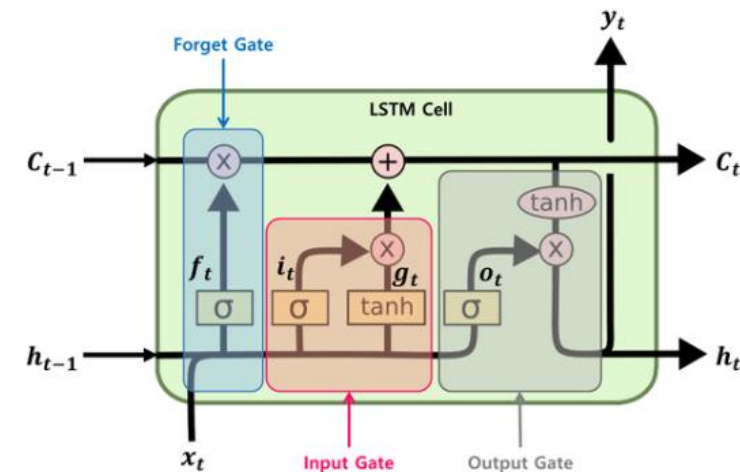
1. (동시진행) 업데이트 벡터 생성 (새로운 정보를 반영할 비율을 결정)

1. 이전 시점의 은닉 상태(h_{t-1}) 과 현재 시점의 입력(x_t)를 하나의 벡터로 합침
2. 입력 게이트의 가중치(W_i)와 생성한 벡터를 곱함
3. 시그모이드 함수를 통과시키면서 “업데이트 벡터”가 생성

1. (동시진행) 추가할 정보 생성 (어떤 새로운 정보를 넣을지)

1. 이전 시점의 은닉 상태(h_{t-1}) 과 현재 시점의 입력(x_t)를 하나의 벡터로 합침
2. 셀의 가중치(W_c)와 생성한 벡터를 곱함
3. tanh를 통과시키면서 “추가할 정보” (tanh: 내용과 방향을 표현하는데 적합, -1~1)

2. 그리고 이 값을 “추가할 정보” * “업데이트 벡터” 을 통해 셀 상태(장기 기억)에 업데이트



✓ LSTM - 게이트(Gate) - 출력 게이트(Output Gate)

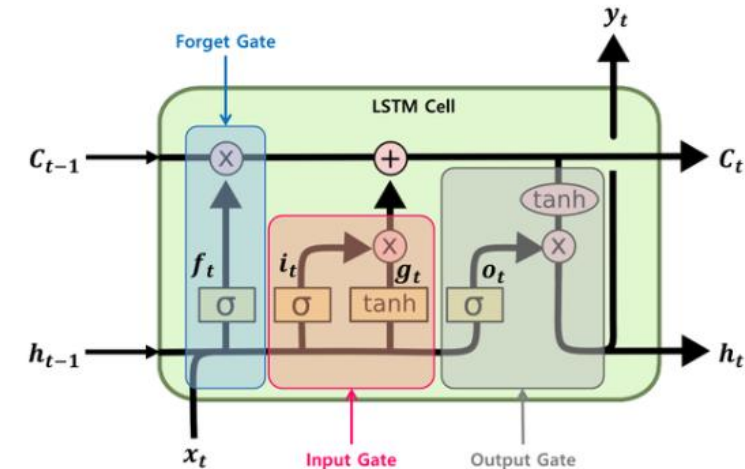
- 셀 상태(장기 기억)의 모든 정보 중에서 예측에 필요한 정보만 선별하여 외부로 내보내는 역할
- 셀의 최종 결과를 결정하는 역할
- 작동 방식

1. 출력 벡터(마스크) 생성 (정보 선별 마스크 역할)

1. 이전 시점의 은닉 상태(h_{t-1})와 현재 시점의 입력(x_t)를 하나의 벡터로 합침
2. 출력 게이트의 가중치(W_o)와 생성한 벡터를 곱함
3. 시그모이드 함수를 통과시키면서 “출력 벡터(마스크)”가 생성

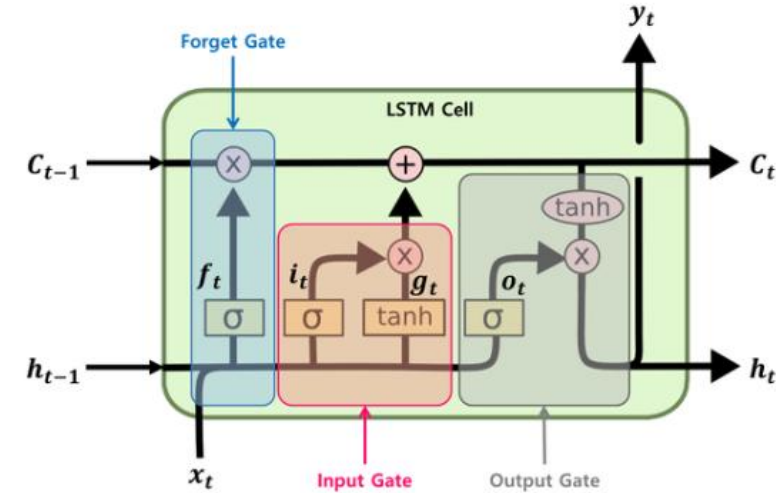
2. “셀 상태” * “출력 벡터(마스크)” 을 통해 중요한 데이터만 출력

- 출력 게이트는 “셀 상태”를 변경하지 않음



✓ LSTM - 학습 방식

- LSTM 학습 과정(오차를 최소로 하는 가중치(W_f, W_i, W_o, W_c)를 찾아가는 과정)
 - 순전파 → 오차 계산 (RNN과 동일하게 진행)
 - 역전파 (Backpropagation Through Time, BPTT)
 - 은닉 상태(단기 기억)와 셀 상태(장기 기억) 두 경로에 오차 정보가 전달 됨
 - 은닉 상태에 영향을 준 건 “출력 게이트(W_o)”이므로, 출력 게이트의 책임량을 구하면서 기존 RNN 처럼 역전파를 진행(기울기 소실 여전)
 - 남은 책임은 셀 상태로 가고, 셀 상태(장기 기억)에 영향을 준 건 “입력(W_i)”, “망각(W_f), 셀(W_c) 게이트” 이므로, 거슬러 올라 가면서 “입력 게이트”, “망각 게이트”, “셀 게이트” 의 책임량을 구하면서 역전파 진행
 - 중요) 셀 상태는 복잡한 활성화 함수를 거치지 않았고, 단순 덧셈과 곱셈의 연산만 진행했기 때문에, 미분을 해도 값이 “1”이 되면서, 오차값이 손실없이 계속 전달 됨
 - 이로 인해 기울기 소실 문제가 해결



✓ LSTM 의 한계

- 고정된 입출력문제 / 병렬 연산불가 / 여전한 기울기소실
 - LSTM은 하나의 cell 을 재사용하고 있고, 그렇기 때문에 입력 길이는 얼마든지 달라도 상관 없음
 - 대신, Cell이 1개 이기 때문에, 입력 값에 대응하는 출력값도 1개가 나오게 됨
 - 고로 입력 값을 3개를 넣으면, 3개의 값만 출력할 수 있음
 - “나는 학생이다” 를 번역하게 된다면,
“나는” => I, “학생이다” => am 까지만 출력하게 되고 끝이 나게 됨
- 이 문제를 해결한 방식이 바로 Seq2Seq
 - (인코더) 입력 받는 LSTM
 - (디코더) 출력하는 LSTM
 - “입력받는 역할(인코더)”과 “출력하는 역할(디코더)”를 나눠서 유연하게 대처할 수 있도록 함

8. 이미지 파운데이션 모델

• INDEX

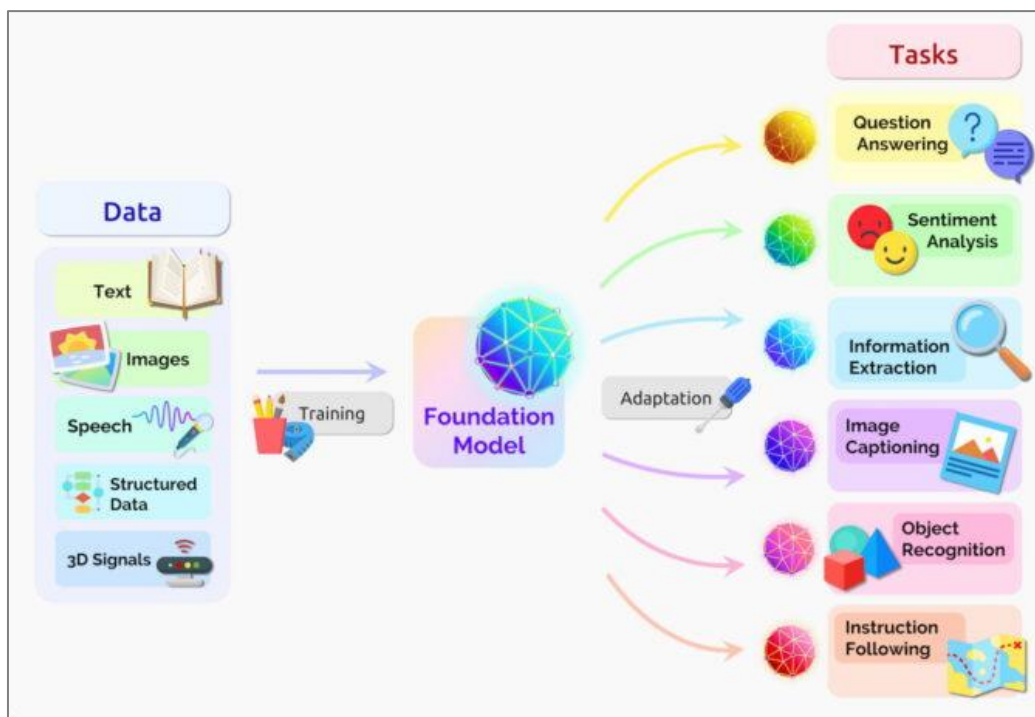
- 파운데이션 모델
- 이미지 생성 모델의 진화
 - GAN
 - Diffusion
 - Latent Diffusion Model
- 멀티 모달
 - CLIP
 - Stable Diffusion
- 지식 증류
 - SANA

파운데이션 모델

✓ 딥러닝 패러다임 변화

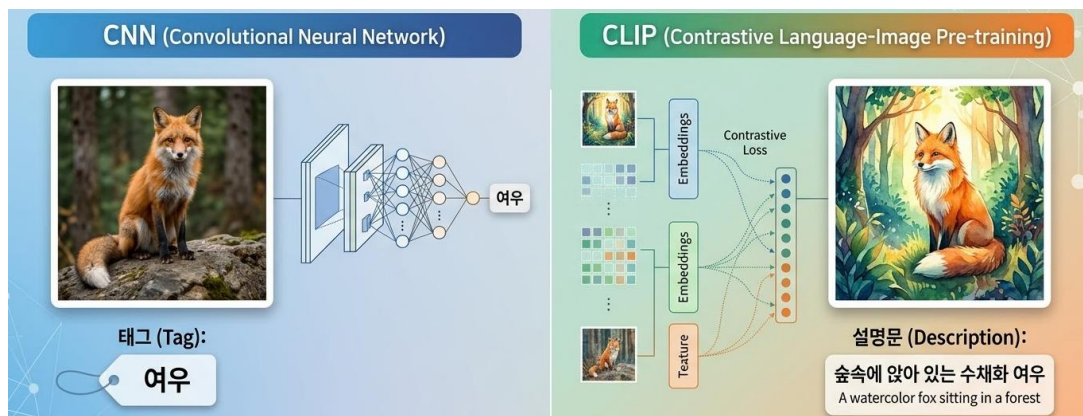
- (과거) 개별 태스크(분류, 검출)마다 모델을 처음부터 따로 학습
- (현재) 방대한 데이터로 사전학습된 하나의 모델을 다양한 용도로 재사용

➤ 파운데이션 모델(Foundation Model)



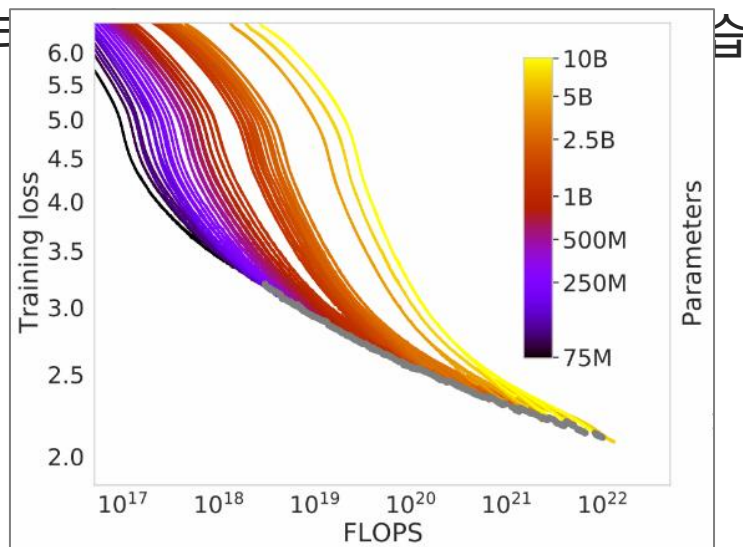
✓ ResNet(CNN) vs Foundation Model(CLIP)

- CNN(ResNet)
 - ImageNet이라는 데이터셋의 1000개 클래스에 대해서만 정답을 맞히도록 훈련되어 있음
 - 따로 배운 개념이 없다면, 사진을 보고 단순 '여우' 라는 결과만 도출
- Foundation Model(CLIP)
 - 4억 개의 이미지-텍스트 쌍을 대조 학습하여, 이미지 속의 사물뿐만 아니라 화풍, 분위기, 동작 등도 함께 학습함
 - 이미지를 설명하는 문장 전체를 하나의 벡터로 이해
 - 예를 들어, "우주에서 서핑하는 고양이" 라는 문장을 "우주", "서핑", "고양이" 개념을 조합해 분류할 수 있음



✓ 파운데이션 모델은 어떻게 강력해졌을까?

- 스케일의 규모
 - 웹상의 수억 개의 이미지와 텍스트를 학습
 - 규모의 법칙: 많이 가르치고, 복잡한 모델일 수록 똑똑하다.
- 자기 지도 학습
 - 레이블이 없는 데이터도, 데이터 자체의 상관관계를 통해 스스로 학습
 - 이미지와 설명 텍스트가 서로 어울려

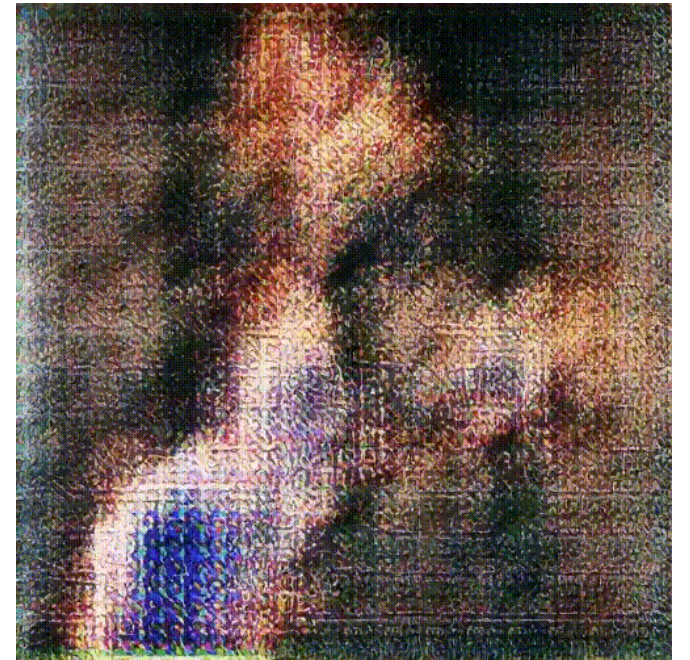


이미지 생성 모델의 진화

GAN

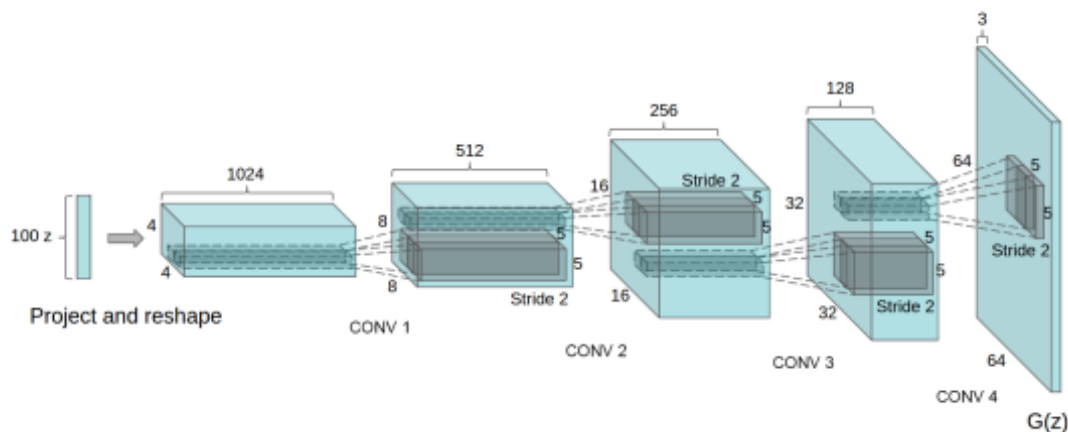
✓ 1세대 - GAN 의 등장

- ✓ 2014년 등장! 적대적 생성 신경망(GAN, Generative Adversarial Network)
- ✓ 현세대 파운데이션 모델로 넘어가기 전, 어떻게 이미지를 만들었는 지를 설명하는 중요한 역할
- ✓ 두 개의 신경망이 서로 경쟁하면서 이미지를 생성
 - ✓ 생성자(Generator)는 “위조범 ” 처럼 이미지를 생성
 - ✓ 판별자(Discriminator)는 “경찰 ” 처럼 진위를 가려냄
- ✓ 시간이 지날수록 위조범은 더 잘 속이게 되고, 완벽한 위작을 만드는 것을 목표로 학습하게 함



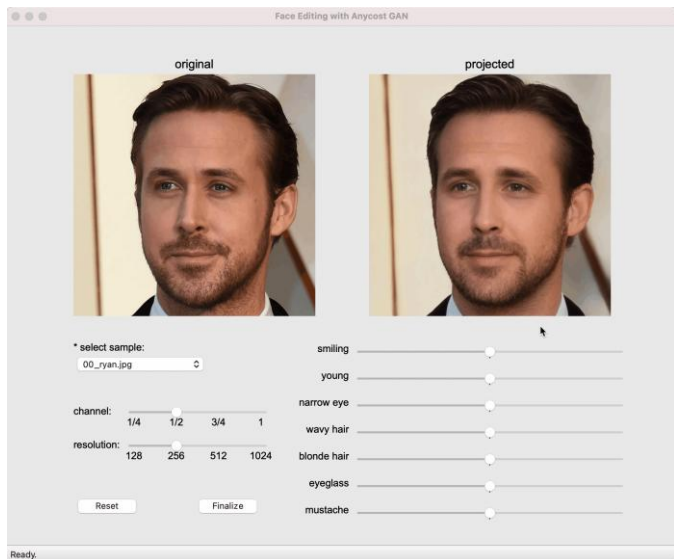
✓ 1세대 - GAN 의 동작 방식

1. 입력: 생성하는 랜덤한 숫자 벡터를 입력받음 (보통 100차원의 벡터)
2. 생성: 입력받은 벡터를 기반으로 이미지를 생성
 - FCL 통과 ($4 \times 4 \times 512$) → 업샘플링1 ($8 \times 8 \times 256$) → 업샘플링2 ($16 \times 16 \times 128$) (→ ... → $64 \times 64 \times 3$ 이미지 완성)
3. 생성된 이미지에 대해서 생성자의 오차와 판별자의 오차를 구하고, 이를 기반으로 가중치를 업데이트 함



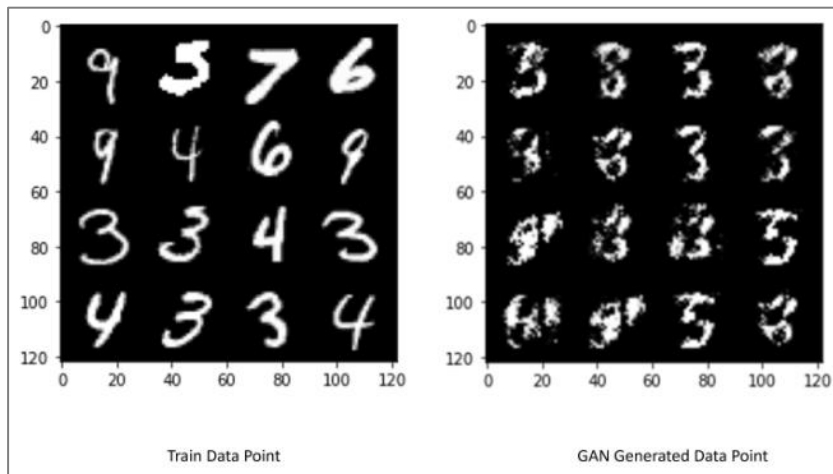
✓ 1세대 - GAN 의 특징

- ✓ 선명하고 사실적인 결과물
- ✓ 생성자가 이미지를 만드는 재료인 '무작위 노이즈'를 조금씩 바꾸면, 이미지도 자연스럽게 바뀜
 - => 처음에 받는 무작위 노이즈가 이미지가 어떤 모습을 가질지 결정하는 역할을 함
 - => 이미지의 특징을 제어할 수 있는 가능성을 열었음



✓ 1세대 - GAN 의 한계

- ✓ 혁신적이었지만... “경찰” 은 너무 똑똑했다.
- ✓ 생성자와 판별자, 두 모델의 균형을 맞추기가 너무 어려웠고, 한 쪽이 똑똑해지면 학습이 제대로 이루어지지 않는 ‘**모드 붕괴**’ 현상이 발생
- ✓ 모드 붕괴
 - ✓ 학습이 진행될수록 생성자가 판별자를 속이기 가장 쉬운, 몇 가지 종류의 이미지만을 반복적으로 생성



Diffusion Model

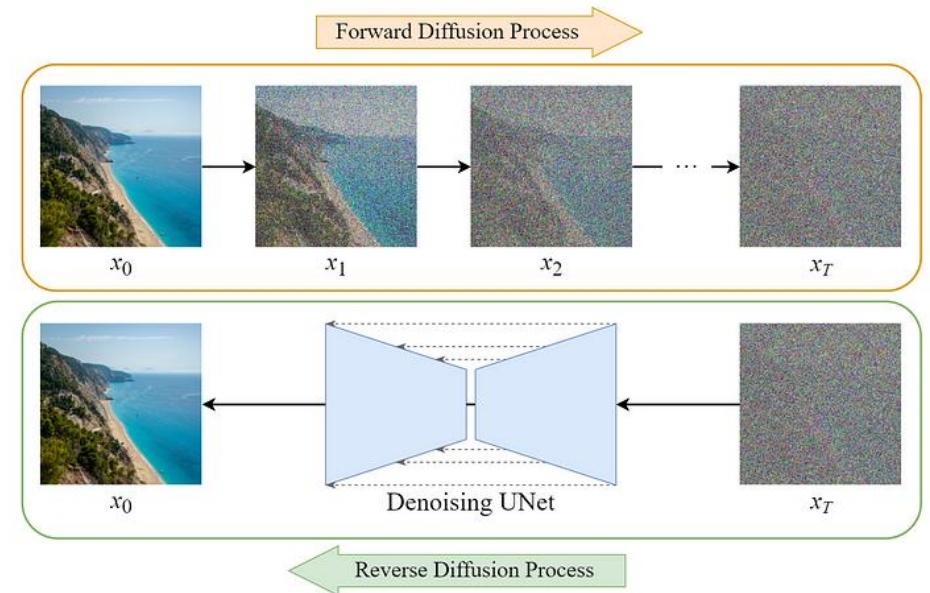
✓ 2세대 - Diffusion 모델의 등장

- ✓ 2020년 등장한 이미지 파운데이션 모델 이론
- ✓ **노이즈** (원래 형태를 완전히 잃은 이미지) -> 추가된 노이즈를 예측하고, 노이즈를 제거 -> 선명한 이미지 탄생
- ✓ 학습 과정에서 특정 이미지가 망가지는 과정을 학습하고, 이 내용을 기반으로 노이즈를 제거
- ✓ 두 모델이 싸우는 방식(GAN)이 아니라, 노이즈를 예측하는 명확한 수학적 목표를 갖기 때문에 학습이 안정적



✓ 2세대 - Diffusion 모델의 동작 과정

- 순방향 과정(Forward Process, 이미지 파괴)
 - 이미지에 아주 미세한 노이즈를 수천 번에 걸쳐 조금씩 더함
 - 최종적으로 원본의 형태를 완전히 잃어버린 '가우시안 노이즈(정적)' 상태가 됨
- 역방향 과정(Reverse Process, 이미지 복원)
 - 완전한 노이즈(가우시안 노이즈)에서 시작
 - “노이즈 예측기”가 예측한 노이즈를 살짝 걷어냄
 - 위 과정에서 우리가 넣은 노이즈와 예측한 노이즈의 차이를 비교
 - 이 차이를 줄이는 방향으로 가중치를 수정
 - 위 과정을 주어진 횟수에 맞게 반복
 - 단계가 너무 적으면, 형태가 무너짐
 - 단계가 너무 많으면, 정교해지지만 시간이 너무 오래 걸림



✓ 2세대 - Diffusion 모델의 특징

- ✓ '노이즈를 예측하고 제거한다' 는 목표는 안정적이고, 학습에 실패할 확률을 줄임
- ✓ 단계적으로 이미지를 만들어가는 방식 덕분에, "모드 붕괴" 없이 다채로운 결과물을 생성
- ✓ 이미지의 노이즈를 제거하는 각 단계마다 '사용자가 작성한 텍스트' 를 참고하여 훌륭한 결과물을 생성
- ✓ Midjourney, Dall-E, Stable Diffusion 와 같은 이미지 생성 모델의 뿌리



✓ 2세대 - Diffusion 모델 한계

1. 일관성 부족

- ✓ 같은 캐릭터로 다른 포즈나 이미지를 만들려고 하면, 매번 얼굴이나 스타일이 미묘 or 완전히 달라짐
- ✓ 이유: '기억' 의 개념이 없기 때문에, 출발점이 조금씩 달라지면 결과물이 완전히 달라짐

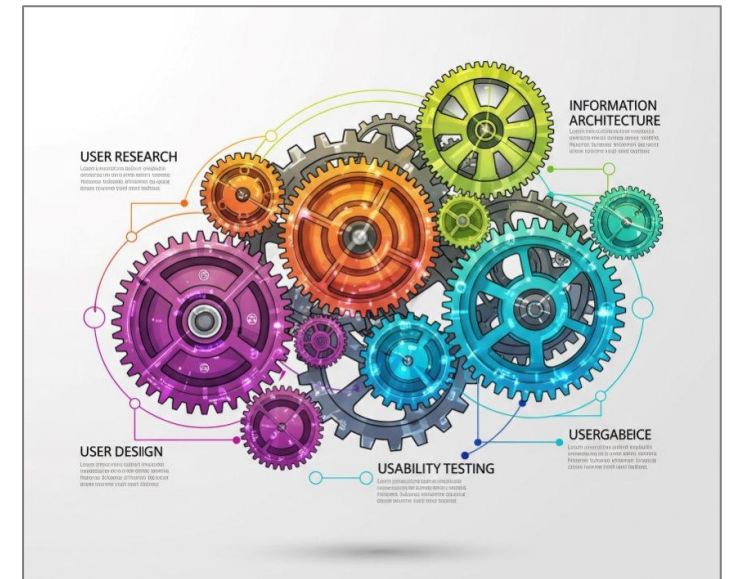


2. 문맥 이해 부족

- ✓ “소파아래, 고양이 옆에 있는 파란 공”같은 텍스트를 치면, ‘소파, 고양이, 파란 공’ 요소는 그리지만 위치가 형편 없음
- ✓ 텍스트를 이해하는 능력이 부족해서, 전체적인 주제만 파악

3. 느린 속도와 비용

- ✓ 생성 과정이 순차적으로 노이즈를 제거하는 과정



Latent Diffusion Model

✓ Diffusion Model은 너무 속도가 느렸다!

- 이미지의 모든 픽셀에서 직접 노이즈를 계산해야해서 계산량이 너무 많음
 - 이미지를 있는 그대로 다루지 말고, **핵심 특징**만 뽑아낸 “요약본(Latent)” 으로 작업하자!
- Latent Diffusion Model
 - 이미지를 “픽셀 공간”에서 “잠재 공간(Latent Space)”으로 옮겨와서 작업을 수행
 - 픽셀 공간: 우리가 눈으로 보는 실제 이미지 공간(매우 크고 복잡)
 - 잠재 공간(Latent Space): 이미지의 핵심 특징(형태, 색상, 구도)만 압축해 놓은 추상적인 공간 (작고 효율)
 - “픽셀 대신 잠재 공간에서 처리하자”라는 기술적 원리이며 이를 통해 구현한 모델이 **“Stable Diffusion”**

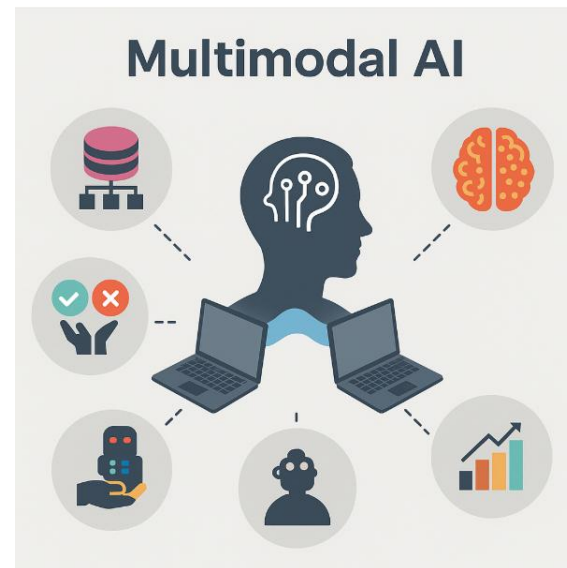
✓ LDM(Latent Diffusion Model)의 동작 과정

1. **Encoder**(압축기)를 활용해 거대한 이미지($512 * 512 * 3$)를 아주 작은 잠재 벡터($64 * 64 * 3$)로 압축
 - Convolution을 반복하여 특징을 뽑아냄
2. 잠재 벡터의 노이즈를 섞고 지우는 작업을 진행
3. **Decoder**(복원기)를 활용해 선명한 이미지($512 * 512 * 3$)로 복원
 - 업샘플링 기술을 활용하여 압축 과정에서 잃어버렸던 세부 픽셀값들을 다시 추측해서 채워 넣음

멀티모달

✓ 멀티모달이란?

- 텍스트, 이미지, 오디오, 비디오 등 서로 다른 형태의 데이터를 함께 이해하고 처리하는 AI 기술
- AI도 인간처럼 세상을 입체적으로 이해하기 위해서는 멀티모달 지능이 필수
- 과거
 - 이미지 모델은 “이 이미지는 고양이입니다” 라는 레이블만 학습
 - 텍스트 모델은 이미지 없이 글자만 학습
- 현재
 - 이미지와 이미지의 “설명”을 통째로 학습시켜, 이미지와 언어를 하나의 개념으로 묶음
- 이러한 멀티모달 시대를 연 주인공, 이미지 생성 AI의 중요한 역할을 하는 모델
 - CLIP(Contrastive Language-Image Pre-training)



CLIP

✓ CLIP(Contrastive Language-Image Pre-training)

- 2021년 OpenAI가 발표한 인공지능 모델 (모델명: clip-vit-base-patch32)
- WIT(WebImage Text)라는 4억개의 이미지-텍스트 쌍을 기반으로 학습
 - 사람이 일일이 정답을 매긴 게 아니라, 사람들이 인터넷에 올린 자연스러운 설명을 학습함
 - 그로 인해 “수채화 풍”, “사이버펑크” 와 같은 추상적인 묘사까지 모두 이해할 수 있음
- “대조 학습”을 기반으로 학습을 진행
 - 이미지와 텍스트를 나란히 두고 서로 맞는지 비교하며 학습
 - **Contrastive** Language-Image Pre-training

✓ CLIP 동작 방식

1. 이미지 인코더와 텍스트 인코더가 각각 사진과 글을 벡터로 변환
 - 변환 후에는 이미지와 텍스트가 비교 가능한 형태가 되며, 좌표값이 된다고 생각하면 됨
2. 만들어진 모든 이미지 벡터와 텍스트 벡터를 각각 쌍으로 보면서 정답이면 가깝게, 오답이면 멀게 설정
 - 이미지3장, 텍스트 3장이 있다면 총 9가지의 경우의 수를 살펴보며 서로 거리를 조절
 - “코사인유사도”라는 방법을 통해 거리를 조절
3. 2번 과정을 4억번(데이터 수) 반복하면, 관련 있는 내용들끼리 서로 모여있는 **거대한 잠재 공간**이 만들어 짐
 - 거대한 잠재 공간을 활용해서 “사과 사진” 이미지 근처에는 “사과” 단어도 있기 때문에 근처에 있는 설명이나 단어를 활용해서 결과를 추출함
 - 이로 인해 학습 때 본 적 없는 새로운 문장(“우주에서 서핑하는 고양이”)를 가져와서 각각의 개념을 조합할 수 있음
(Zero-shot Learning)

Stable Diffusion

✓ Stable Diffusion

- LDM(Latent Diffusion Model) + CLIP(Contrastive Language-Image Pre-training)
- 언어 해석 및 잠재 공간 생성(CLIP) -> 압축 및 복원 (LDM)
- LDM 구조 덕분에 슈퍼컴퓨터가 아닌 일반 PC에서도 이미지 생성이 가능해짐
- 모델의 구조와 가중치를 공개하여, 이를 기반으로 수많은 변형 모델들이 탄생함
- <https://github.com/compvis/stable-diffusion>

Stable Diffusion

Stable Diffusion was made possible thanks to a collaboration with [Stability AI](#) and [Runway](#) and builds upon our previous work:

[High-Resolution Image Synthesis with Latent Diffusion Models](#)

[Robin Rombach*](#), [Andreas Blattmann*](#), [Dominik Lorenz](#), [Patrick Esser](#), [Björn Ommer](#)

[CVPR '22 Oral](#) | [GitHub](#) | [arXiv](#) | [Project page](#)



[Stable Diffusion](#) is a latent text-to-image diffusion model. Thanks to a generous compute donation from [Stability AI](#) and support from [LAION](#), we were able to train a Latent Diffusion Model on 512x512 images from a subset of the [LAION-5B](#) database. Similar to Google's [Imagen](#), this model uses a frozen CLIP ViT-L/14 text encoder to condition the model on text prompts. With its 860M UNet and 123M text encoder, the model is relatively lightweight and runs on a GPU with at least 10GB VRAM. See [this section](#) below and the [model card](#).

✓ Stable Diffusion의 현실적인 한계

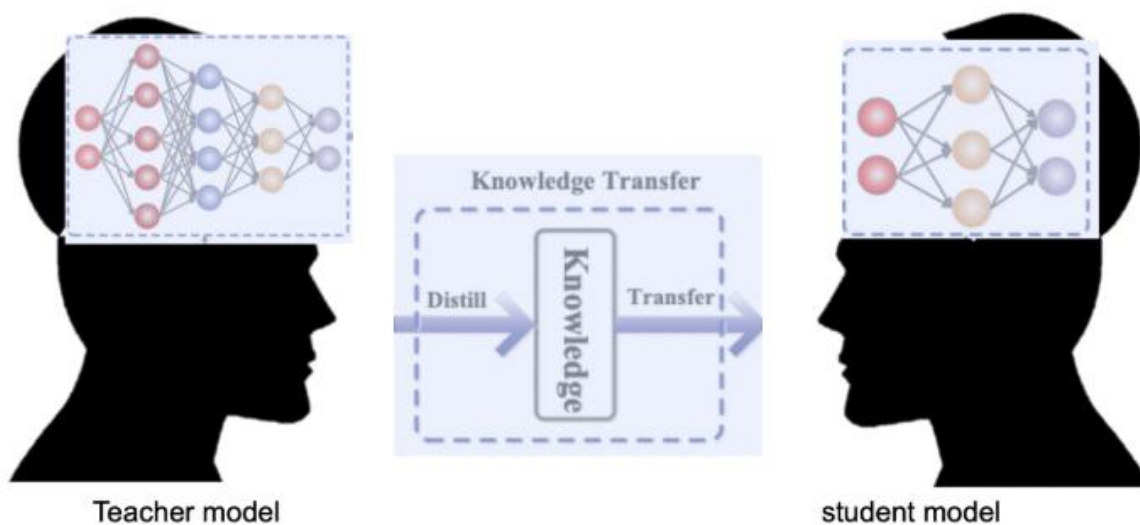
- 추론 속도(Latency)
 - 20~50단계의 반복 연산이 필요하여 실시간 생성에는 여전히 느린 속도
- 의미론적 오류
 - 복잡한 문장의 형용사와 명사를 혼동하는 경우가 잦음
- 구조적 결함
 - 인간의 손가락 개수, 복잡한 문자 표시 등 세밀한 논리 구조에서 오류가 발생
- 높은 하드웨어 성능

➤ 거대 모델의 지능은 그대로 가져오고, 크기를 줄일 수는 없을까????

지식증류

✓ 지식 증류(Knowledge Distillation)

- 방대한 지식을 작고 가벼운 모델에게 효율적으로 전달하는 모델 압축 기술
- 단순히 결과값만 전달하는 것이 아니라, 문제를 해결하는 사고방식을 가르치는 것
- Teacher Model의 확률 분포를 Student Model 이 학습함으로써, 적은 파라미터로도 정교한 판단 능력을 배움



✓ 지식 증류를 하는 이유

- 경량화
 - 모델의 크기를 수십 배까지 줄일 수 있음
 - 모바일이나 웹 브라우저에서도 구동 가능
- 속도
 - 연산량이 적어 추론 속도가 빨라짐
 - 실시간 이미지 생성 및 서비스 운영 비용을 절감할 수 있음
- 성능 유지
 - 크기는 작아졌지만, 거대 모델의 지능을 상당 부분 유지

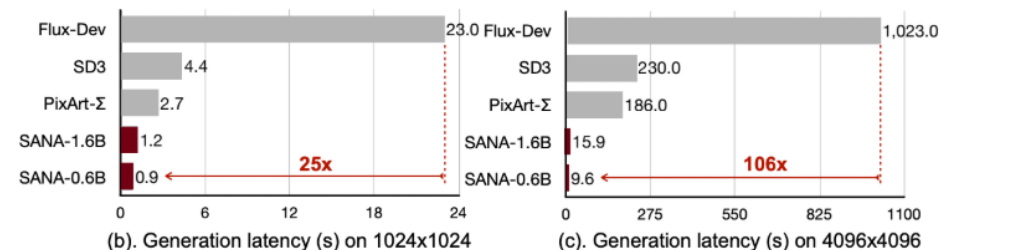
SANA

✓ SANA

- Efficient High-Resolution Image Synthesis with Linear Diffusion Transformers
- LDM+CLIP+지식증류
- 엔비디아에서 개발하여 배포한 모델
- 아주 적은 컴퓨팅 자원으로도 4K 해상도 이미지를 빠르게 만들 수 있음
- <https://github.com/NVlabs/Sana>



 Docs | [SANA](#) | [SANA-1.5](#) | [SANA-Sprint](#) | [SANA-Video](#) | [Demo](#) |  [HuggingFace](#)
[ComfyUI](#) | [SGLang](#) | [Cosmos-RL](#)



9. 토큰화와 임베딩, GPT

• INDEX

- 토큰화와 임베딩
 - 토큰화
 - 임베딩
 - 원핫 인코딩
 - Word2Vec
 - BERT
 - GPT

토큰화와 임베딩

✓ 일상에서 주어진 문장을 컴퓨터에게 그대로 넘기면 어떻게 될까?

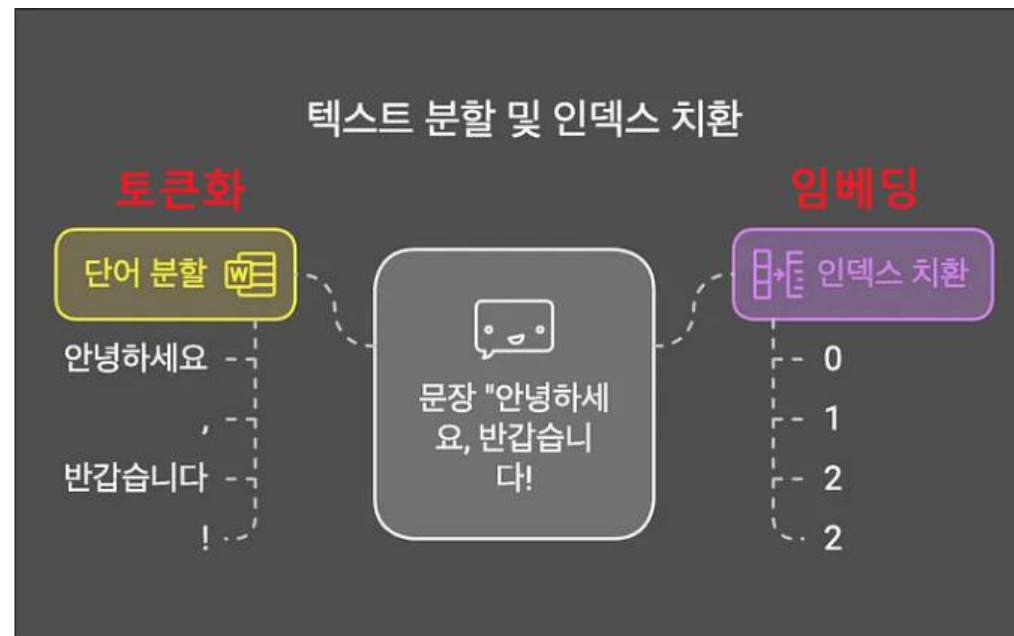
- 컴퓨터는 인간의 문장을 이해할 수 없다!



- ✓ 인간의 언어를 별도의 처리 없이 컴퓨터에게 건네면, 사람의 방식과는 다르게 문장을 인식
 - 같은 의미의 단어를 완전히 다른 데이터로 인식
 - “사과”와 “사과는”을 다른 단어로 인식
 - 단어 하나하나를 컴퓨터에게 외우게 하는 것은 너무 많은 시간과 메모리가 필요
 - 단어간의 관계를 인지하지 못함
 - “좋다”와 “나쁘다”가 날씨를 나타내는 반대말이라는 사실을 알기 어려움
 - 데이터 사이의 연결을 인지하지 못함

✓ 그러면 어떻게 사람의 언어를 컴퓨터가 이해할 수 있는 언어로 바꿀 수 있을까?

- 1단계 (토큰화)
 - 문장을 의미 있는 최소 단위(토큰)로 나누기
- 2단계 (임베딩)
 - 나누어진 각 토큰에 고유한 숫자를 붙여주기



토큰화

✓ 토큰화(Tokenization)

- 문장을 “토큰” 이라는 단위로 나누는 것
- 컴퓨터에게 숫자(단어가 바뀐 값)를 전달하기 전에 진행하는 전처리
- 어떻게 토큰을 나누느냐에 따라서 성능이 천차만별로 달라짐
- **대표적인 토큰화 방식**
 - 단어 단위 토큰화(Word-level)
 - 문자 단위 토큰화(Character-level)
 - 서브워드 토큰화(Subword-level)

✓ 단어 단위 토큰화(Word-level)

- 가장 직관적인 방법으로, 주로 공백을 기준으로 나눔
- 각 토큰이 명확한 의미를 가진다는 장점이 있지만, **OOV(Out-of-Vocabulary)** 문제가 발생

Sample Data:

"This is tokenizing."

Character Level

[T] [h] [i] [s] [i] [s] [t] [o] [k] [e] [n] [i] [z] [i] [n] [g] [.]

Word Level

[This] [is] [tokenizing] [.]

Subword Level

[This] [is] [token] [izing] [.]

✓ OOV(Out-Of-Vocabulary)

- AI 모델이 아무리 열심히 학습하더라도 세상 모든 단어를 학습할 수는 없음
- 이 때, 학습하지 않은 단어가 등장하는 문제가 OOV(단어 집합 외 단어)
- 모르는 단어가 나온다면, 특수 토큰인 [UNK]로 치환해서 보여주는 방식으로 OOV를 해결
- 하지만 아래와 같은 이유로 위의 방식은 대안이 될 수 없음
 - **의미 손실**: 가장 중요한 단어가 UNK가 된다면 문장 전체의 의미를 파악할 수 없음
 - **신조어 대응 불가**: "두쫌쿠"와 같은 신조어를 거의 OOV가 됨

✓ 문자 단위 토큰화(Character-level)

- 모든 문장을 글자 하나씩 나누는 것
- 세상 모든 문자를 조합할 수 있으므로 모르는 단어가 생기지 않음
- 단, 토큰 하나가 아무런 의미를 갖지 못하며, 문장이 너무 길어져서 컴퓨터가 처리하기 어려움

Sample Data:

"This is tokenizing."

Character Level

[T] [h] [i] [s] [i] [s] [t] [o] [k] [e] [n] [i] [z] [i] [n] [g] [.]

Word Level

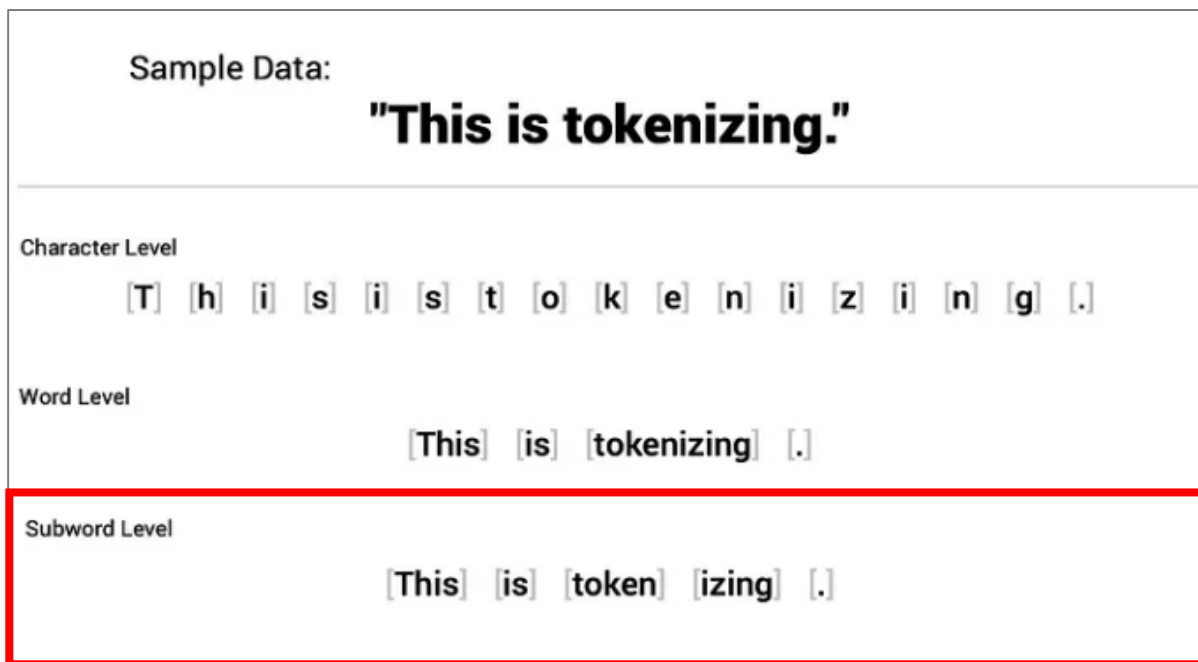
[This] [is] [tokenizing] [.]

Subword Level

[This] [is] [token] [izing] [.]

✓ 서브워드 토큰화(subword-level)

- 자주 쓰이는 단어는 그대로 두고, 드물게 쓰이는 단어는 의미 있는 단위로 쪼개는 것
- 모든 단어를 외우지 못해도, 모든 단어의 “부품 ” 을 외우고, 처음 보는 단어도 조합해서 읽는 방식
- 이러한 방식은 구현하는 대표적인 알고리즘이 “BPE(Byte Pair Encoding)”

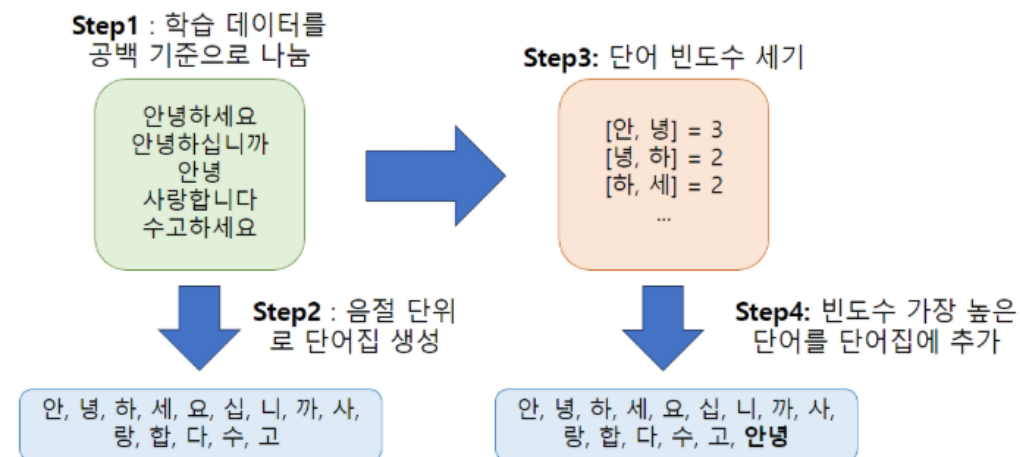


✓ BPE (Byte Pair Encoding)

- 주어진 문장을 서브워드 토큰으로 나누는 알고리즘
- 아무리 생소한 단어라도 결국 글자 단위까지 쪼개지므로 OOV 문제를 해결
- 무한정에 가까운 단어를 다 외우지 않고, 수만 개의 서브워드만으로 세상의 모든 문장을 표현
- 신조어 및 오타에 대한 대응이 훌륭함
 - 웃겨요, dntruzzz, 웃기네 과 같은 단어를 “웃기” 라는 공통 서브워드로 묶어서 처리

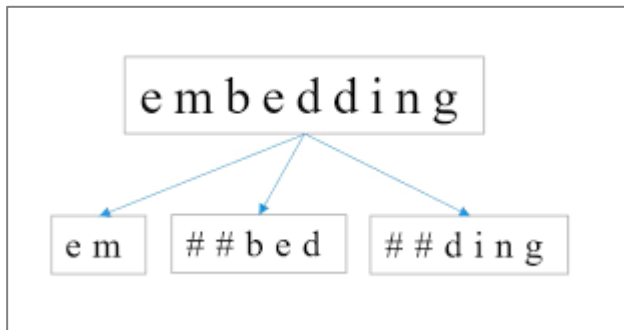
✓ BPE (Byte Pair Encoding) 동작 방식

1. 모든 단어를 문자 단위로 쪼개기
2. 데이터 전체에서 가장 자주 붙어서 나오는 문자 쌍을 찾기
3. 가장 많이 나온 쌍을 하나의 새로운 토큰으로 합치기
4. 정해진 사전 크기에 도달할 때까지 2~3번을 반복



✓ WordPiece

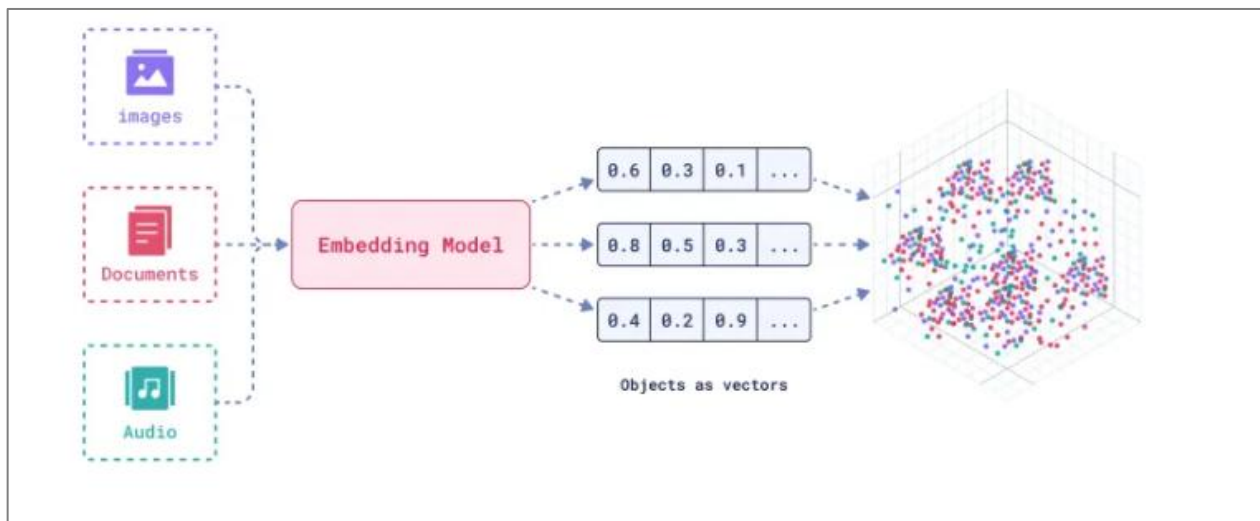
- “두 문자를 합쳤을 때, 전체 데이터를 얼마나 더 잘 설명할 수 있는가?” 를 계산
- BPE는 횃수를 체크하지만, WordPiece 는 A, B가 따로 나올 확률에 비해, 붙어 나올 확률이 높은지를 계산
- 단순 많이 나오는 것보다는 서로 자주 붙어나오는 경우가 높은 경우를 토큰으로 설정
- 토큰에 붙는 단어들은 ‘##’이 붙음



임베딩

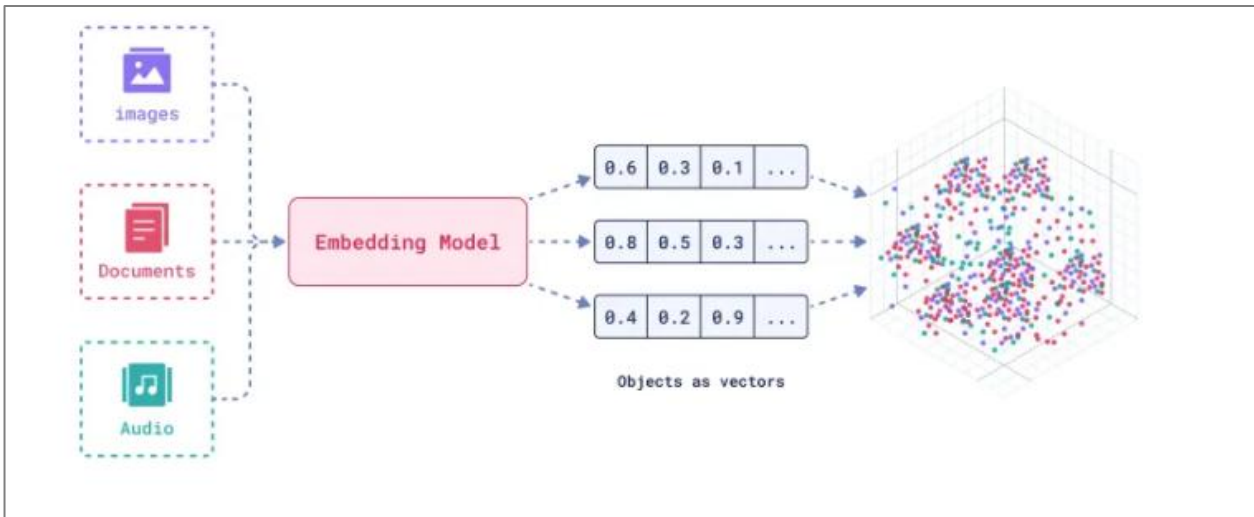
✓ 단어를 나누면 끝일까?

- 단어를 나눴으니, 단어에 그냥 적절한 숫자를 할당해서 컴퓨터에게 넘기면 될까?
➤ No!
- 적절한 의미를 부여한 숫자를 건네야 한다.



✓ 임베딩

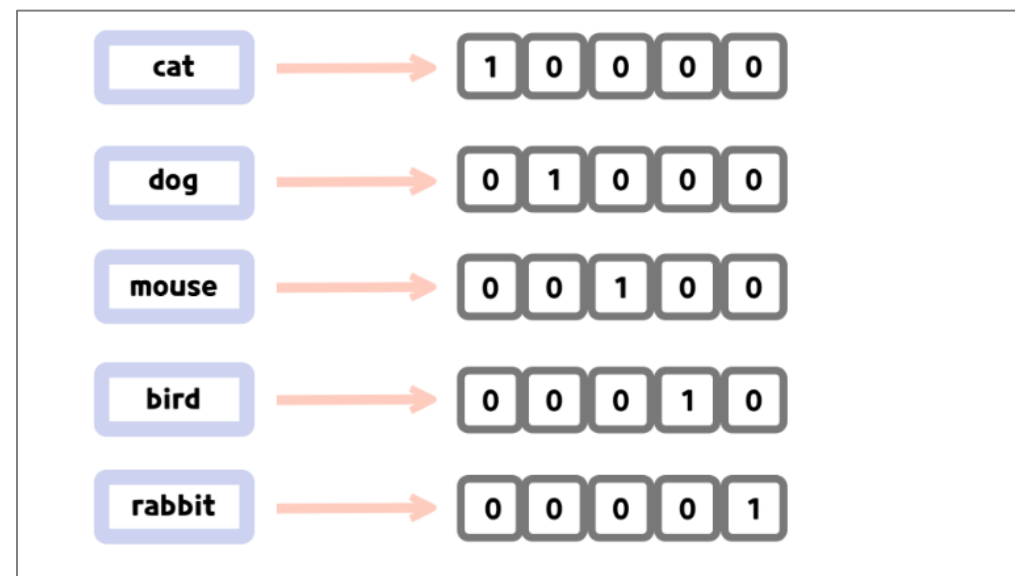
- 컴퓨터가 이해할 수 있도록 단어를 다차원 공간상의 좌표(벡터)로 바꾸는 기술
- 단순한 숫자로 바꾸는 것이 아니라, 비슷한 의미를 가진 단어끼리 가깝게 모이도록 하는 것!
- 그러면 단어가 의미를 지니게 하는 방법은 어떤 게 있을까?



원핫 인코딩

✓ 원핫 인코딩(One-hot Encoding)

- 가지고 있는 단어(vocab, Corpus)의 수만큼 차원을 할당하고, 각 단어는 해당하는 차원만 1로 표현하는 것
- 아주 직관적이지만 치명적인 단점이 있음
- 치명적인 단점
 - 희소성과 메모리 낭비
 - 우리가 사용하는 단어가 5만 개라면,
단어 하나를 표현하기 위해 숫자 5만 개가 담긴 리스트가 필요함(희소 벡터)
 - 메모리를 엄청 잡아먹지만 정작 필요한 정보는 별로 없는 비효율적인 데이터
 - 단어 간의 관계를 모름
 - 모든 단어는 서로 독립적으로 존재



Word2Vec

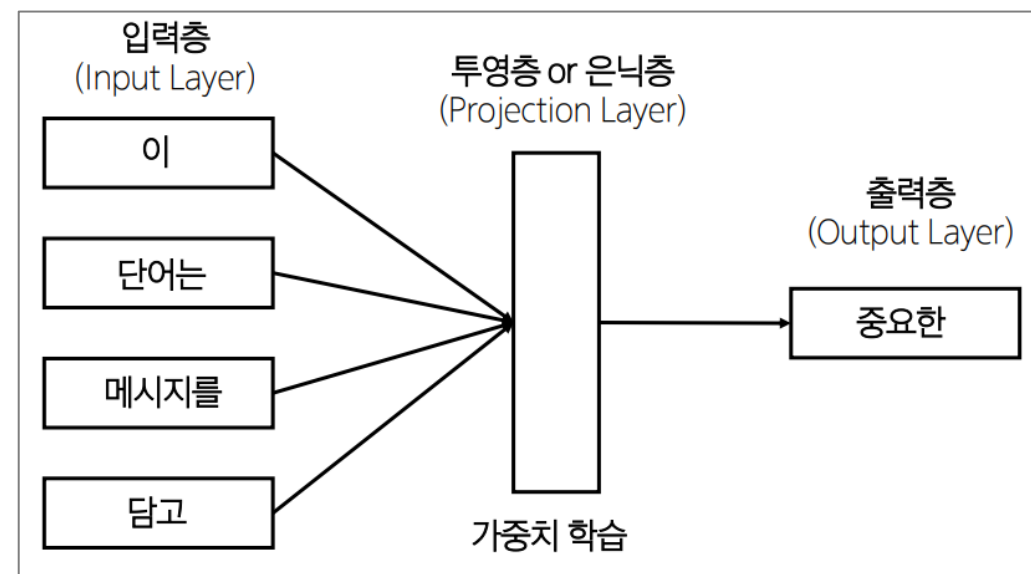
✓ Word2Vec

- 비슷한 분포에서 등장하는 단어들은 비슷한 벡터 위치에 놓인다는 원리를 활용
- 단어를 저차원 실수 벡터로 변환하여 문맥적 의미와 유사도를 반영하여 임베딩
- 학습 방식 종류
 - CBOW(continuous BoW)
 - 주변 단어들을 입력으로 받고, 그 중심 단어를 예측하는 방식
 - Skip-gram
 - 중심 단어를 입력으로 받고, 주변 단어들을 예측하는 방식

✓ CBOW(continuous BoW) (1/2)

- 주변에 있는 단어들을 보고, 가운데에 빠진 단어가 무엇인지 맞추는 과정을 진행하면서 학습
- 동작 방식
 - 입력층에 주변 단어가 들어가고, 주변 단어에 적절한 가중치가 곱해져서 특정 벡터를 꺼내고, 어떤 단어에 가장 가까운지 계산을 진행
 - 역전파를 통해 가중치를 구하고, 해당 가중치를 이용해서 단어들의 임베딩 벡터를 구할 수 있음

중심 단어 ↓ 주변 단어	중심 단어 벡터	주변 단어 벡터 (예시)
이 단어는 중요한 메시지를 담고 있다.	[1, 0, 0, 0, 0, 0]	[[0, 1, 0, 0, 0, 0], [0, 1, 0, 0, 0, 0]]
이 단어는 중요한 메시지를 담고 있다.	[0, 1, 0, 0, 0, 0]	[[1, 0, 0, 0, 0, 0], [0, 0, 1, 0, 0, 0], ...]
이 단어는 중요한 메시지를 담고 있다.	[0, 0, 1, 0, 0, 0]	[[1, 0, 0, 0, 0, 0], [0, 1, 0, 0, 0, 0], ...]
이 단어는 중요한 메시지를 담고 있다.	[0, 0, 0, 1, 0, 0]	[[0, 1, 0, 0, 0, 0], [0, 0, 1, 0, 0, 0], ...]
이 단어는 중요한 메시지를 담고 있다.	[0, 0, 0, 0, 1, 0]	[[0, 0, 1, 0, 0, 0], [0, 0, 0, 1, 0, 0], ...]
...		

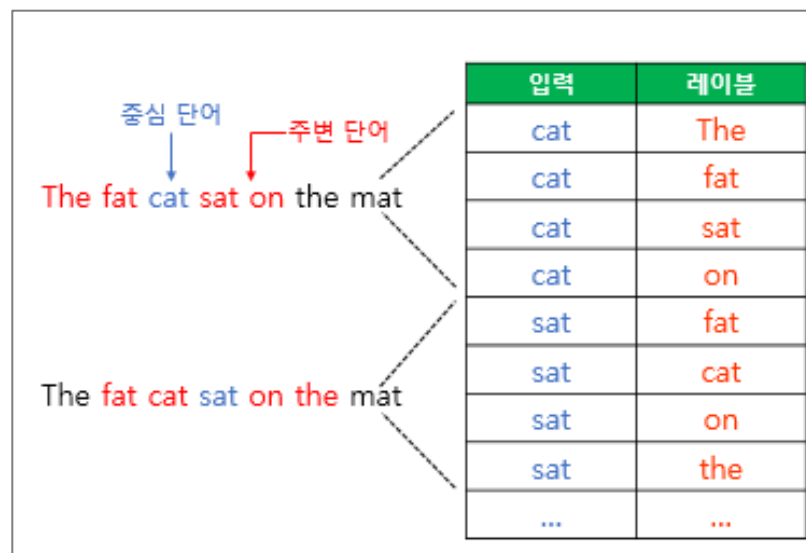
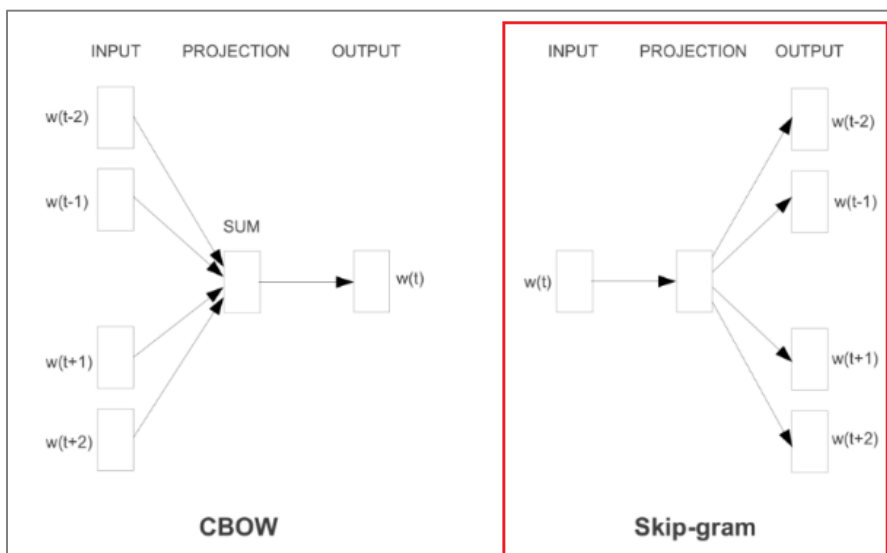


✓ CBOW(continuous BoW) (2/2)

- (장점) 학습 속도가 매우 빠름
- (장점) 빈번한 단어 예측에 유리
 - 자주 등장하는 단어들은 주변 맥락이 풍부하기 때문에, 안정적인 벡터값을 얻을 수 있음
- (장점) 주변 단어의 평균을 활용하기 때문에 데이터의 노이즈에 비교적 강한 편
- (단점) 희귀 단어에 약함
 - 드물게 등장하는 중요한 단어의 정보가 자주 등장하는 평범한 단어들의 기운에 묻힘
- (단점) 문맥의 세부 정보 손실
 - 단어들의 순서를 무시하고 주변 단어를 보기 때문에, 미세한 의미 차이가 있음

✓ Skip-gram (1/2)

- 가운데 단어 하나를 보고 주변에 올 단어를 예측하는 모델
- CBOW와 반대로 동작하며, 실전에서는 skip-gram이 더 많이 쓰임
- 중심 단어를 보고 주변 단어를 맞출 때, 전체 단어장(V)에 있는 모든 단어(예: 10만 개)에 대해 확률을 계산 (CBOW도 마찬가지)



✓ Skip-gram (2/2)

- (장점) 단어간의 복잡한 관계를 더 잘 표현
 - 단어 하나하나가 메인으로 관계를 해석하기 때문
 - CBOW와의 차이점: 주변 단어들을 평균내서 하나의 벡터를 만드는 CBOW와는 다르게 각 단어가 고유하게 해석 됨
- (장점) 희귀 단어도 임베딩을 잘 함
 - 희귀 단어를 맞추기 위해서 가중치를 크게 조정해야 하고, 그로 인해 희귀 단어의 위치도 비교적 잘 찾아냄
 - CBOW와의 차이점: 주변 단어 중 하나가 희귀한 단어여도, 자주 나오는 단어들과 함께 평균 내어지면 존재감이 사라짐
- (단점) 연산량이 많고, 학습 속도가 느림
 - 한 단어당 양쪽 주변(2K)를 살펴봐야하기 때문
 - CBOW와의 차이점: 일반적으로 10배정도 느림

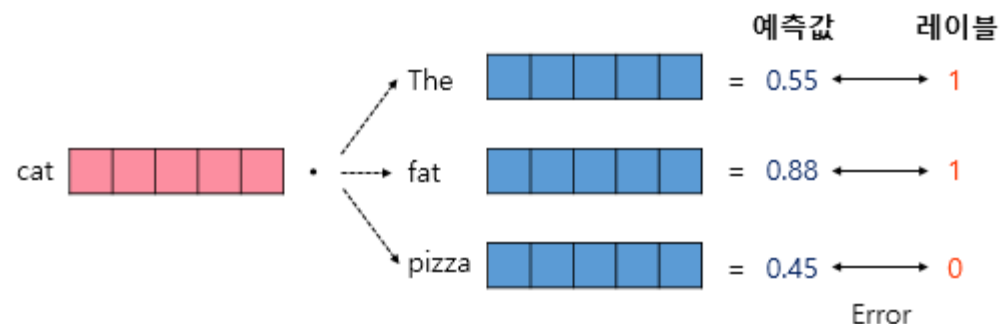
✓ Skip-gram with Negative Sampling(SGNS) (1/2)

- 학습 과정에서 전체 단어 집합이 아니라 일부 단어 집합에만 집중할 수 있도록 하는 방법
- 즉, 랜덤한 단어 집합을 가져와서 랜덤한 단어들이 나올 확률과 주변 단어들이 나올 확률을 기반으로 학습

Negative Sampling

입력1	입력2	레이블
cat	The	1
cat	fat	1
cat	pizza	0
cat	computer	0
cat	sat	1
cat	on	1

단어 집합에서 랜덤으로
선택된 단어들을
레이블 0의 샘플로 추가.



✓ Skip-gram with Negative Sampling(SGNS) (2/2)

- (장점) 학습 속도가 빠름
 - 단어장이 10만개일 때, 10만 번 연산했어야 하는 것을 적은 수(5~20)의 연산으로 줄여버림
- (장점) 임베딩 품질 유지
- (단점) 샘플링 편향
 - 운이 나쁘면 중요한 단어를 학습할 기회를 놓칠 수 있음 (희귀단어가 랜덤으로 선택되지 않아서)
- (단점) 하이퍼 파라미터 지정
 - 오답을 몇 개 뽑을지(K)를 정하는 게 번거로움

✓ Word2Vec의 한계

- **동음이의어**를 잘 처리하지 못함
 - Ex) “나는 배가 고파서 배를 먹었다“
 - “배“ 라는 단어를 같은 단어로 인식하게 됨
- 주변 단어들을 확인하기 때문에 **문단이나 전체를 보지 못함**
- 위 문제를 해결하기 위해서 “문장 전체를 양방향으로 읽고, 상황에 따라 벡터가 변하는 모델 ” 을 만듦
 - **BERT**(Bidirectional Encoder Representations from Transformers)
- ❖ **그러면 Word2Vec은 안쓰어나요??**
 - 앞으로 배울 BERT는 엄청나게 무겁기 때문에 가벼운 텍스트 분류기나 키워드 추출기에는 Word2Vec을 많이 사용
 - 특히 추천 시스템에 많이 사용

BERT

✓ BERT(Bidirectional Encoder Representations from Transformers) (1/2)

- 문장 전체를 양방향으로 읽고, 상황에 따라 벡터가 변하는 모델
- Attention 과 Transformer 를 활용해 전체의 상황을 파악하고, 중요한 부분에만 집중하는 원리로 동작
- 구글 검색 엔진 등 다양한 분야에서 활용되고 있음
- (장점) 문맥을 잘 이해하고, 문장 전체의 구조를 훨씬 깊이있게 파악함
- (단점) 너무 느리며, 긴 글은 잘 읽지 못함
- (한계) 문맥을 이해하는 건 좋았지만, 새로운 글을 창작하는 능력은 부족

✓ BERT(Bidirectional Encoder Representations from Transformers) (2/2)

- 구글이 엄청난 데이터와 위키피디아로 기본 학습을 시켜 놨음
- 가져와서 적절히 전이학습만 진행한다면 전문가급 성능을 낼 수 있음
- BERT 임베딩 모델을 활용하기 위해서 BERT에 맞게 쪼개는 토큰라이저도 같이 활용

```
from tokenizers import BertWordPieceTokenizer

tokenizer = BertWordPieceTokenizer(
    lowercase=False,
    strip_accents=False,
)
tokenizer
```

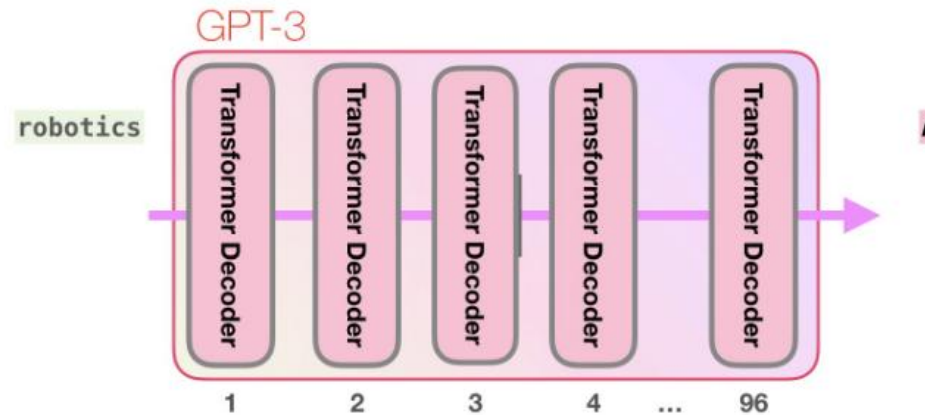
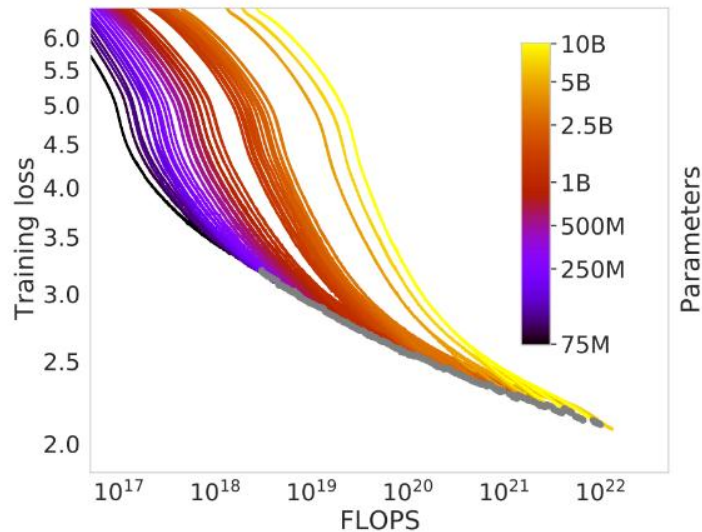
```
from transformers import AutoTokenizer, AutoModelForMaskedLM

model_name = "bert-base-cased"
tokenizer = AutoTokenizer.from_pretrained(model_name)
model = AutoModelForMaskedLM.from_pretrained(model_name).to(device)
```


GPT

✓ GPT-3(Generative Pre-trained Transformer, 2020)

- 모델 크기 엄청 키우고 많은 데이터를 학습시켜서 탄생한 **LLM**
- 스케일링의 법칙
- BERT 와의 파라미터 비교 : BERT(3.4억), GPT-3(1,750억)



✓ GPT-3(Generative Pre-trained Transformer)

- 다음에 올 말을 자연스럽게 이어가는 게 목적인 모델
- 수 많은 데이터로 훈련되어 단어들이 고정된 벡터로 정해져 있음
- Decoder-Only (쓰기 특화)
- (장점) 길고 자연스러운 글쓰기가 가능하고, 범용성이 있음
- (단점) 실행 비용이 비쌈 / 환각 / 욕설이나 편견을 뱉음
- (한계) 지시 이행능력이 부족(자신이 하고 싶은 말만 함)
 - GPT-3의 목적이 “다음에 올 단어를 그럴싸하게 맞추기” 이기 때문
 - (분홍 코끼리 효과) “사과를 말하지마” 라고 하면, “사과 ” 라는 단어에 더 기울이게 됨(Attention)

✓ InstructGPT (ChatGPT-3.5, 2022)

- GPT-3의 가장 큰 문제인 창작에만 집중한 나머지, 질문과는 다른 답변을 하는 문제를 해결
- **RLHF**를 통해서 사람의 의도를 파악하고 그에 맞는 대답을 하도록 고안된 모델
- **RLHF**(인간 피드백 기반 강화학습, Reinforcement Learning from Human Feedback)
 - 1단계: 사람이 직접 질문에 대한 모범 답안을 작성해서 학습
 - 2단계: AI가 여러 개의 답변을 내놓으면, 사람이 어떤 답변이 더 좋은 지 순위를 매김
 - 3단계: 사람의 점수를 잘 받기 위해 AI가 스스로 답변 방식을 수정하면서 학습
- 이를 통해, 대화의 규칙을 학습
- **(한계) 추론의 한계, 할루시네이션**
 - 겉으로는 논리적으로 보이지만, 실제로는 확률적으로 가장 그럴듯한 단어를 나열하는 것에 불과함

✓ GPT-4(2023)

- 추론의 한계를 극복하기 위해서 논리적인 데이터를 엄청나게 학습시킨 모델(추론 능력 향상)
- '자기 기억'에만 의존하여 발생한 할루시네이션을 극복하기 위해 **RAG** 도입
- **RAG**(Retrieval-Augmented Generation, 검색 증강 생성)
 - 외부 데이터를 참고하는 응답하는 기술
 - 외부 데이터 참조 → 외부 데이터를 기반으로 답변 구성 → 답변
 - 최신 정보를 반영하고, 근거 중심의 답변을 통해서 할루시네이션을 비약적으로 줄임
- (한계) 글자로만 세상을 이해하기 때문에 아직 추론 능력이 부족

✓ GPT-4o (2024)

- 멀티모달(텍스트, 오디오, 이미지, 비디오)를 처리하며 추론 능력이 비약적으로 향상된 모델
- 글자, 소리, 이미지를 같은 벡터 공간에 할당
- 텍스트 문서 뿐만 아니라 이미지, 영상, 표까지 RAG의 대상이 되어 더욱 성능이 향상됨
- 실시간 통역, 감정 파악이 가능해짐
- (한계) 복잡한 코딩이나 수학 문제를 풀 때, 첫 단추를 잘못 끼우면 끝까지 틀린 답을 밀고 나감

✓ OpenAI o1(2024)

- GPT-4o 의 틀린 논리를 끝까지 밀고 나가는 문제를 깊은 추론으로 해결
- 문제를 받으면 스스로 가설을 세우고, 검토하고, 틀리면 수정하는 과정을 거친 뒤에 최종 답을 내놓음
- 어려운 수학, 코딩에서 GPT-4o 보다 압도적인 성능을 보임
- (한계) 너무 느리고 비쌈
- (한계) 논리적인 부분에 집중하다보니 감정적인 부분이 약함
- (한계) 코드만 완벽하게 짜고, 실제로 코드를 실행하거나 웹사이트를 만들지 못함

✓ GPT-5 (2025)

- GPT-4o의 감정적인 부분과 GPT-o1 의 논리적인 부분을 통합한 모델
- 단순히 대화하는 수준을 넘어 컴퓨터 화면을 직접 보고 제어하는 “에이전트” 능력이 탑재
- 다양한 최적화와 고도화로 비용도 더 싸짐
 - 양자화/필요한 파라미터만 활용 / AI 전용칩 개발 / 학습 데이터 퀄리티 향상
- (한계) 학습할 데이터가 없음
- (한계) 너무나 많은 에너지 소비
- (한계) Agent로 인한 책임의 소재
- (한계) 진짜 논리인가? 블랙박스

문제

1. 인공지능(AI), 머신러닝(ML), 딥러닝(DL)의 포함 관계를 수학적 기호로 올바르게 나타낸 것은 무엇입니까?

A. $DL = ML = AI$

B. $AI \subset ML \subset DL$

C. $ML \subset DL \subset AI$

D. $DL \subset ML \subset AI$

Q. 다음 중 결정계수(R^2)에 대한 설명으로 가장 적절하지 않은 것은?

1. 결정계수는 0과 1 사이의 값을 가지며, 1에 가까울수록 설명력이 높다.
2. 단순선형회귀분석에서 독립변수와 종속변수의 상관계수를 제곱하면 결정계수와 같다.
3. 독립변수의 개수가 증가하면 모델의 유의성과 관계없이 결정계수(R^2)는 증가하는 경향이 있다.
4. 결정계수가 0.9라면, 종속변수의 총 변동 중 90%가 오차에 의해 설명된다는 뜻이다.

잔차 제곱합(RSS)의 수학적 정의로 가장 올바른 것은 무엇입니까?

A. $\sum (y_i - \hat{y}_i)^2$

B. $\sum (y_i - \bar{y})^2$

C. $\sum (\hat{y}_i - \bar{y})^2$

D. $\sum |y_i - \hat{y}_i|$